Network Computing
(058172)

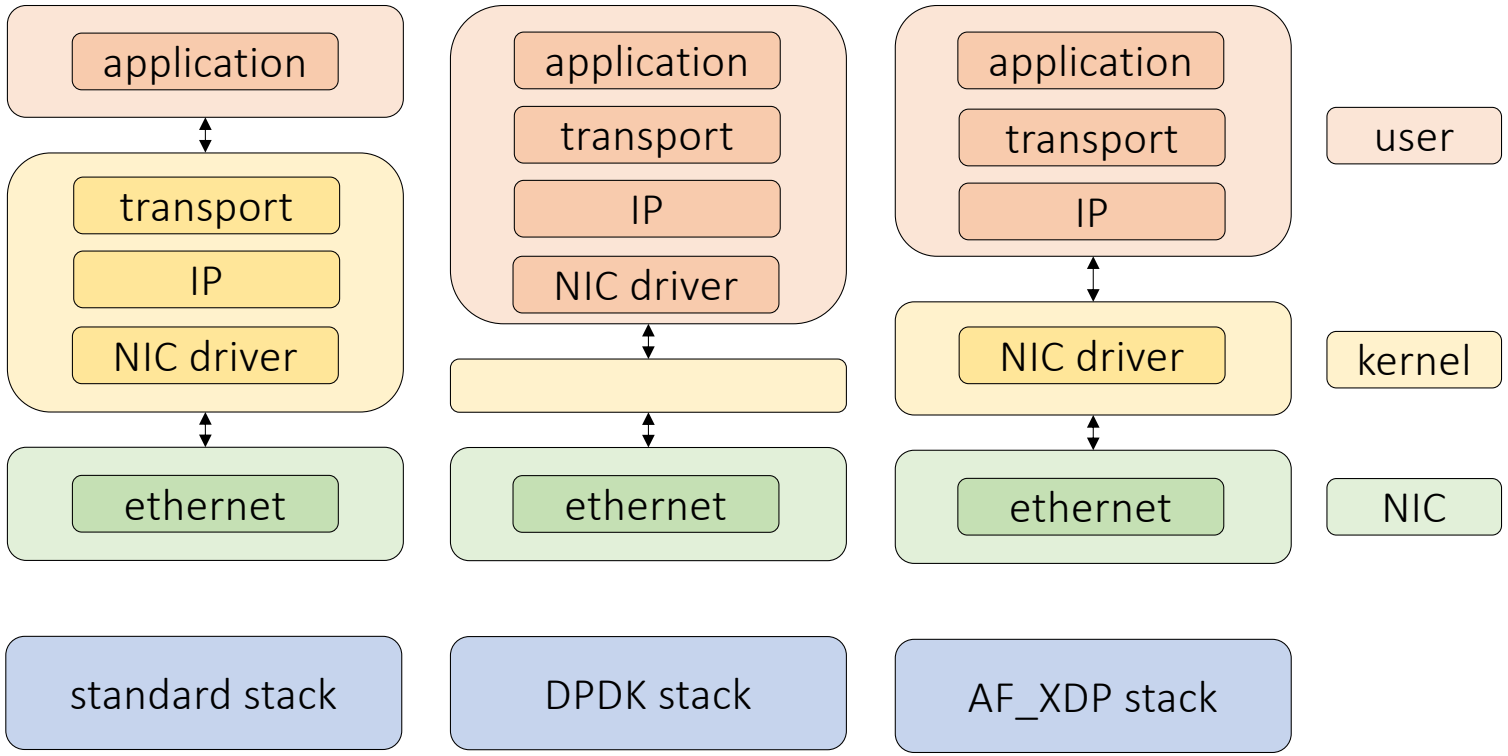
POLITECNICO
MILANO 1863

Remote Direct Memory Access (RDMA)

Content

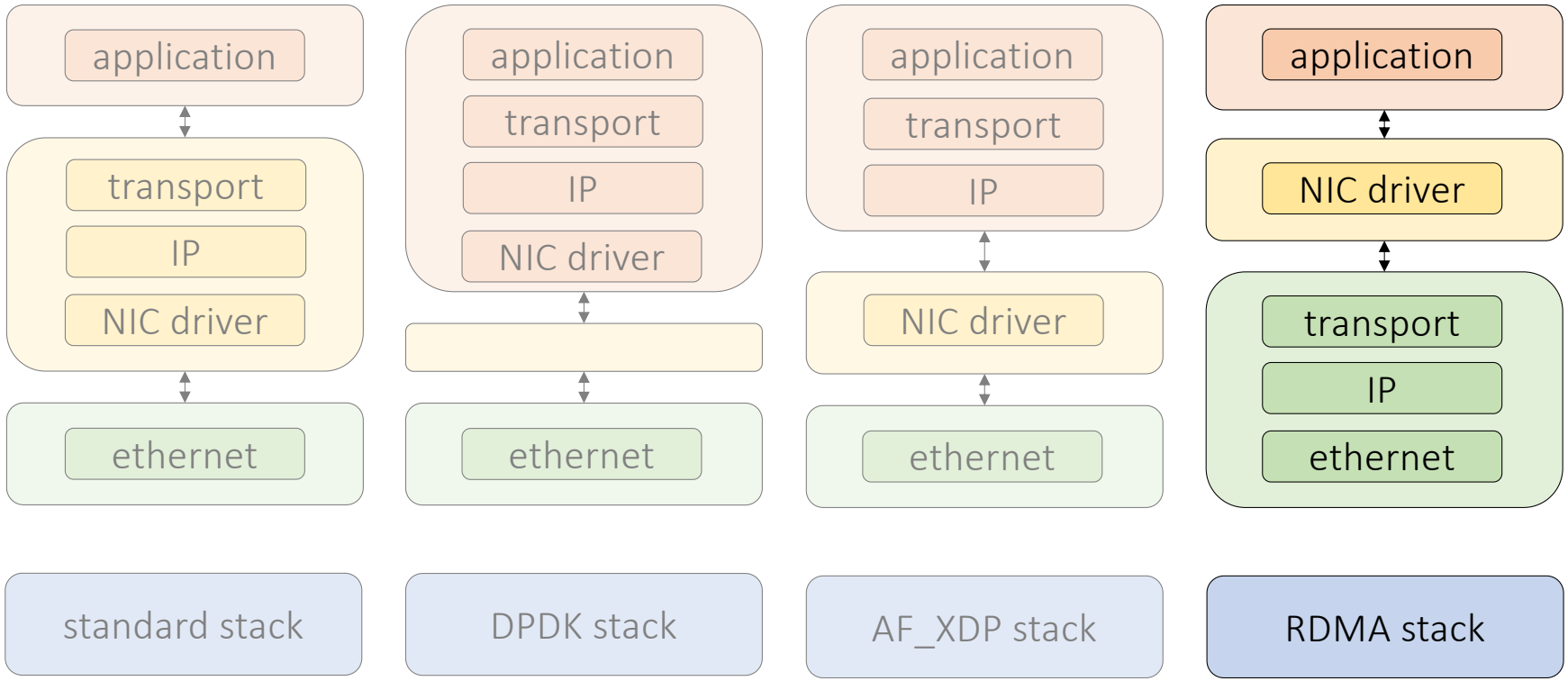
- RDMA basics
- Data Center Quantized Congestion Notification (DCQCN)
- RDMA in AI/ML datacenters

The stacks we have seen so far



RDMA vs other stacks

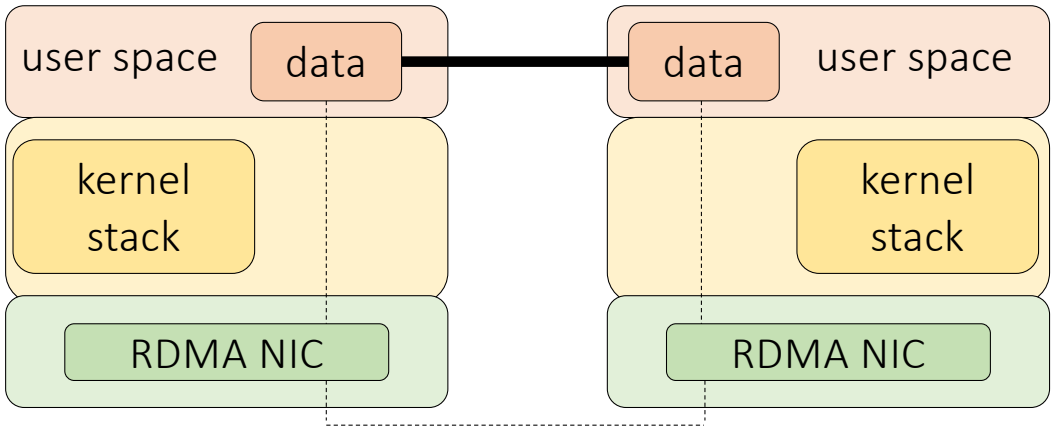
The NIC plays a key role in RDMA!



RDMA basics

- RDMA is a mechanism that allow to access memory on a remote system bypassing the kernel stack

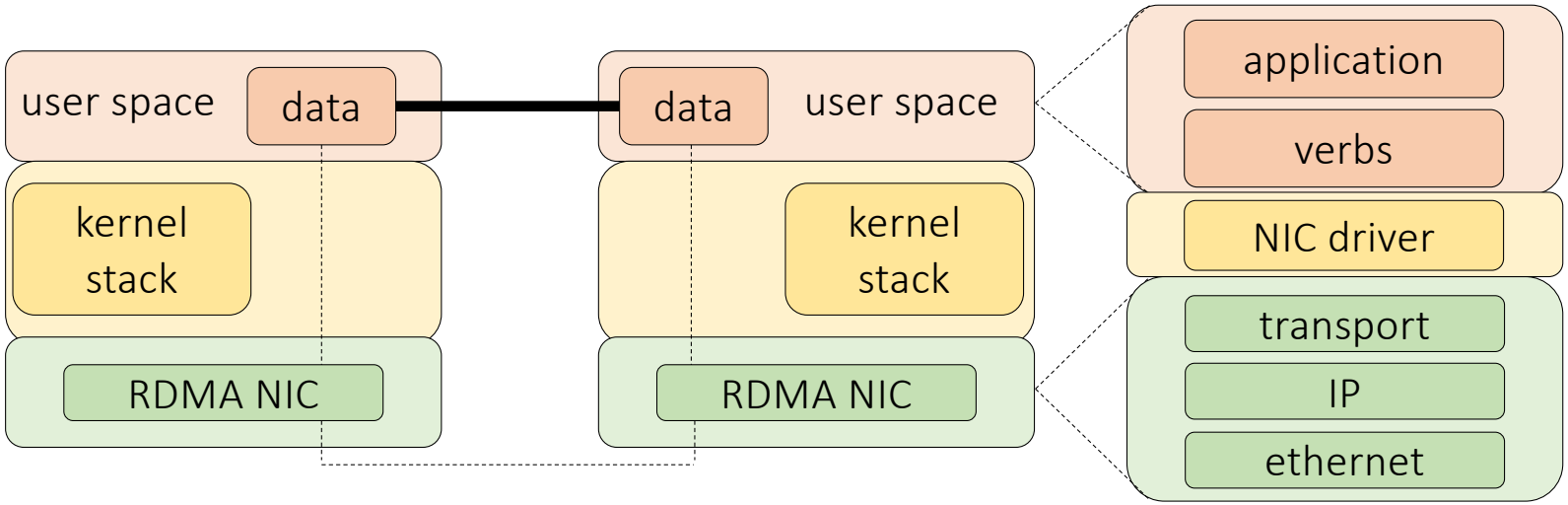
- RDMA can run over ethernet (RoCE v2) and it is encapsulated over IP/UDP



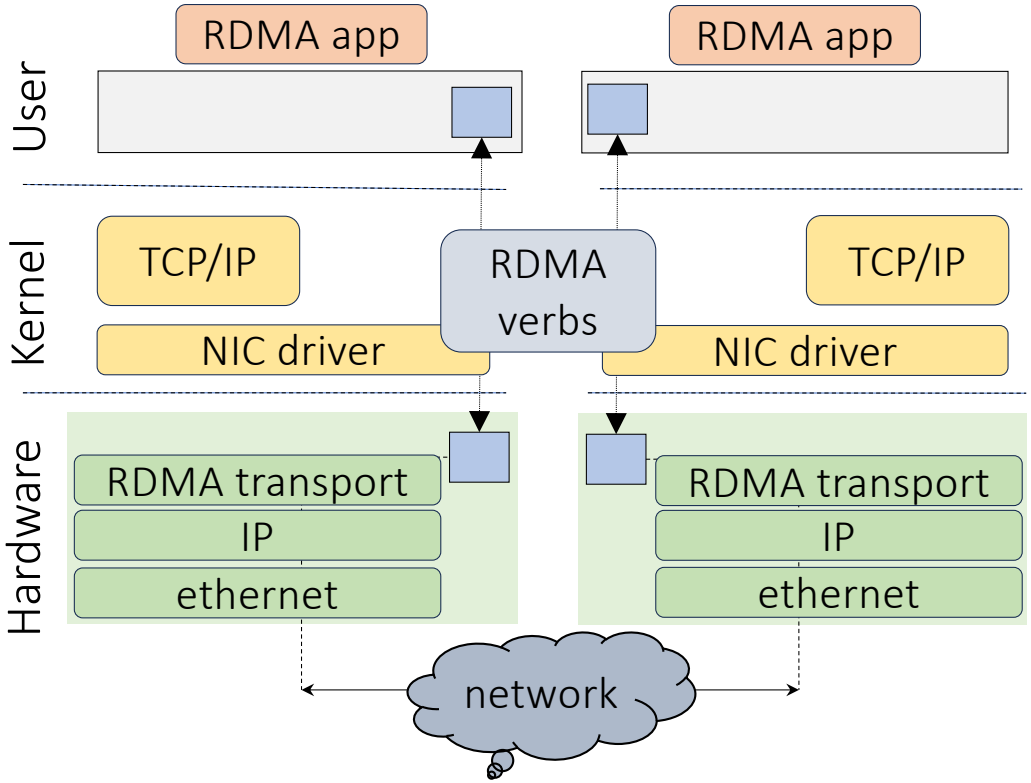
RDMA basics

- RDMA is a mechanism that allow to access memory on a remote system bypassing the kernel stack

- RDMA can run over ethernet (RoCE v2) and it is encapsulated over IP/UDP

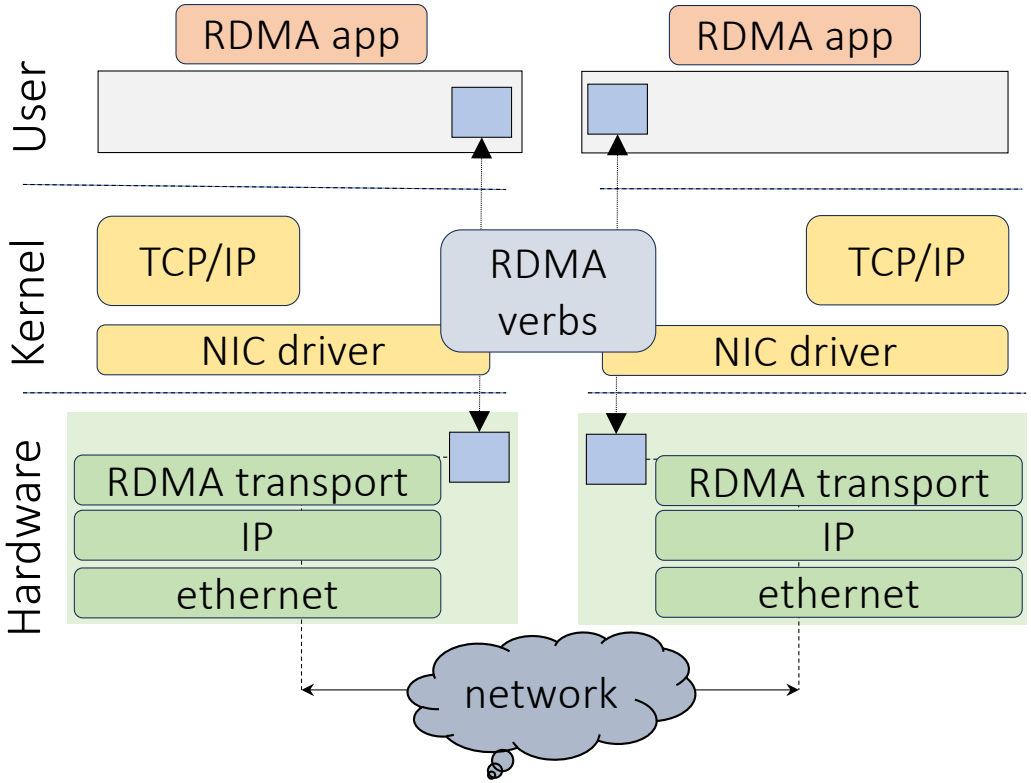


RDMA basics



Applications register memory regions with the NIC

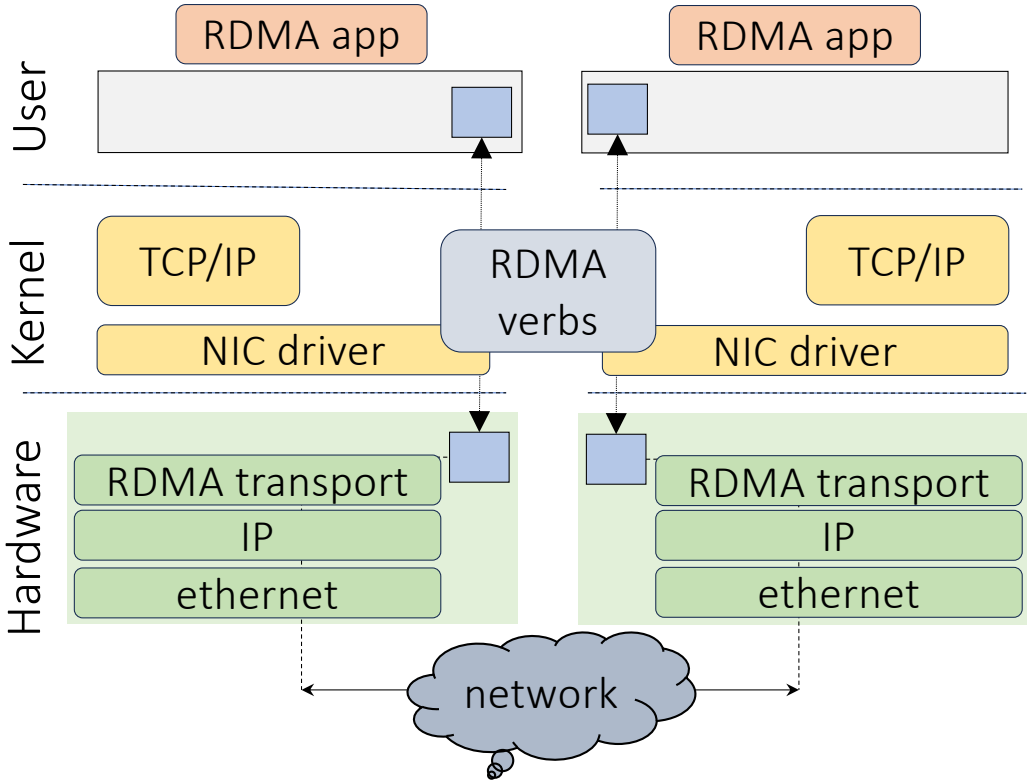
RDMA basics



Applications register memory regions with the NIC

The memory regions are used to store the data to be sent or received

RDMA basics

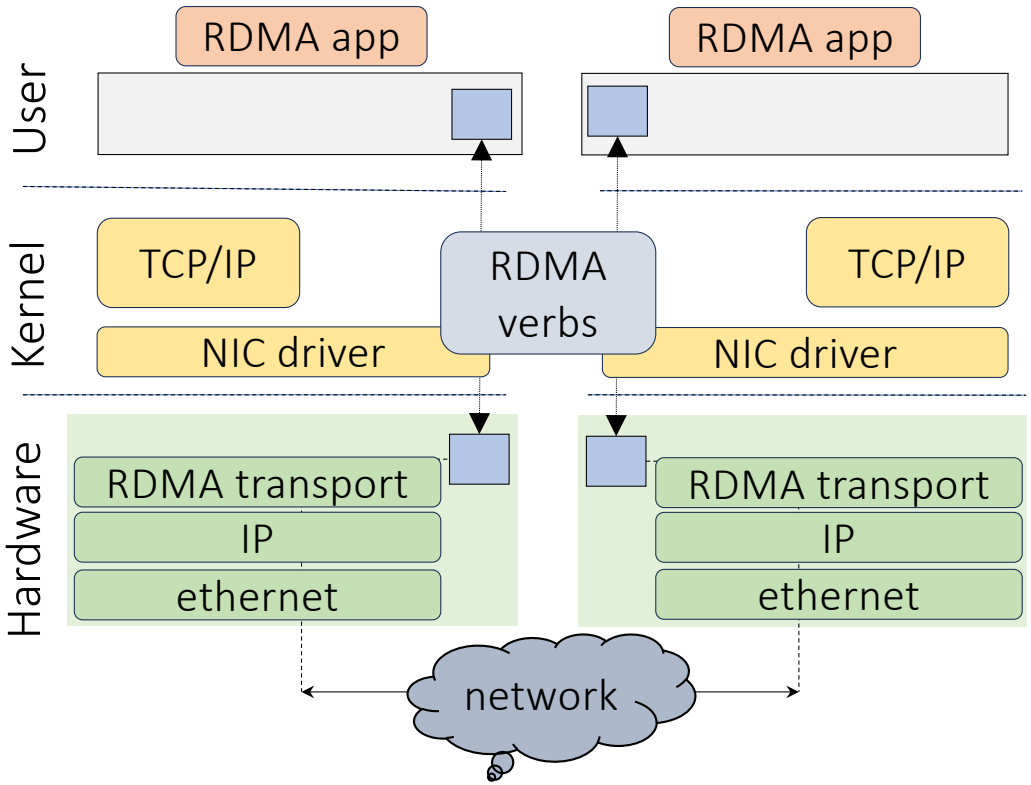


Applications register memory regions with the NIC

The memory regions are used to store the data to be sent or received

RDMA verbs are used by applications to interact directly with the NIC bypassing the kernel

RDMA basics



Applications register memory regions with the NIC

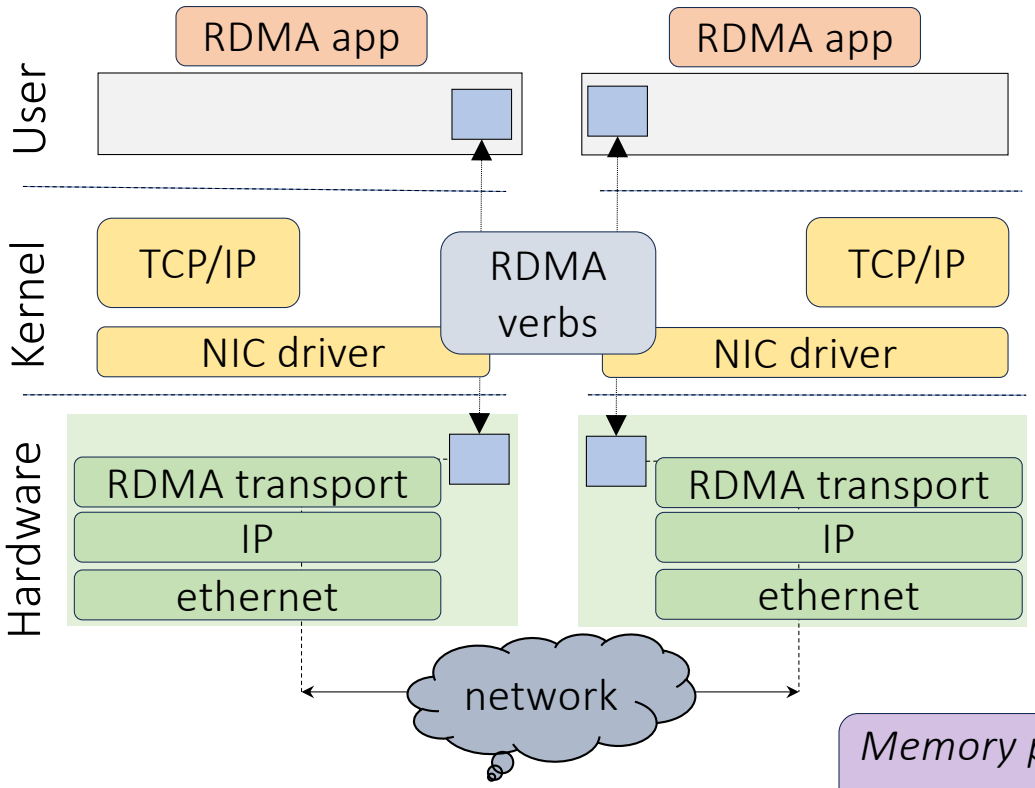
The memory regions are used to store the data to be sent or received

RDMA verbs are used by applications to interact directly with the NIC bypassing the kernel

The NIC implements:

- Memory protection
- IP processing
- Transport

RDMA basics



Applications register memory regions with the NIC

The memory regions are used to store the data to be sent or received

RDMA verbs are used by applications to interact directly with the NIC bypassing the kernel

The NIC implements:

- Memory protection
- IP processing
- Transport

Memory protection: no remote applications will access unwanted memory regions

RDMA verbs

RDMA networks support two types of memory access models

One-sided

Examples: RDMA read, RDMA write

Those are memory verbs:

- They specify the remote memory address on which operations should be carried out
- No involvement on the receive side

RDMA verbs

RDMA networks support two types of memory access models

One-sided

Examples: RDMA read, RDMA write

Those are memory verbs:

- They specify the remote memory address on which operations should be carried out
- No involvement on the receive side

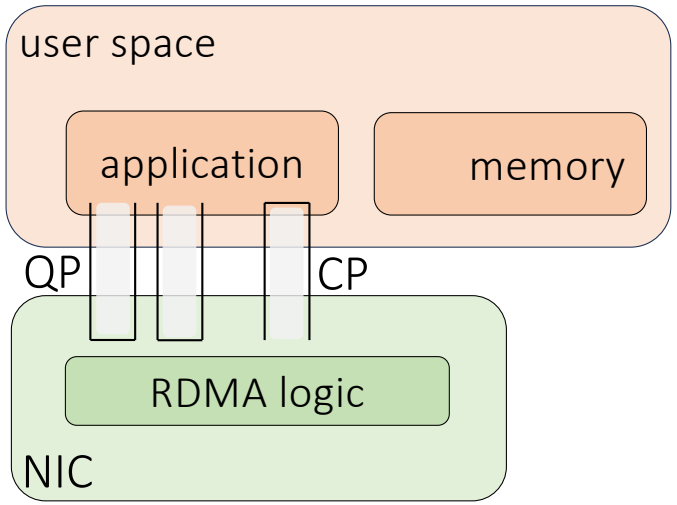
Two-sided

Examples: RDMA send, RDMA receive

Those are messaging verbs:

- This is like TCP sockets. Server must listen before client issues messages
- Sender and receiver do not know each other virtual memory location

Work Queues



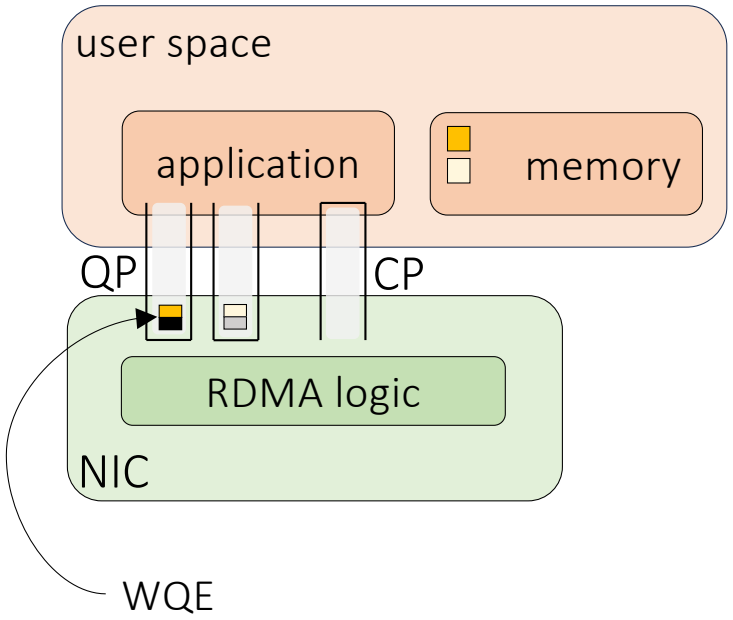
- RDMA communication is based on a set of three queues
  - Send
  - Receive
  - Completion

  Work queues, always created as a Queue Pair (QP)

- The send and receive queues are there to schedule the work to be done

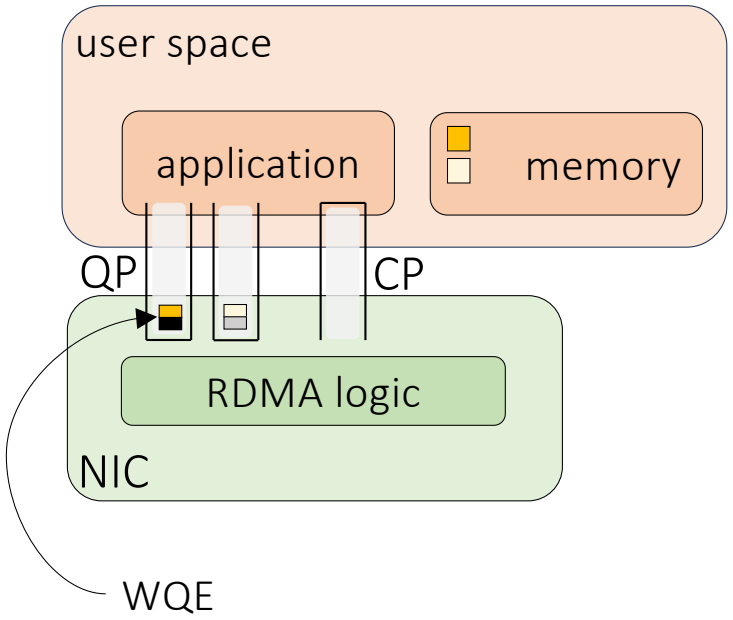
- A completion queue is used to notify when the work has been completed

Work Queues



- Applications issue (post) a job using a Work Request Element (WQE)
- A WQE is a small struct with a pointer to a buffer
  - In a *send queue*: it is the pointer to a message to be sent
  - In a *receive queue*: it shows where an incoming message shall be places
- Once a work request has been completed, the NIC creates a Completion Queue Element (CQE) and enqueues it in the *completion queue*

Work Queues

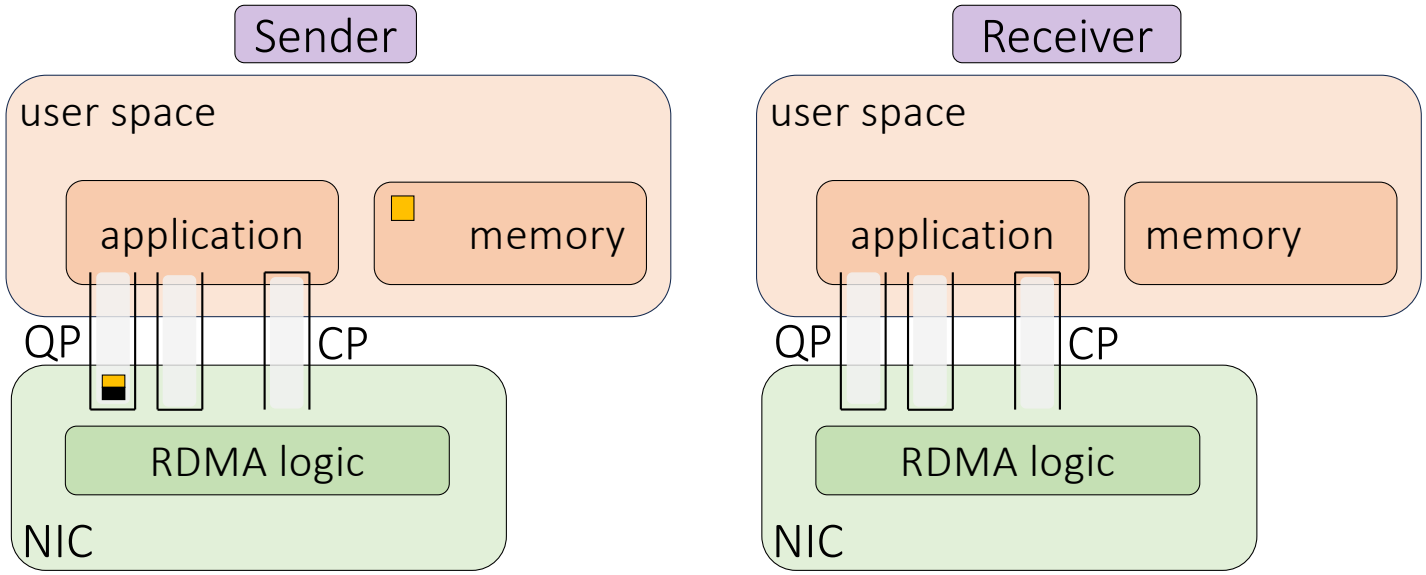


- Applications issue (post) a job using a Work Request Element (WQE)
- A WQE is a small struct with a pointer to a buffer
  - In a *send queue*: it is the pointer to a message to be sent
  - In a *receive queue*: it shows where an incoming message shall be places
- Once a work request has been completed, the NIC creates a Completion Queue Element (CQE) and enqueues it in the *completion queue*

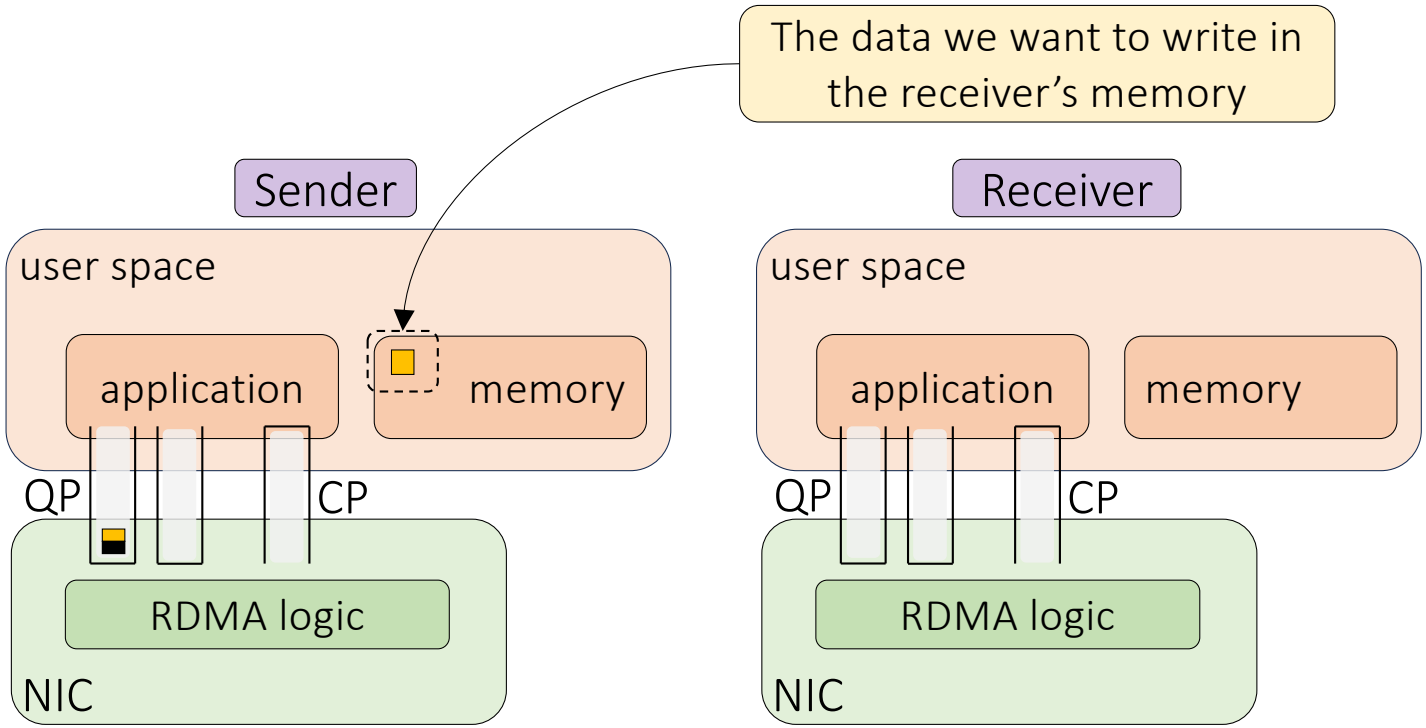
Between posting and completion the state of the memory involved shall not be touched!

RDMA Write (Read is similar)

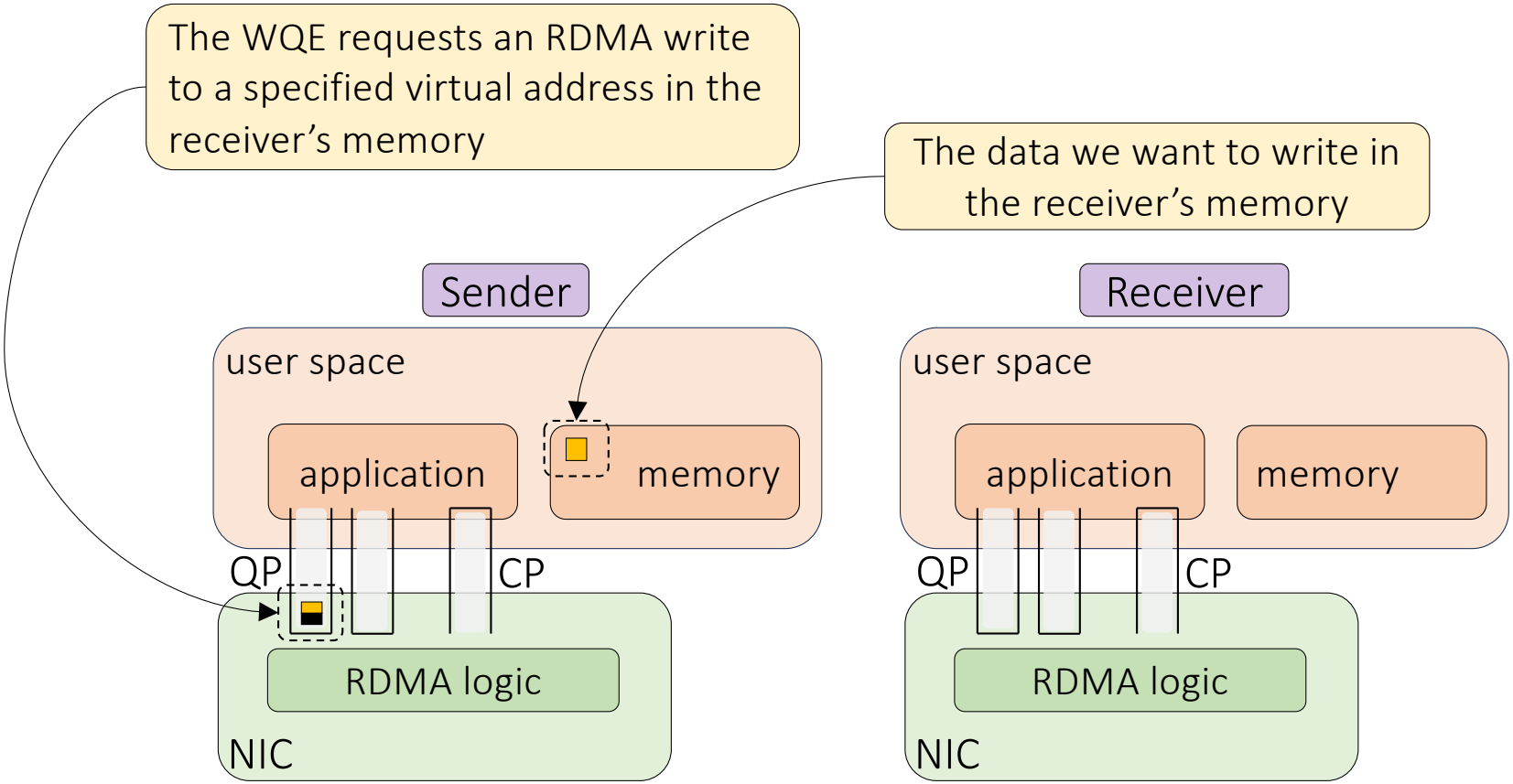
Only the sender side is active, and the receiver is passive (no operations, no CPU cycles, no indication that a write has happened)



RDMA Write (Read is similar)

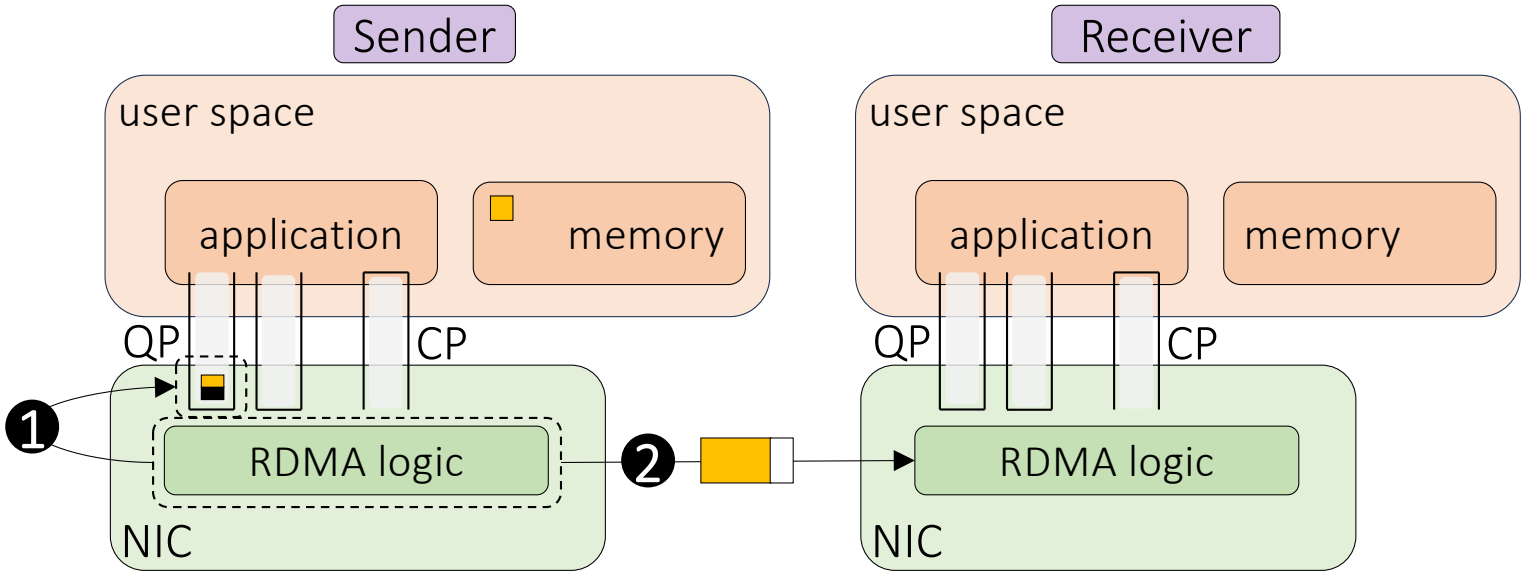


RDMA Write (Read is similar)



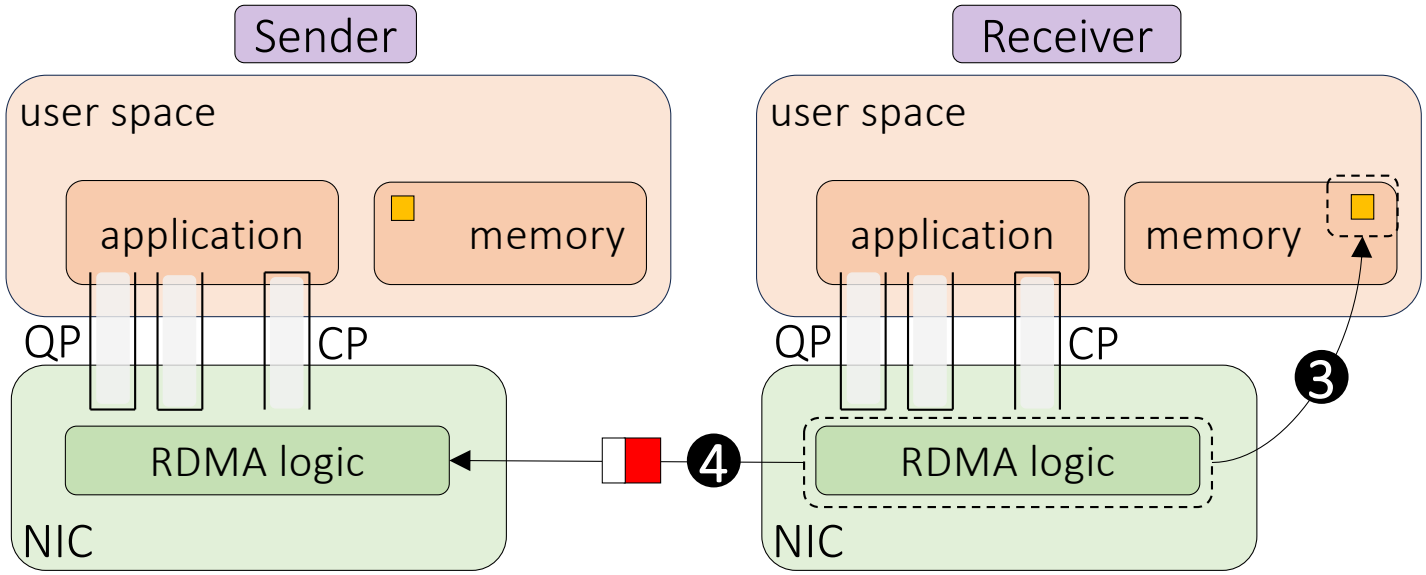
RDMA Write (Read is similar)

The NIC retrieves the WQE, assembles the corresponding RDMA packet, and sends it over the network



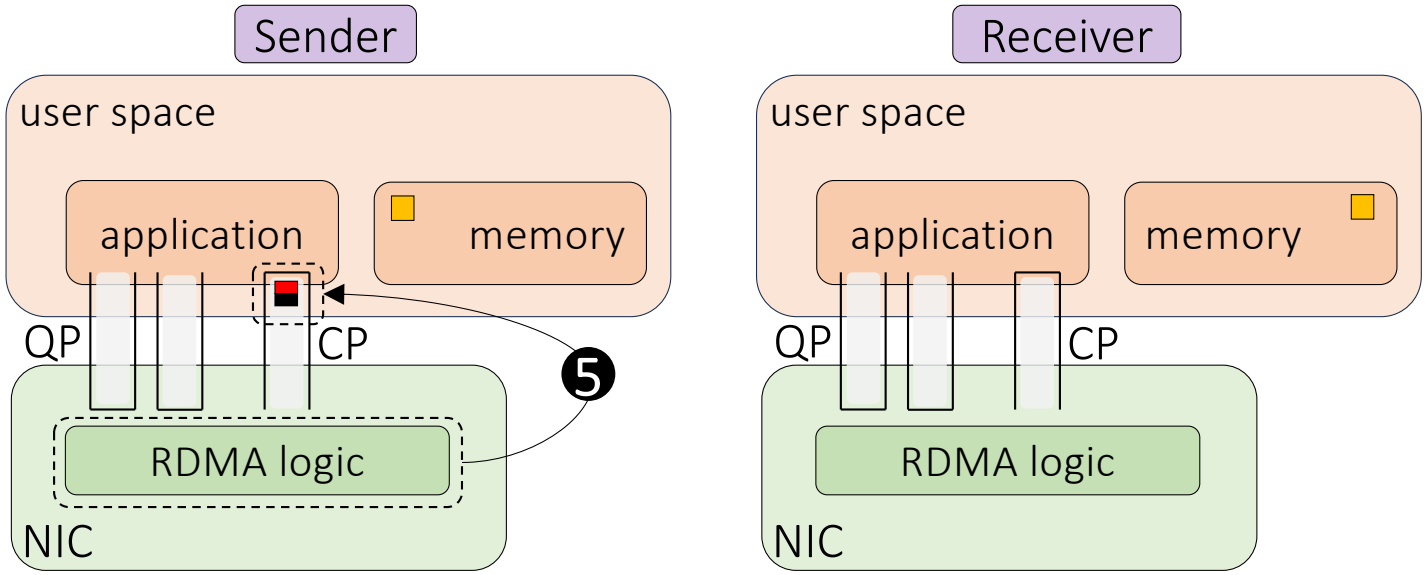
RDMA Write (Read is similar)

The receiver NIC performs a DMA write at the requested memory location and acknowledges completion to the sender



RDMA Write (Read is similar)

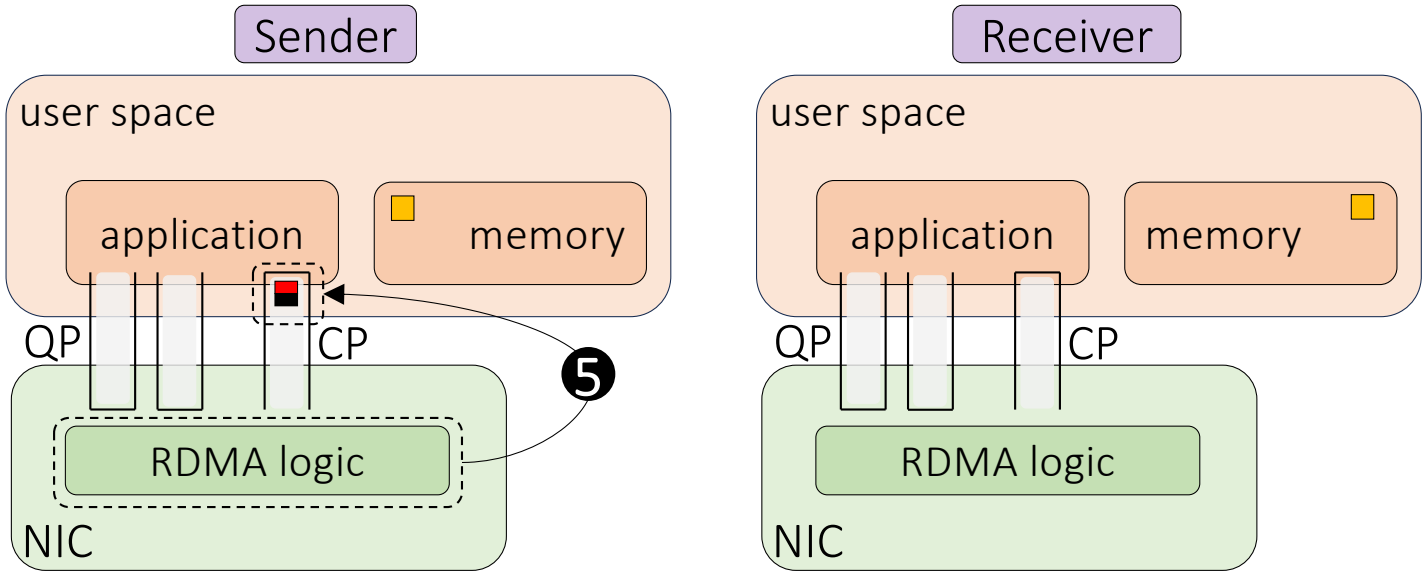
Upon reception of the ACK, the NIC sender enqueue a completion message in the CP queue



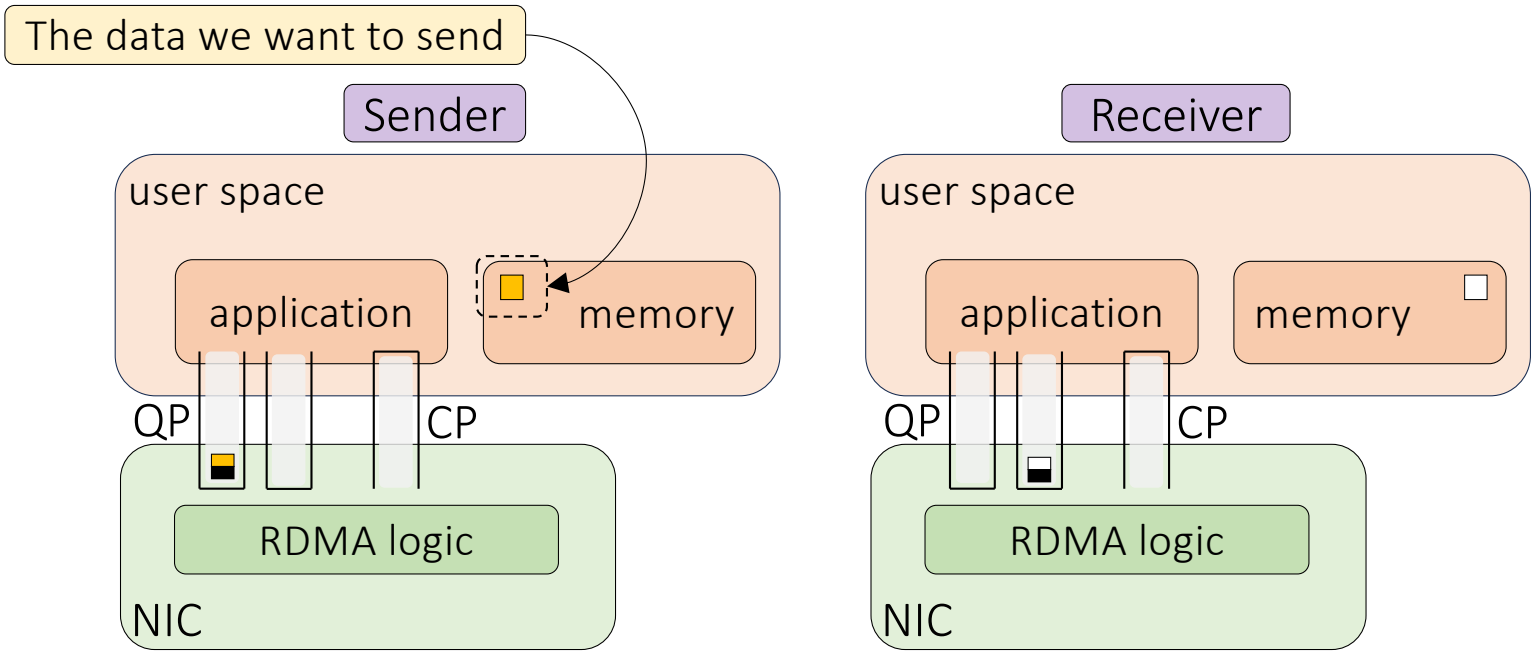
RDMA Write (Read is similar)

Upon reception of the ACK, the NIC sender enqueue a completion message in the CP queue

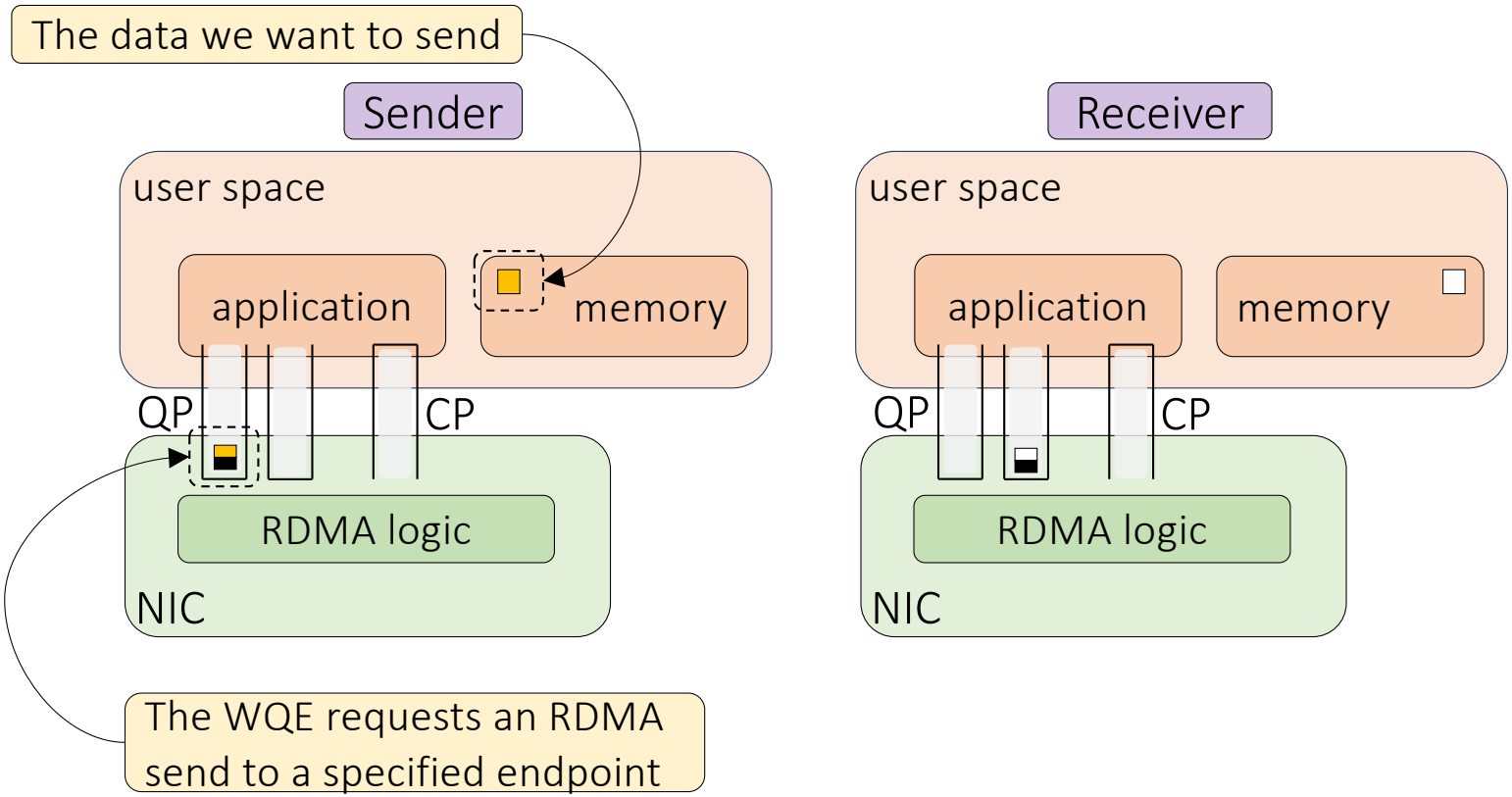
Note: When an RDMA write operation requires multiple packets, the completion queue is updated only after all packets have been correctly received



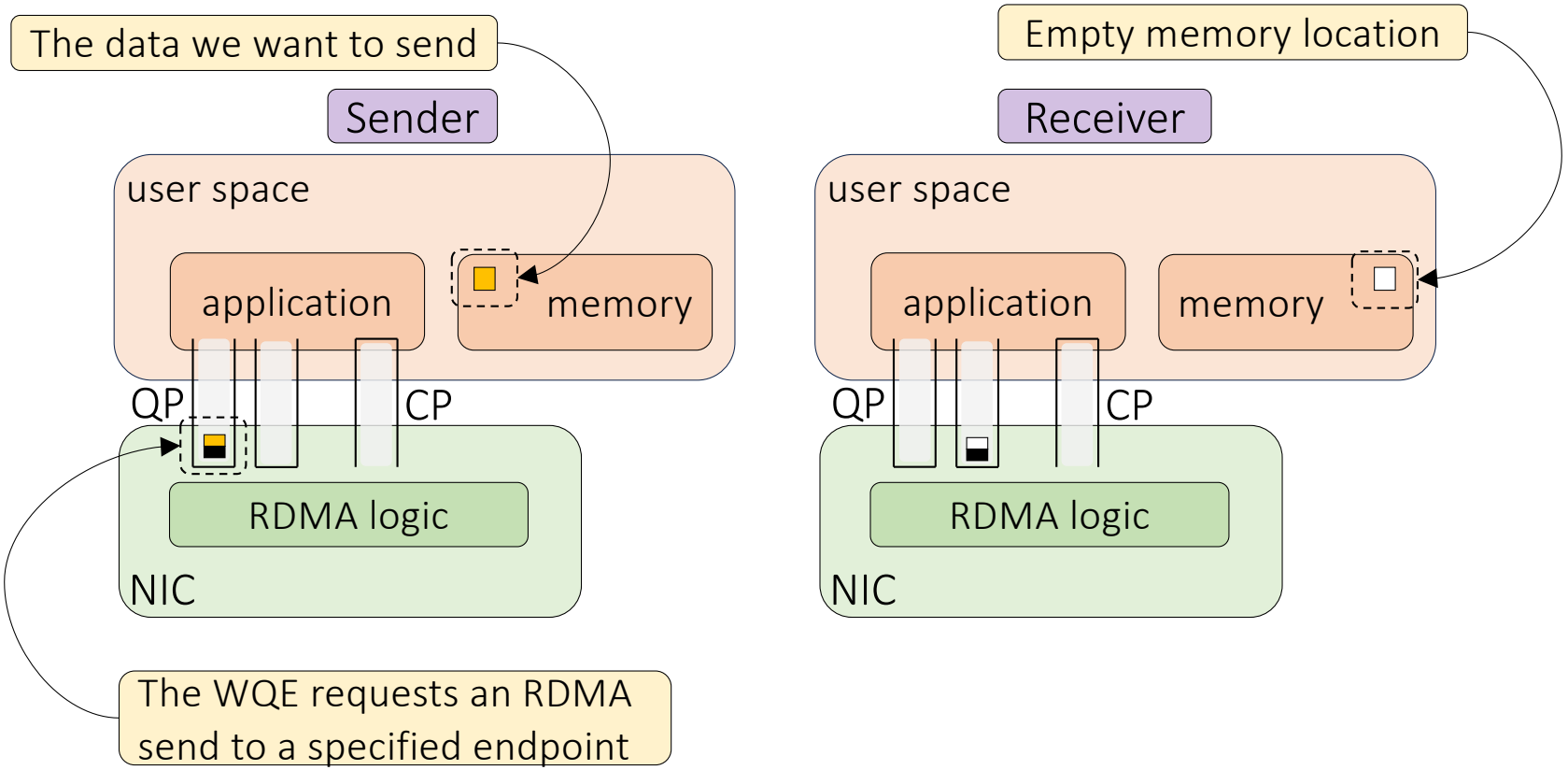
RDMA Send/Receive



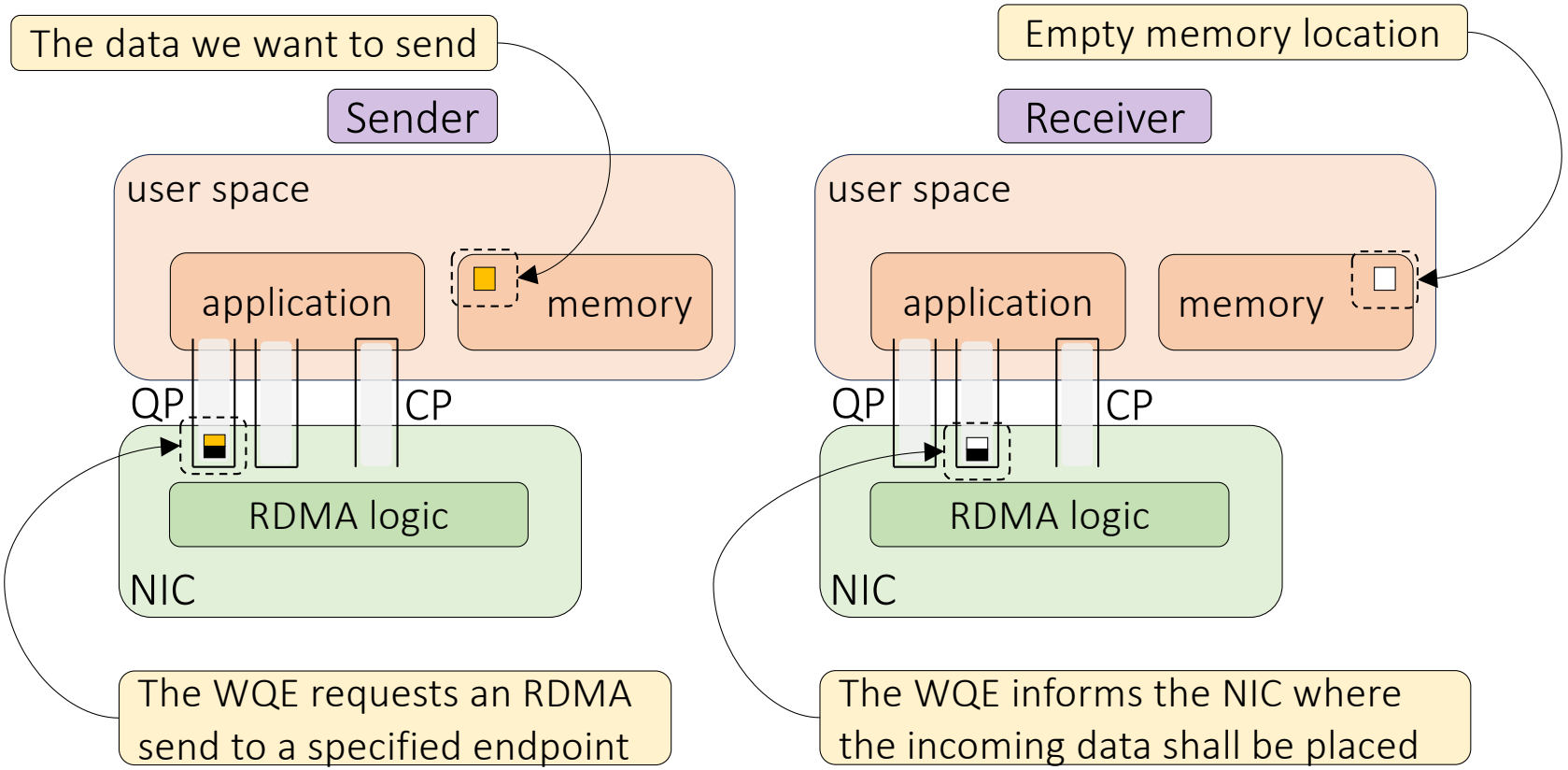
RDMA Send/Receive



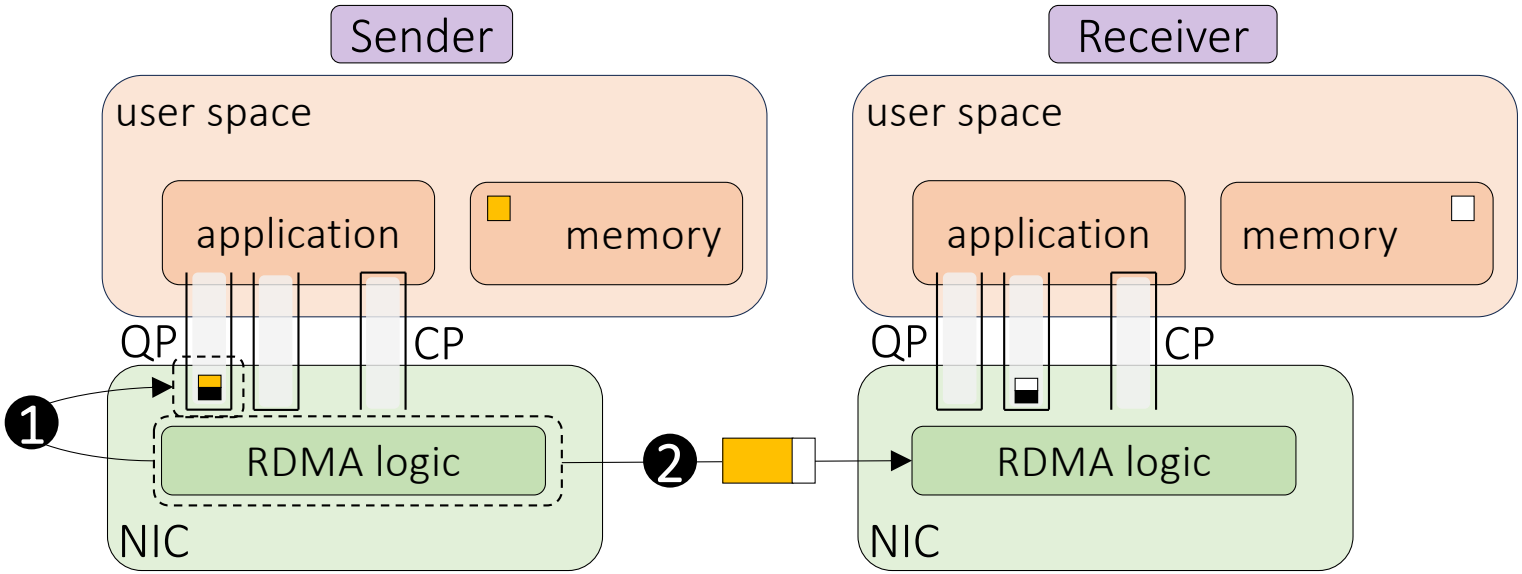
RDMA Send/Receive



RDMA Send/Receive

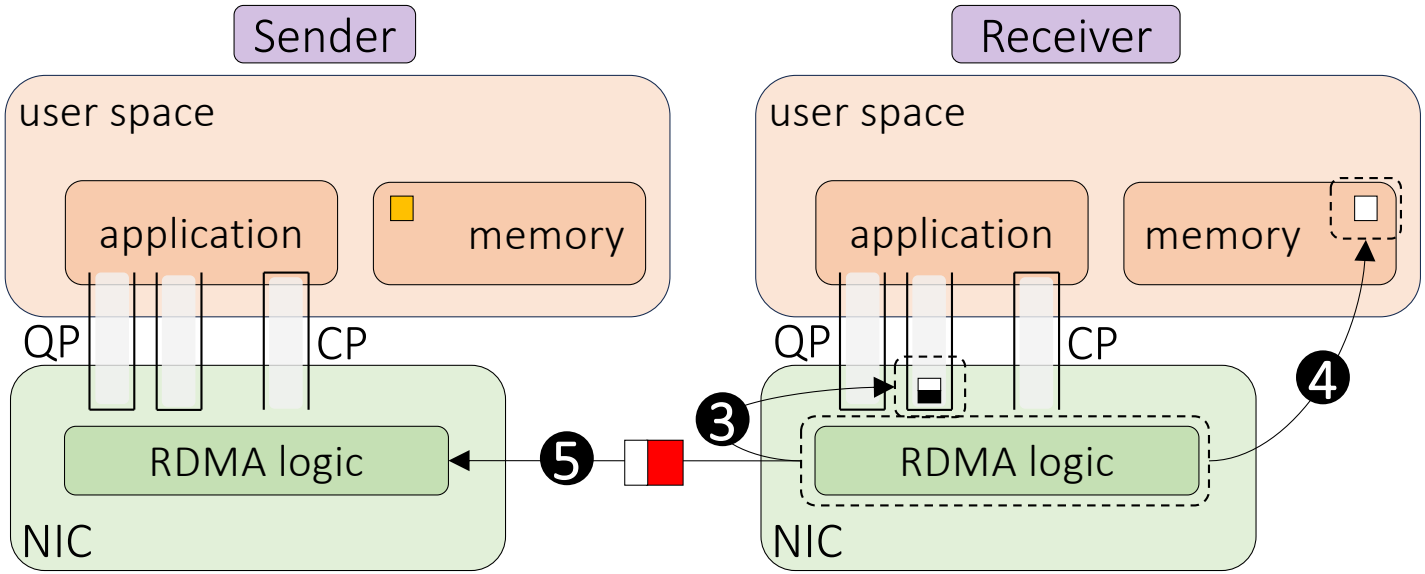


RDMA Send/Receive



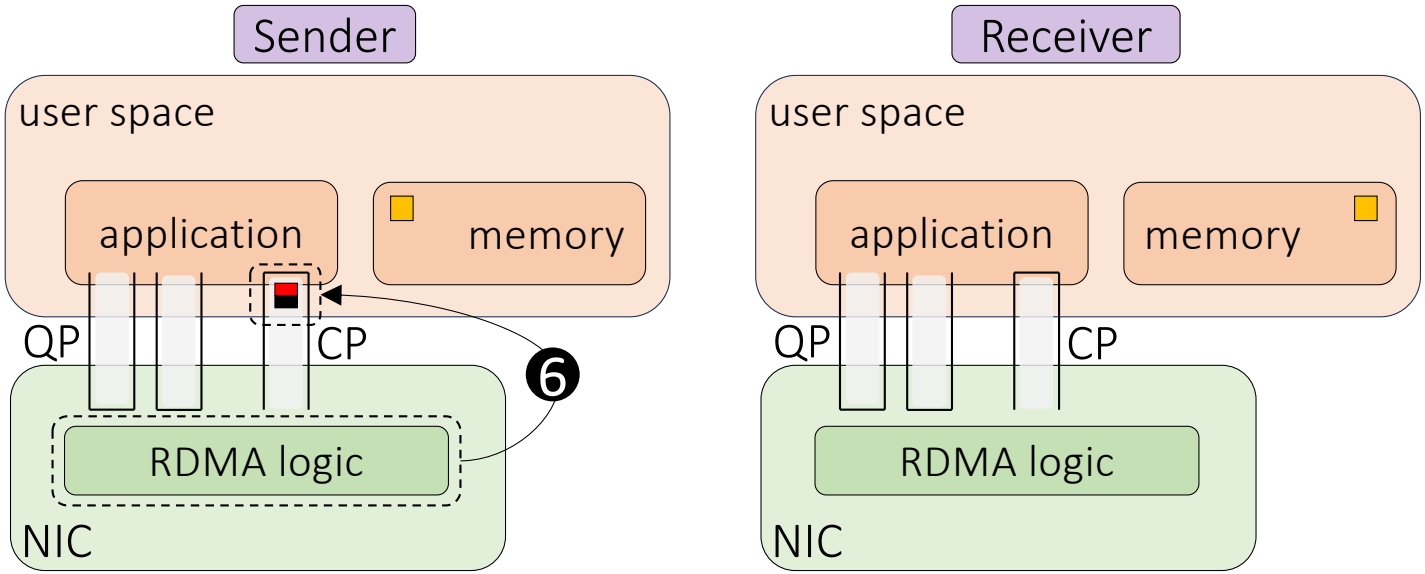
The NIC retrieves the WQE, assembles the corresponding RDMA packet, and sends it over the network

RDMA Send/Receive



The receiver NIC fetches a receive WQE to resolve the target memory address, executes a DMA write to that location, and acknowledges completion to the sender

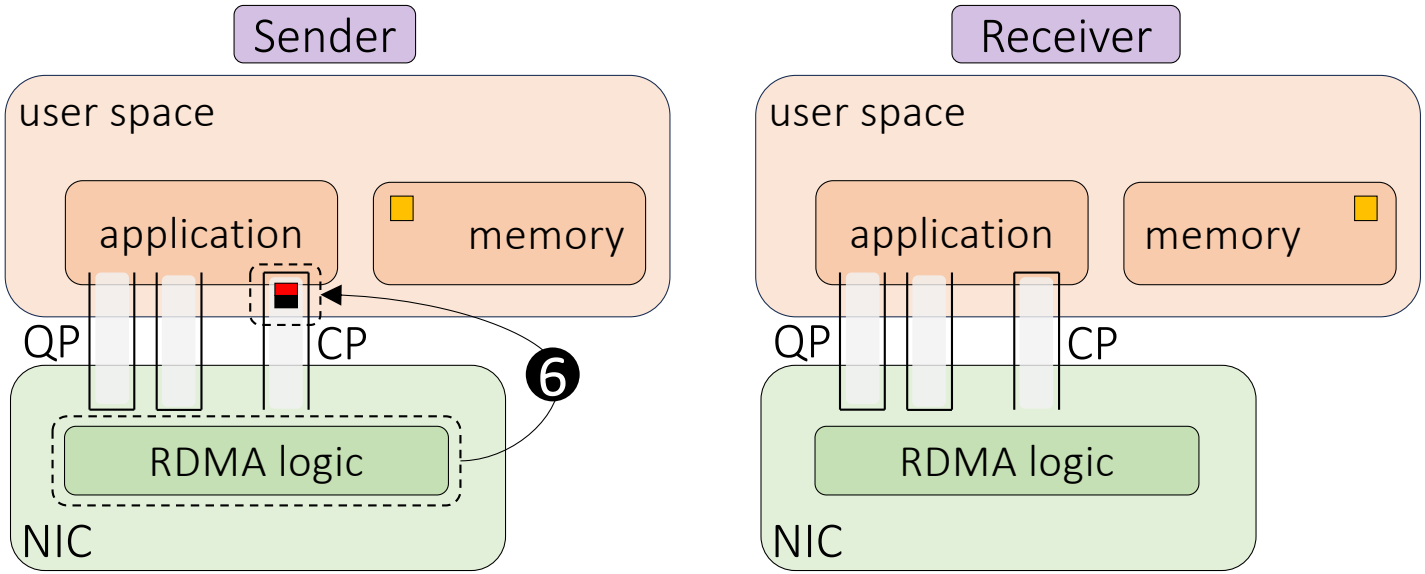
RDMA Send/Receive



Upon reception of the ACK, the NIC sender enqueue a completion message in the CP queue

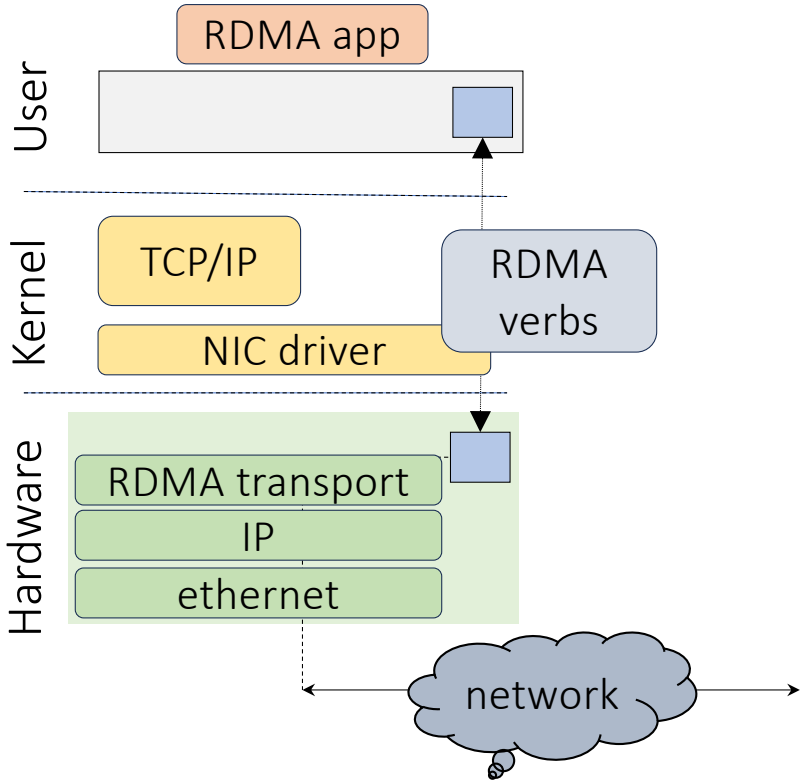
RDMA Send/Receive

Note: When an RDMA send operation requires multiple packets, the completion queue is updated only after all packets have been correctly received



Upon reception of the ACK, the NIC sender enqueue a completion message in the CP queue

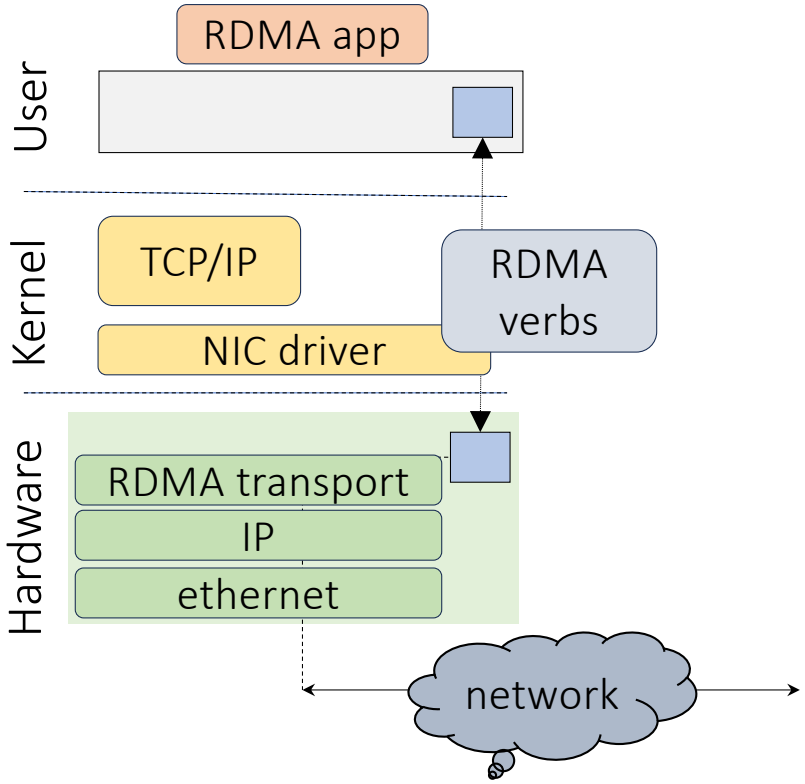
The NIC as fundamental building block in RDMA



The NIC implements:

- Memory protection
- IP processing
- Transport

The NIC as fundamental building block in RDMA

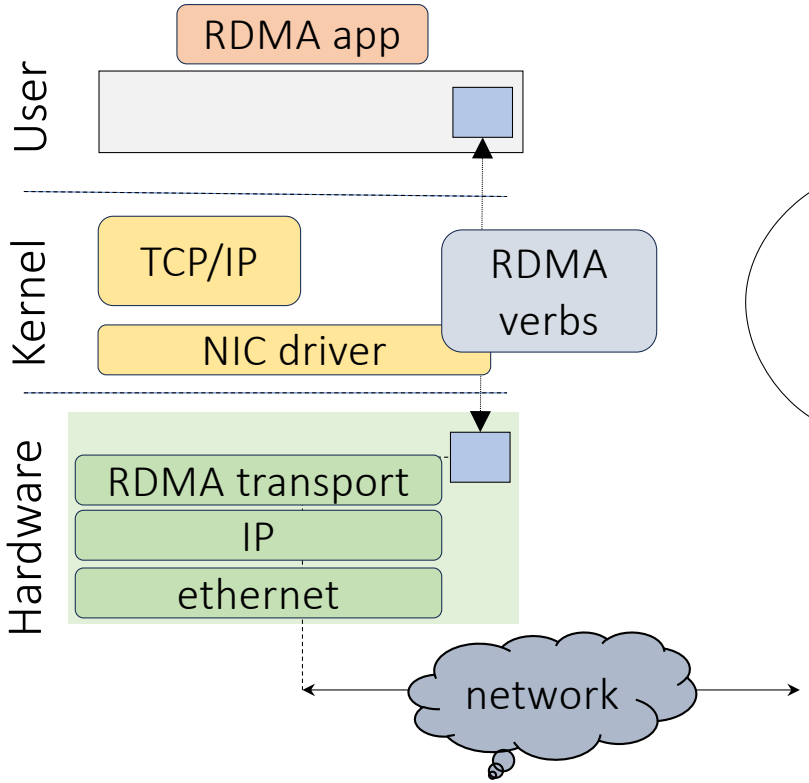


The NIC implements:

- Memory protection
- IP processing
- Transport

All of these features needs physical resources that are limited

The NIC as fundamental building block in RDMA



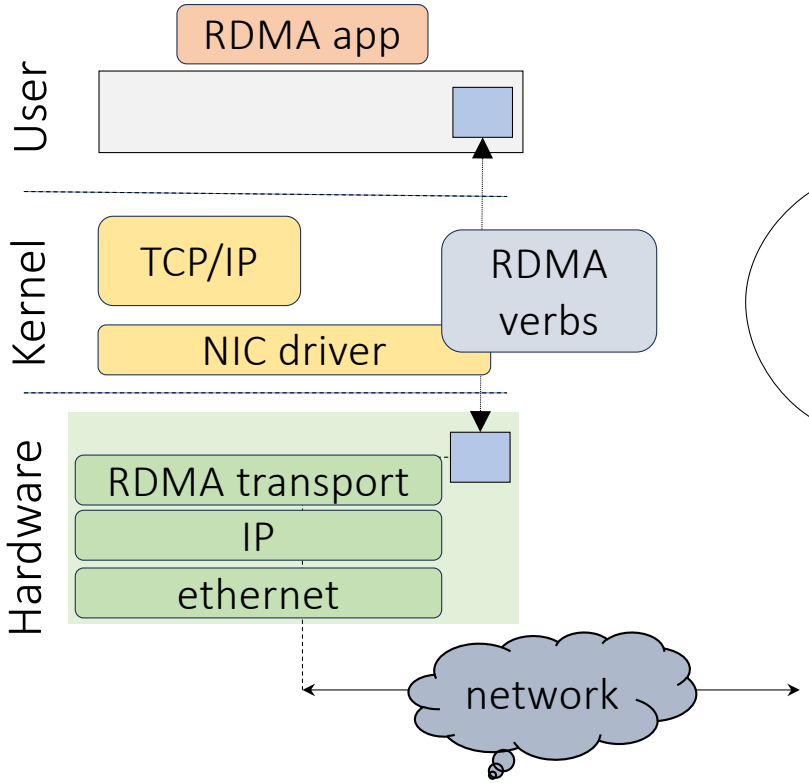
The NIC implements:

- Memory protection
- IP processing
- Transport

All of these features needs physical resources that are limited

Full TCP-like implementation would be too expensive (in terms of HW resources)

The NIC as fundamental building block in RDMA



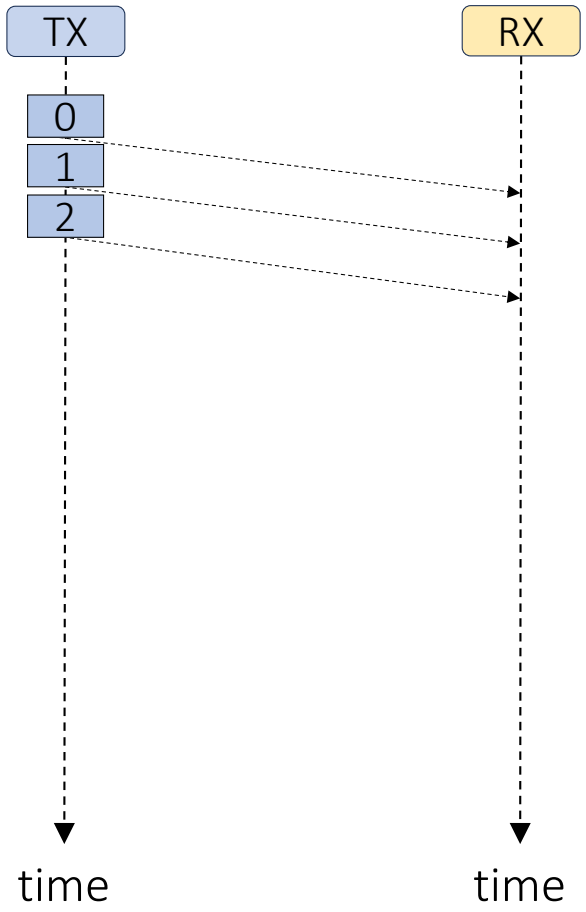
The NIC implements:
- Memory protection
- IP processing
- Transport

All of these features needs physical resources that are limited

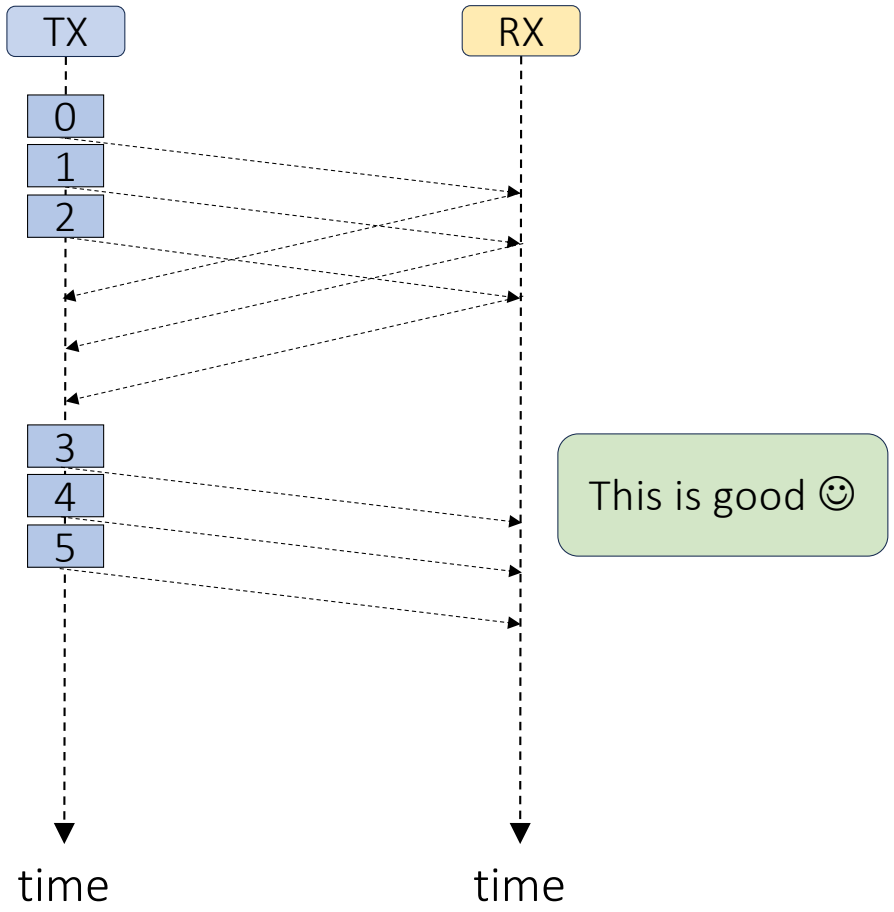
Full TCP-like implementation would be too expensive (in terms of HW resources)

Historically, RDMA NICs implement go-back-N as loss recovery to keep minimal state

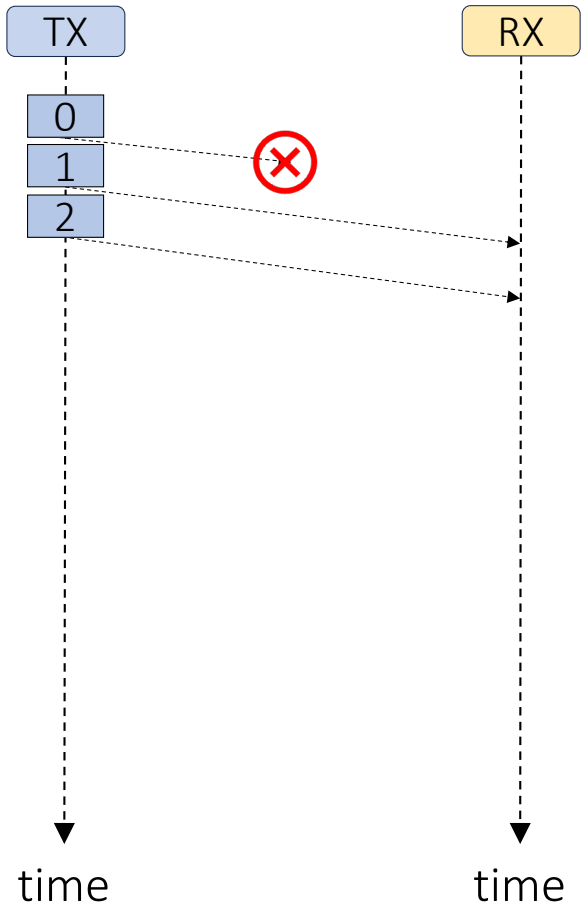
Go-back-N Loss Recovery



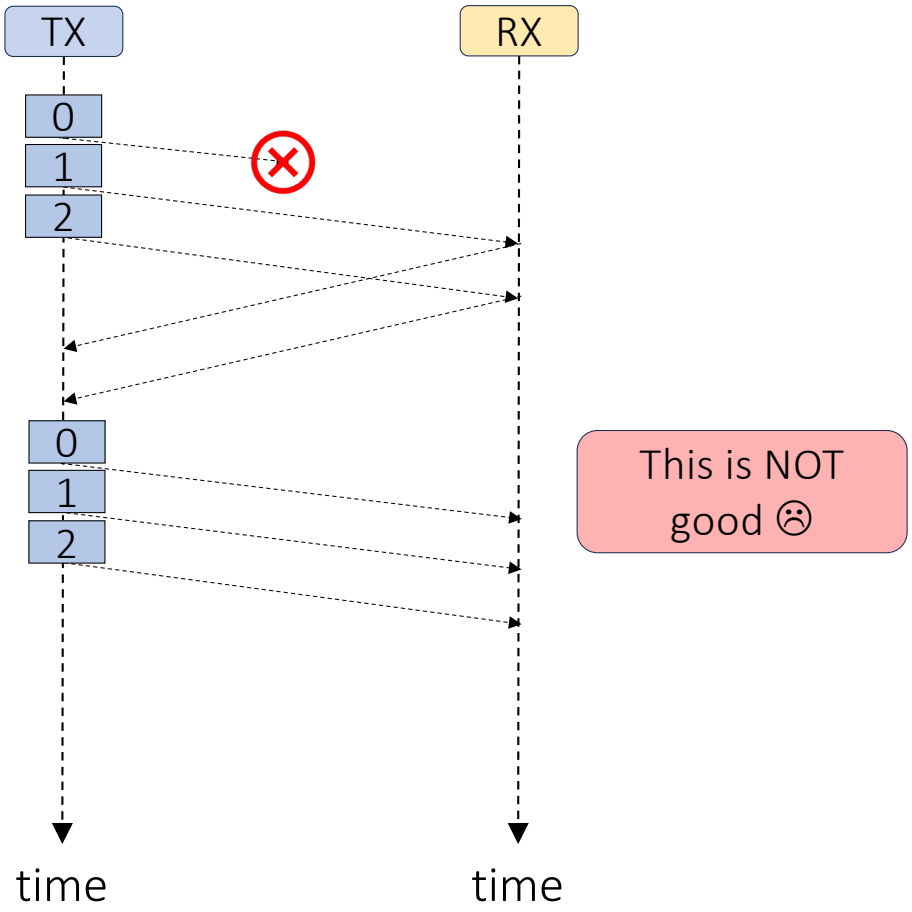
Go-back-N Loss Recovery



Go-back-N Loss Recovery

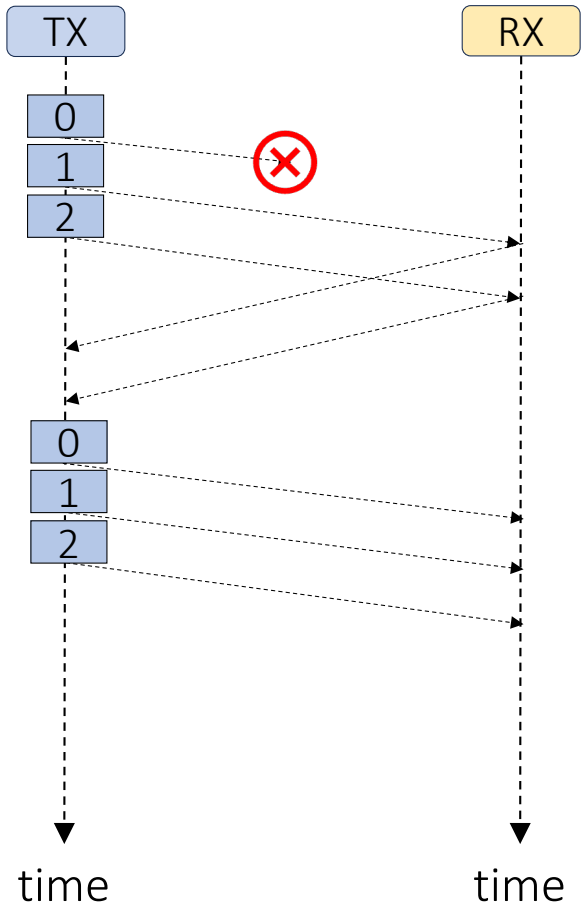


Go-back-N Loss Recovery



This is NOT good ☹

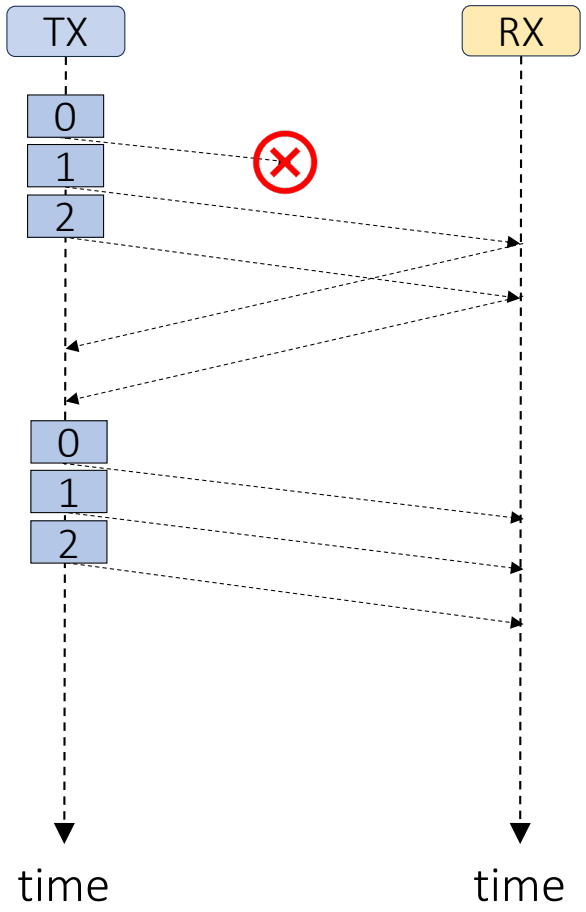
Go-back-N Loss Recovery



Receiver discards all out-of-order packets

Sender retransmits all packets sent after the last acked packet

Go-back-N Loss Recovery

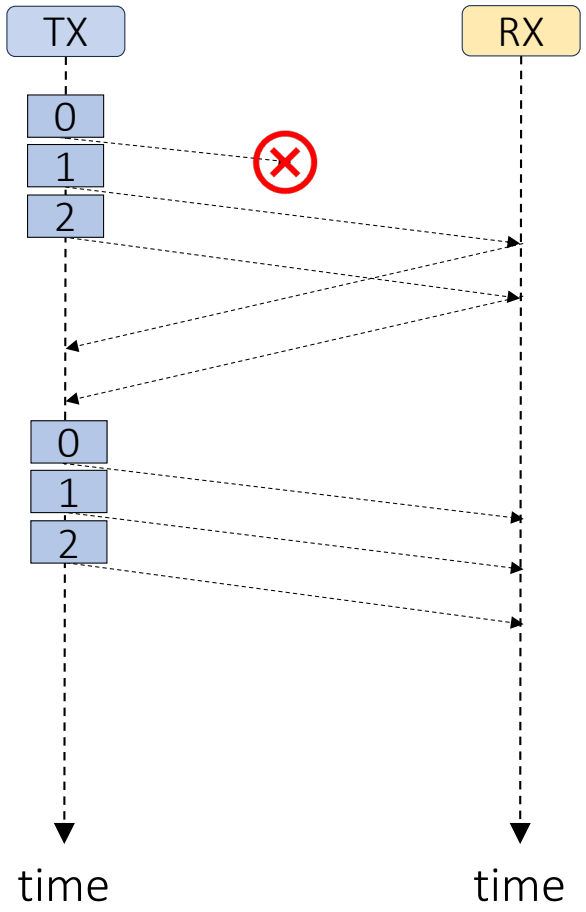


Receiver discards all out-of-order packets

Sender retransmits all packets sent after the last acked packet

This makes NICs implementation easier:
no need to keep much per-flow state

Go-back-N Loss Recovery



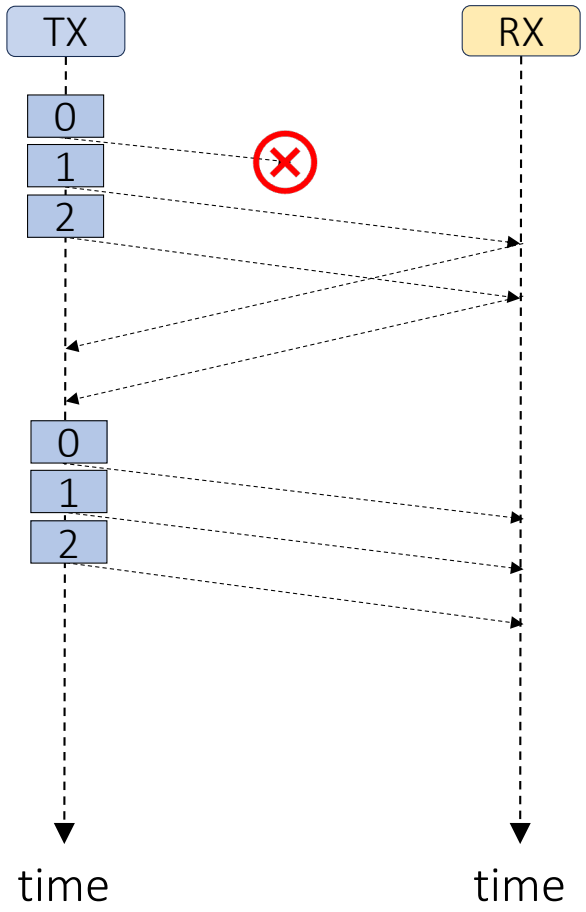
Receiver discards all out-of-order packets

Sender retransmits all packets sent after the last acked packet

This makes NICs implementation easier:
no need to keep much per-flow state

But makes performance easily affected by:
1. Packets reorder
2. Packets loss

Go-back-N Loss Recovery



Receiver discards all out-of-order packets

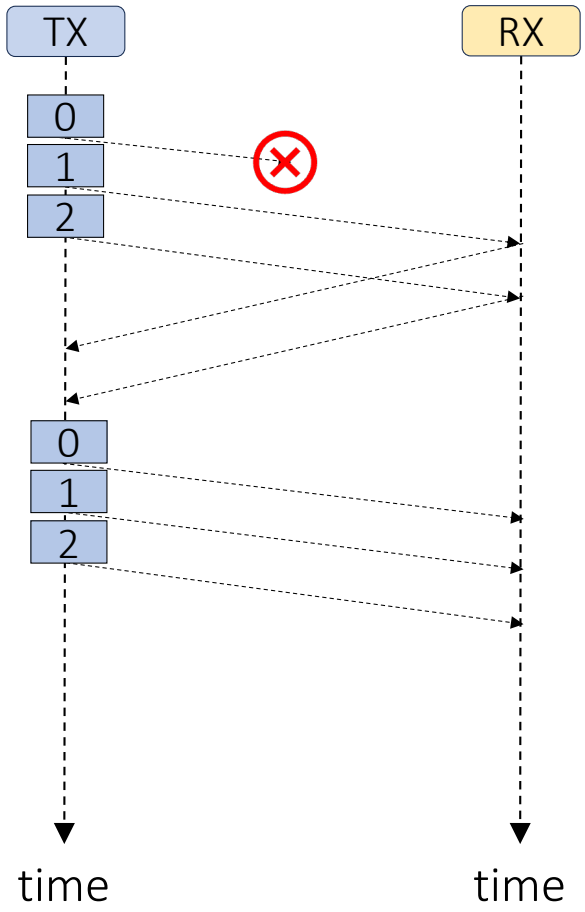
Sender retransmits all packets sent after the last acked packet

This makes NICs implementation easier:
no need to keep much per-flow state

But makes performance easily affected by:
1. Packets reorder
2. Packets loss

how to avoid them?

Go-back-N Loss Recovery



Receiver discards all out-of-order packets

Sender retransmits all packets sent after the last acked packet

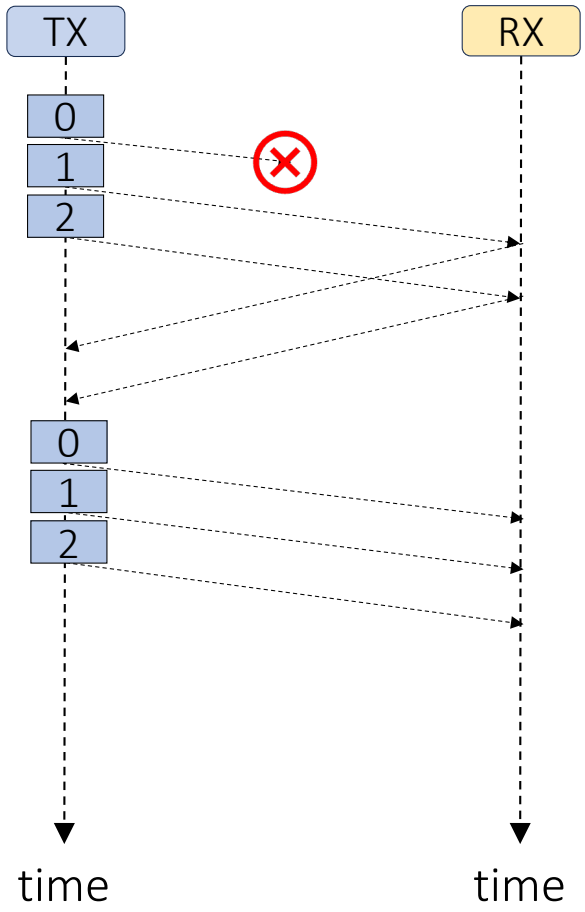
This makes NICs implementation easier:
no need to keep much per-flow state

But makes performance easily affected by:
1. Packets reorder
2. Packets loss

how to avoid them?

Routing/load balancing that keeps flow/path consistency (e.g., ECMP)

Go-back-N Loss Recovery



Receiver discards all out-of-order packets

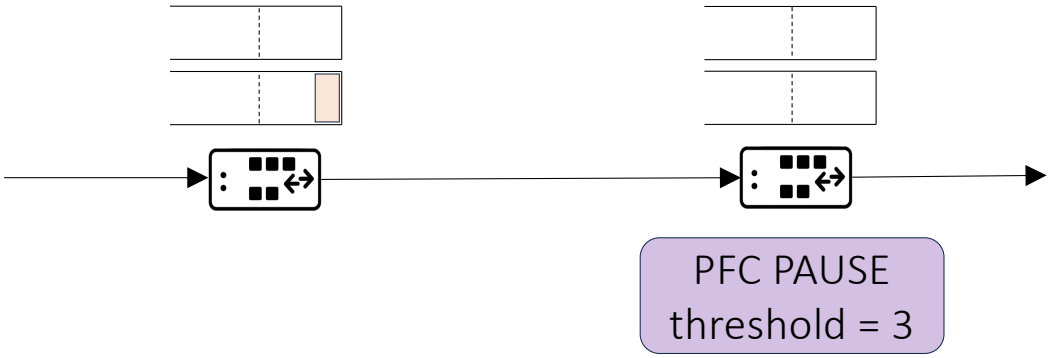
Sender retransmits all packets sent after the last acked packet

This makes NICs implementation easier:
no need to keep much per-flow state

But makes performance easily affected by:
1. Packets reorder
2. Packets loss

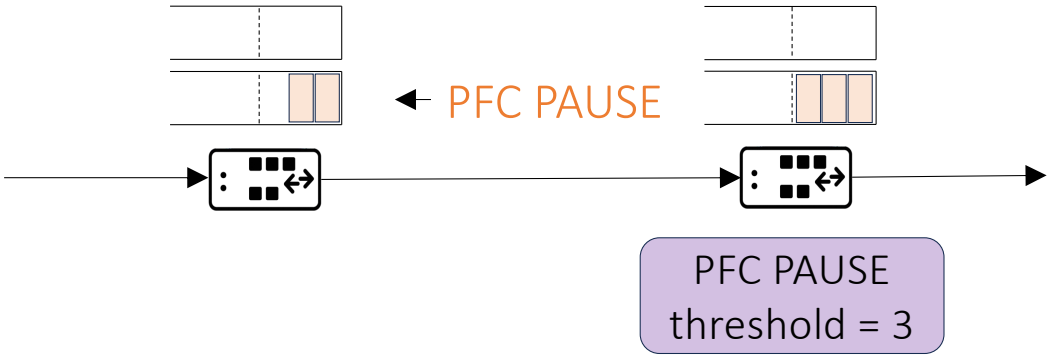
how to avoid them?

Drop-free Ethernet with Priority Flow Control (PFC)



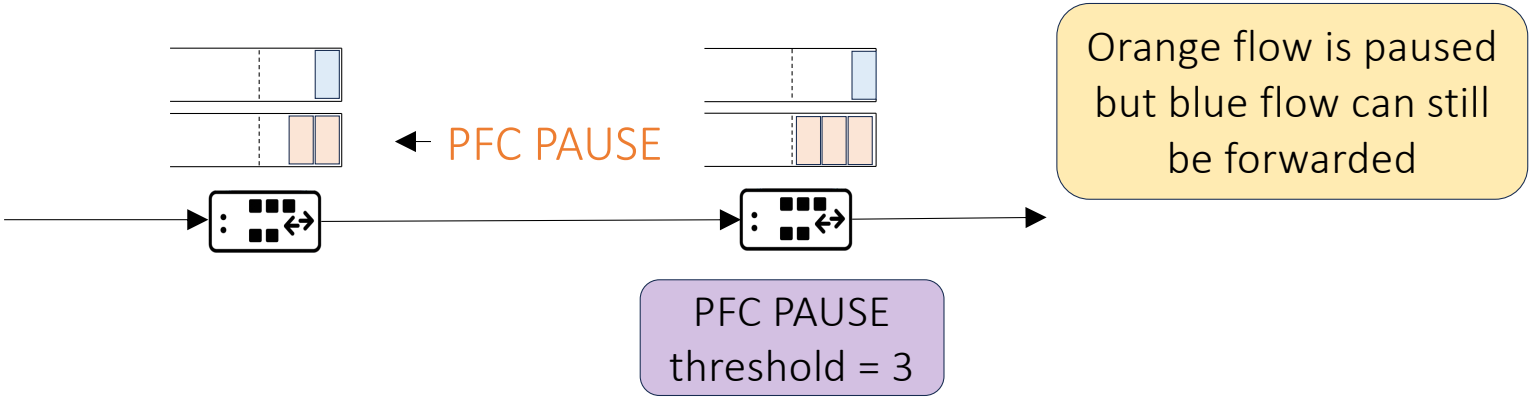
- Pause upstream neighbour when queue size reaches PFC threshold
  - No packets get dropped because of congestion
- Per queue: most ASICs support 8 queues

Drop-free Ethernet with Priority Flow Control (PFC)



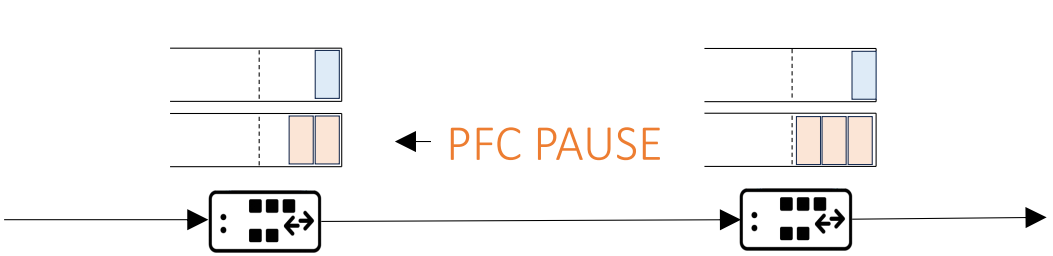
- Pause upstream neighbour when queue size reaches PFC threshold
  - No packets get dropped because of congestion
- Per queue: most ASICs support 8 queues

Drop-free Ethernet with Priority Flow Control (PFC)



- Pause upstream neighbour when queue size reaches PFC threshold
  - No packets get dropped because of congestion
- Per queue: most ASICs support 8 queues

Drop-free Ethernet with Priority Flow Control (PFC)



Orange flow is paused but blue flow can still be forwarded

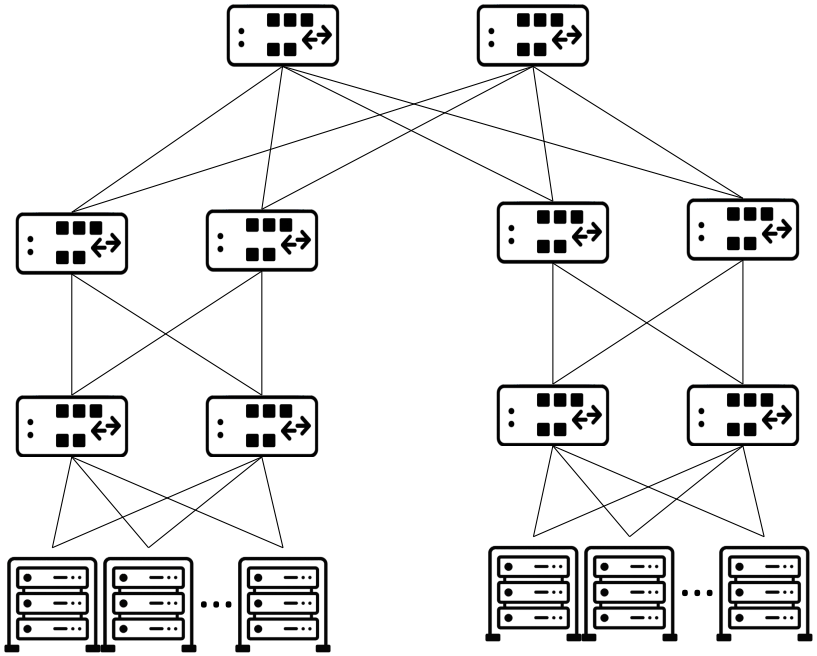
PFC PAUSE threshold = 3

- Pause upstream neighbour when queue size reaches PFC threshold
  - No packets get dropped because of congestion
- Per queue: most ASICs support 8 queues

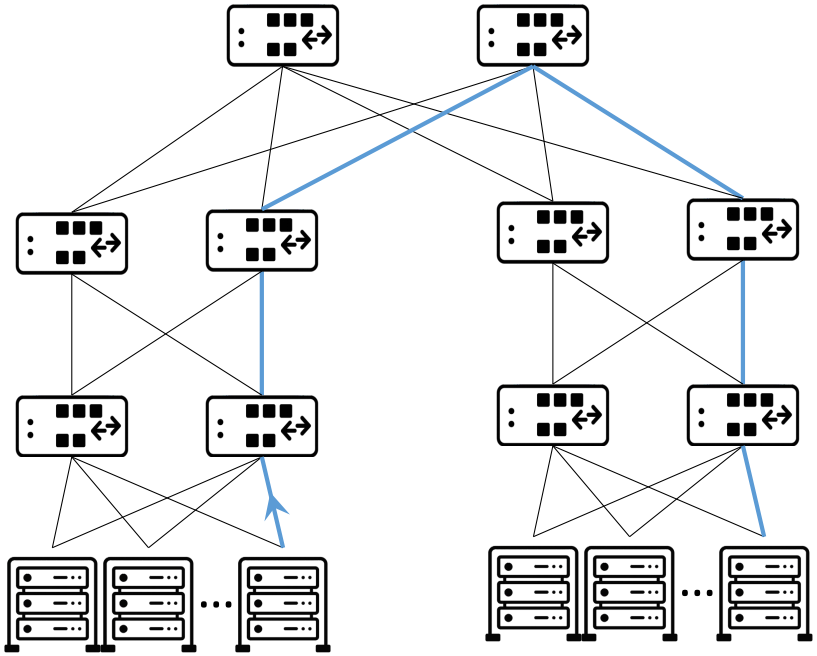
There are way more flows in a network than PFC queues

Consequence?

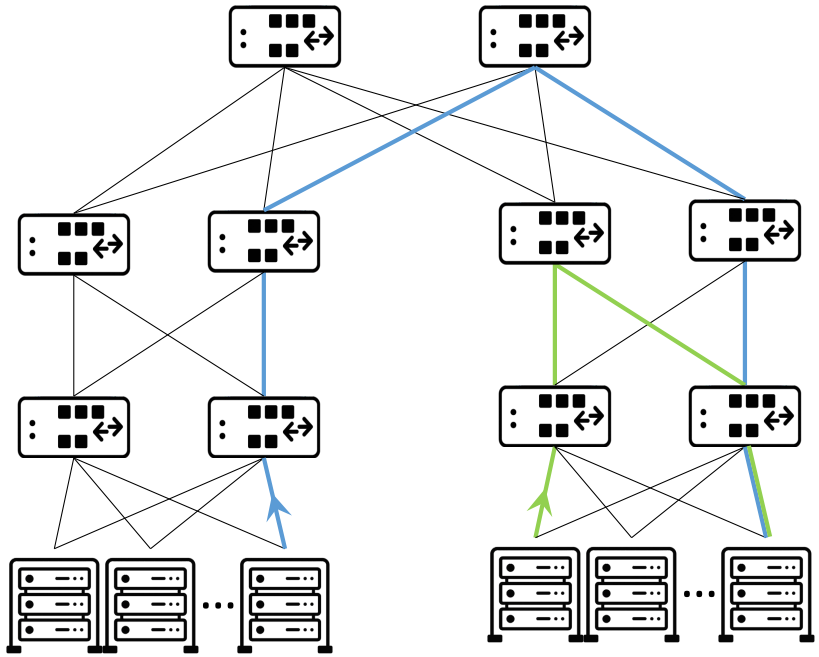
Victim Flow Problem



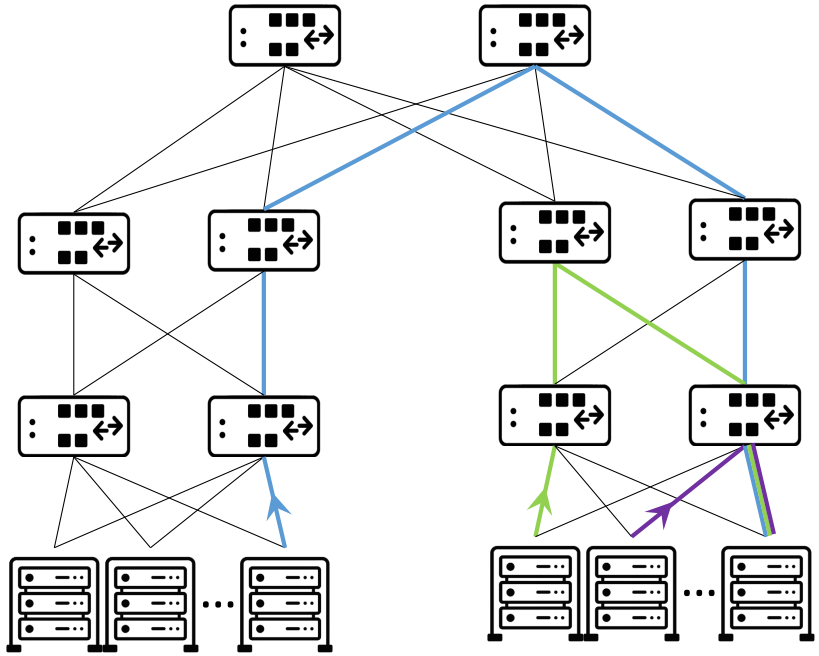
Victim Flow Problem



Victim Flow Problem

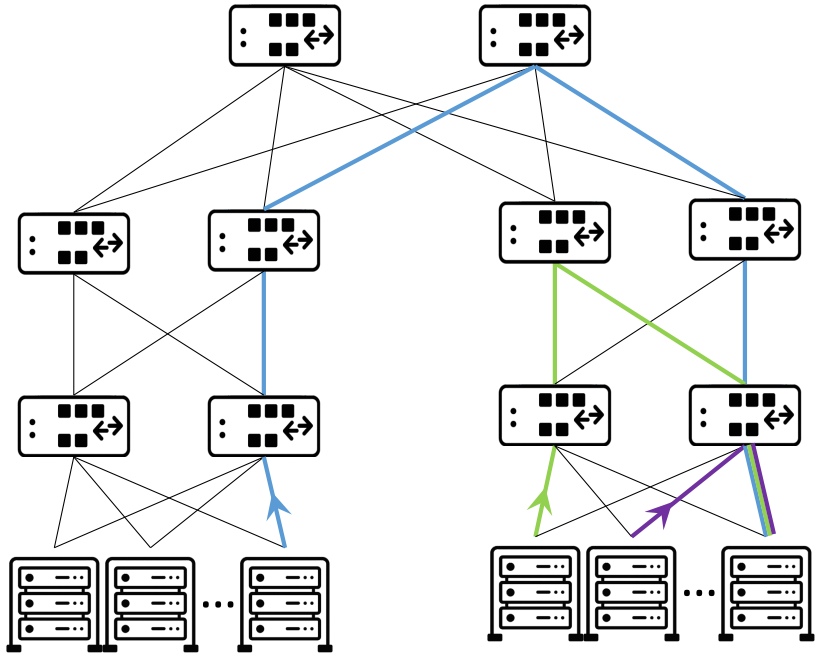


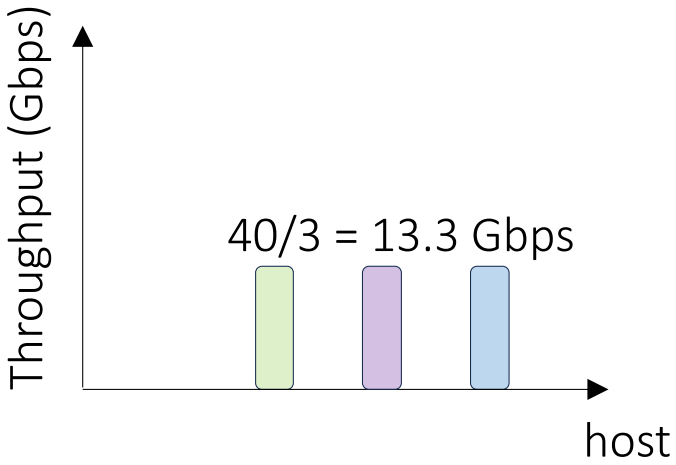
Victim Flow Problem



Victim Flow Problem

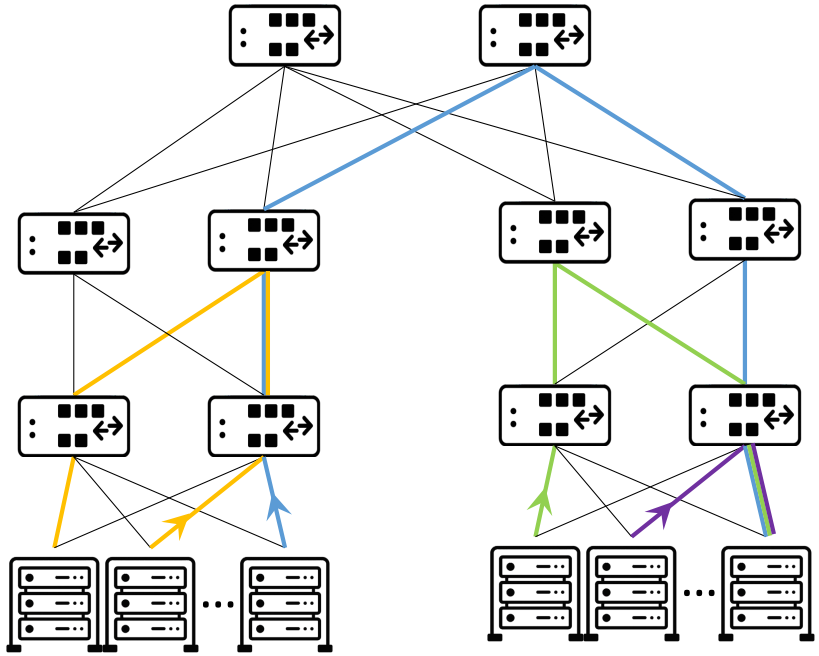
Assuming
40Gbps links

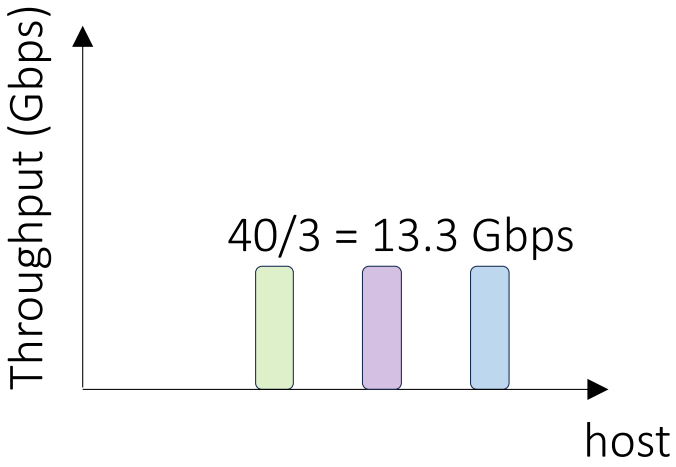




Victim Flow Problem

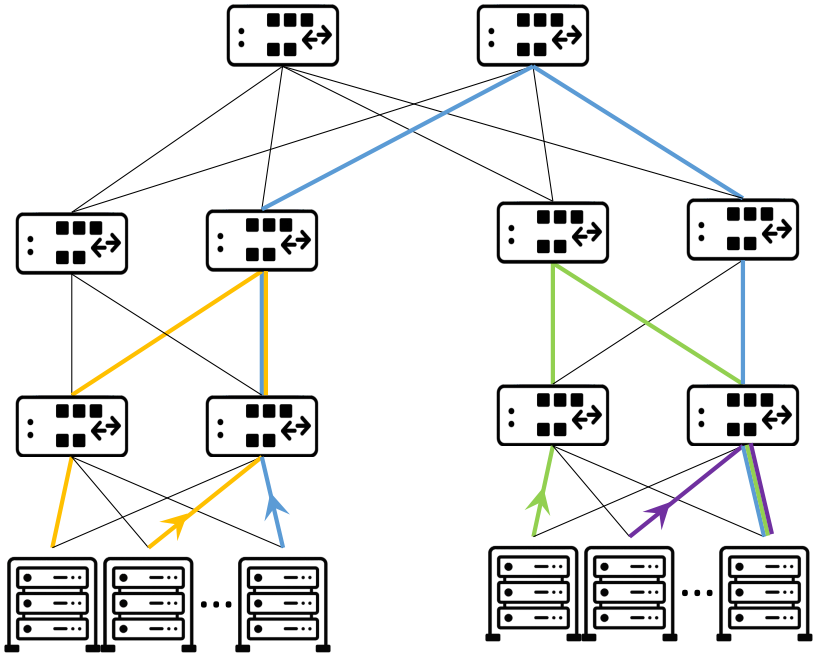
Assuming 40Gbps links



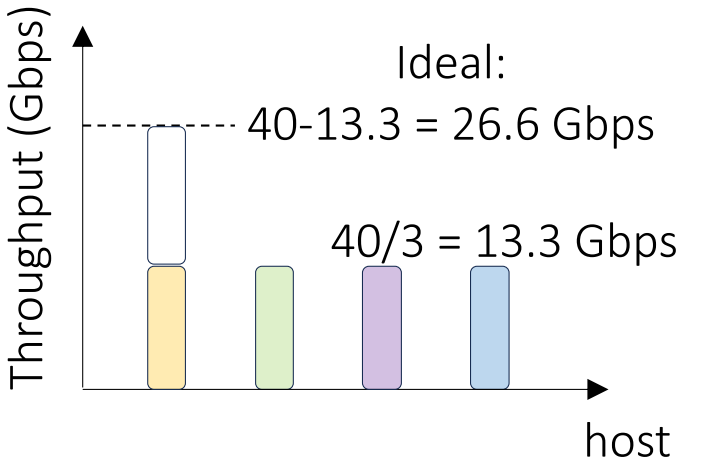


What about yellow flow?

Victim Flow Problem



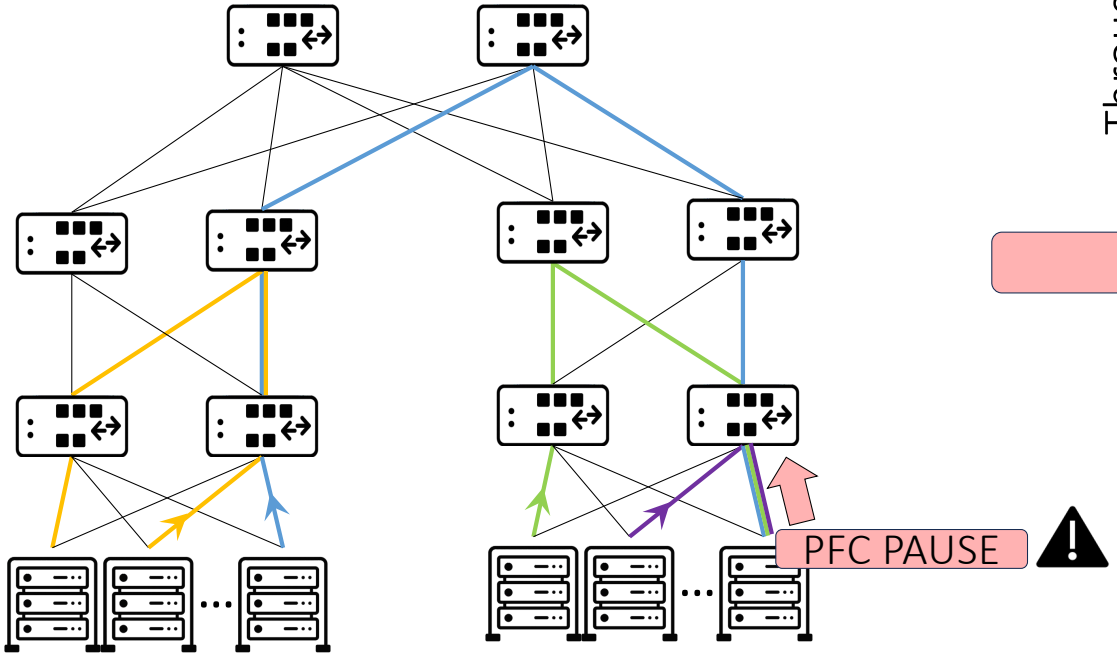
Assuming 40Gbps links

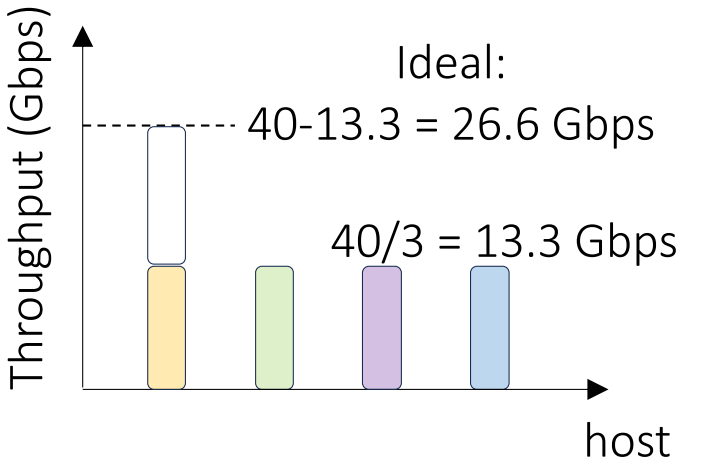


Why?

Victim Flow Problem

Assuming 40Gbps links

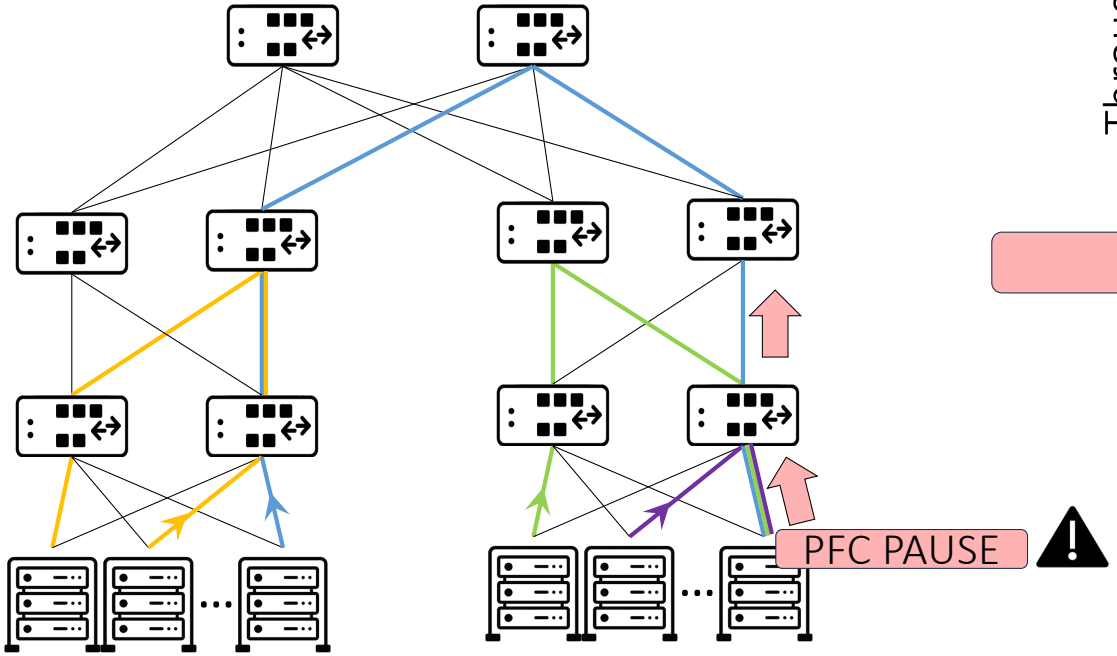


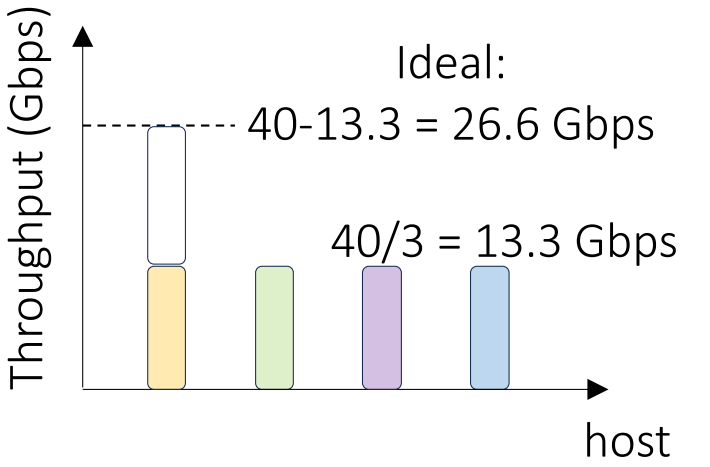


Why?

Victim Flow Problem

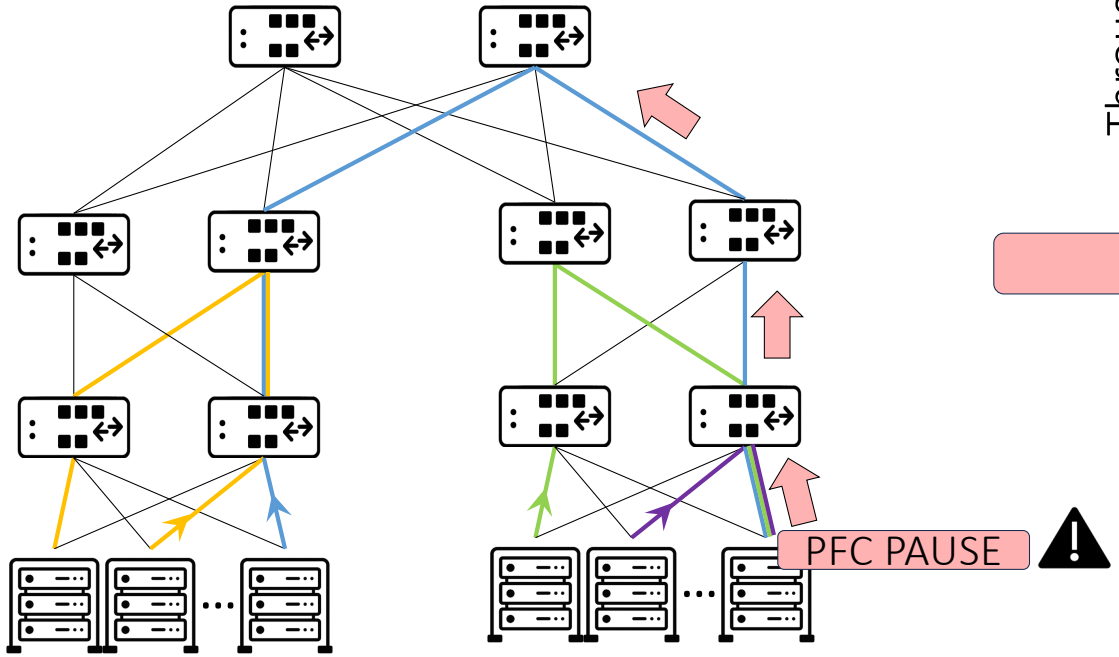
Assuming 40Gbps links



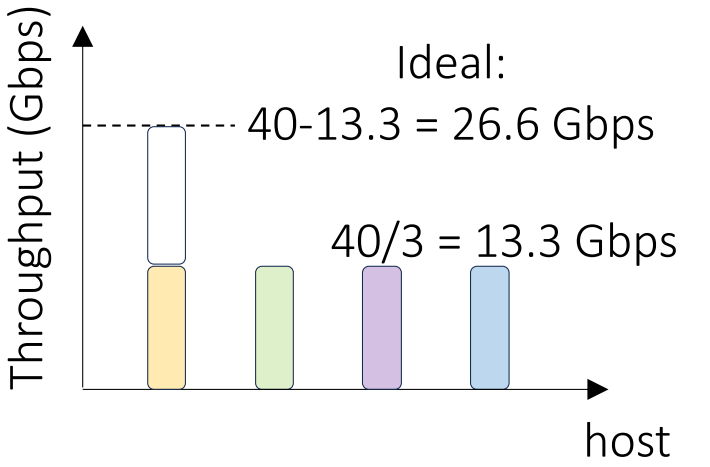


Why?

Victim Flow Problem



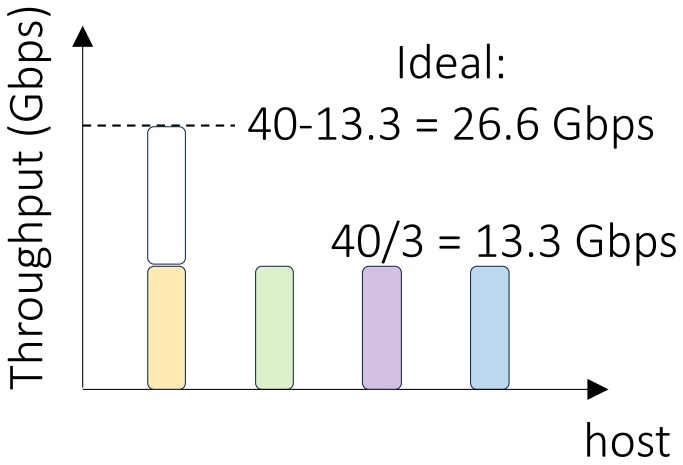
Assuming 40Gbps links



Why?

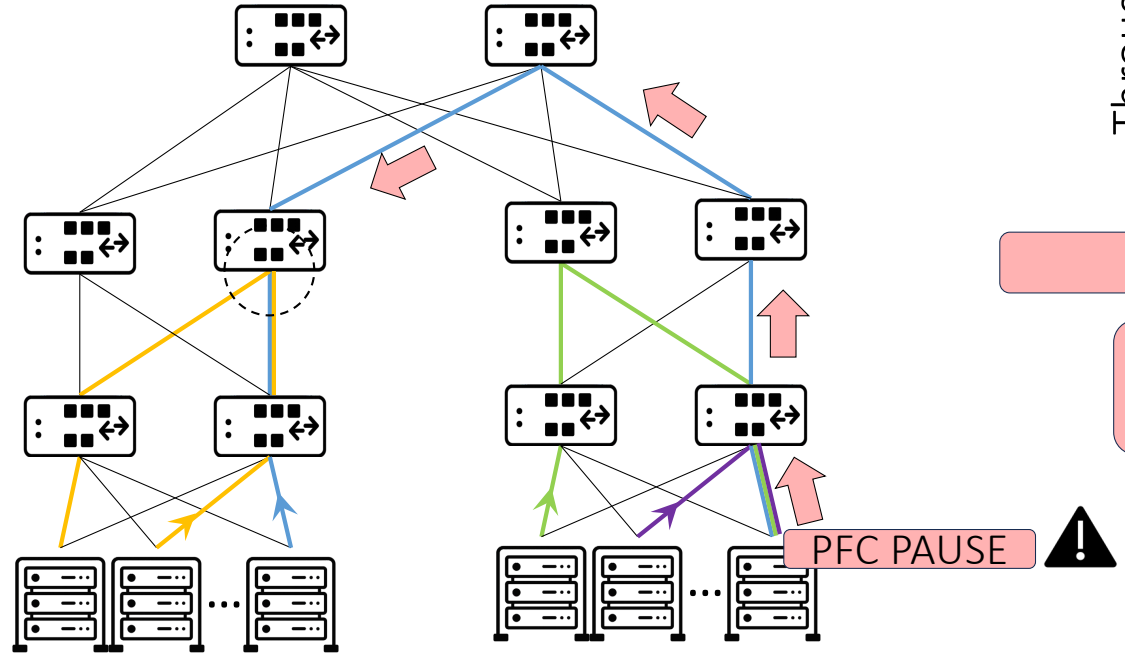
Victim Flow Problem

Assuming 40Gbps links



Why?

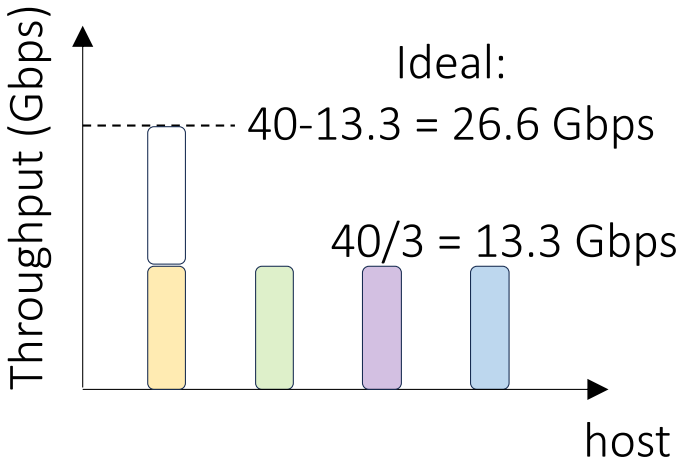
The blue and yellow flows receive the same treatment



Cascading link PAUSEing

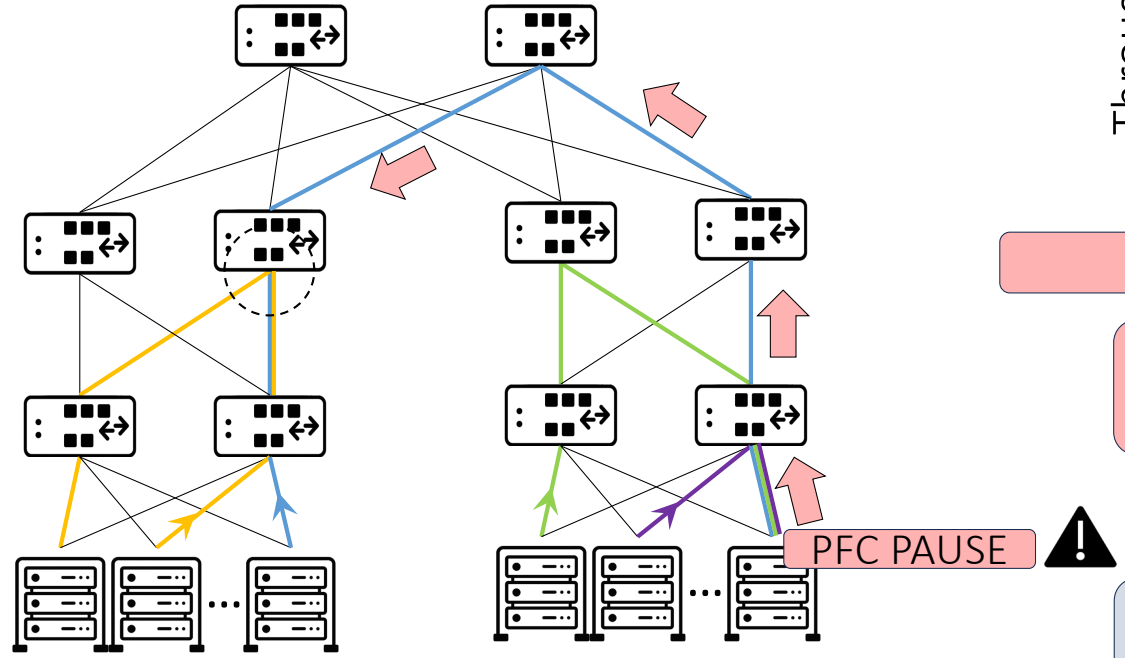
Victim Flow Problem

Assuming 40Gbps links



Why?

The blue and yellow flows receive the same treatment

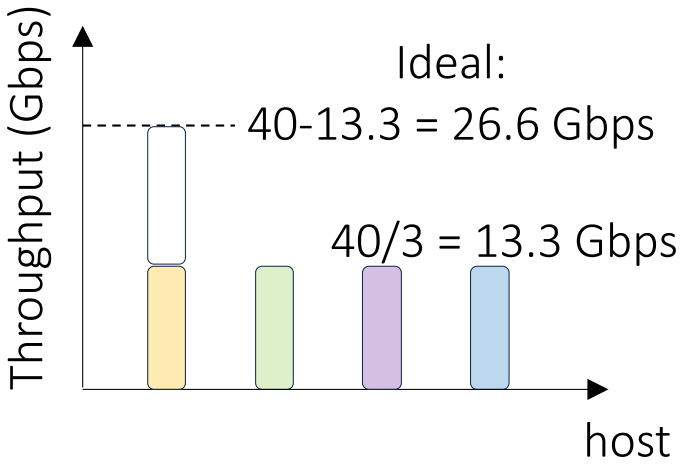


PFC operates on *port-level*, instead of *flow-level*

Cascading link PAUSEing

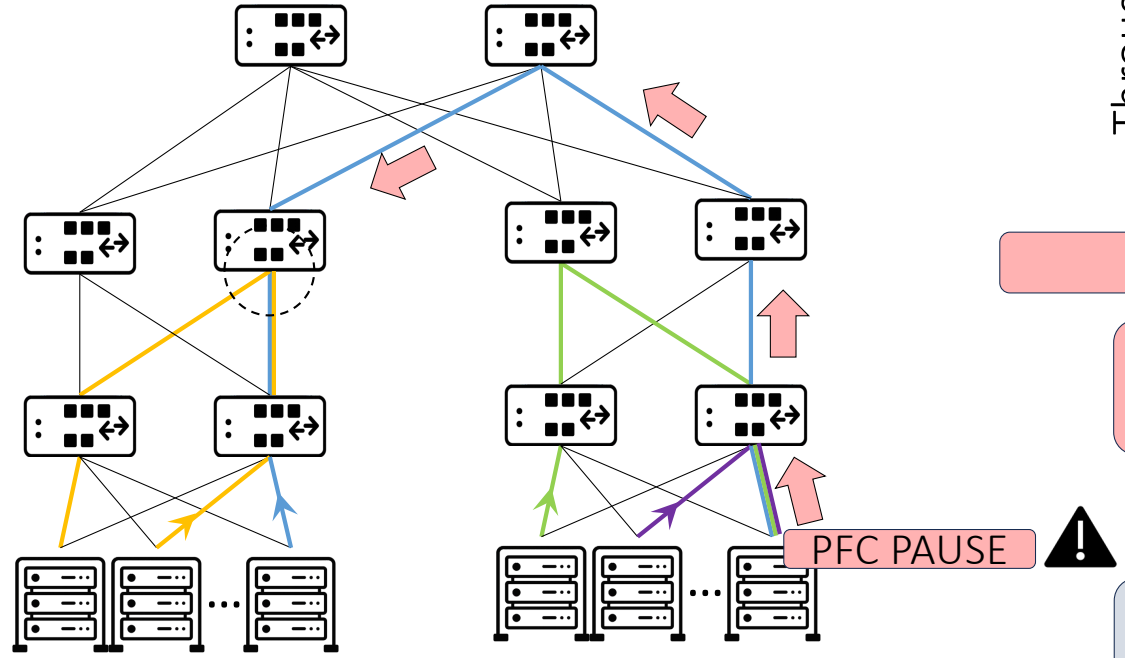
Victim Flow Problem

Assuming 40Gbps links



Why?

The blue and yellow flows receive the same treatment

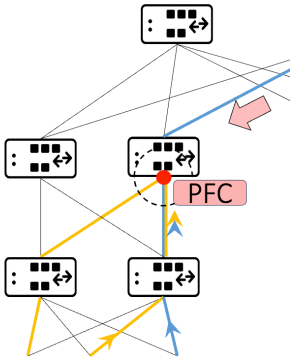


Cascading link PAUSEing

PFC operates on *port-level*, instead of *flow-level*

Yellow flow is victimized because it has nothing to do with congestion

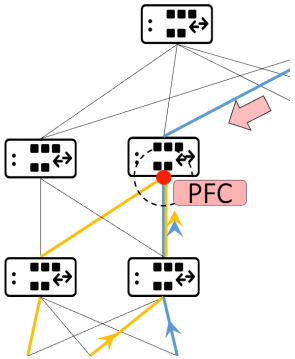
Drawbacks of PFC



Yellow flow is blocked albeit congestion is in a different part of the network

- Congestion spreading
  - it pauses all traffic on a link, including uncongested flows, without distinguishing between them

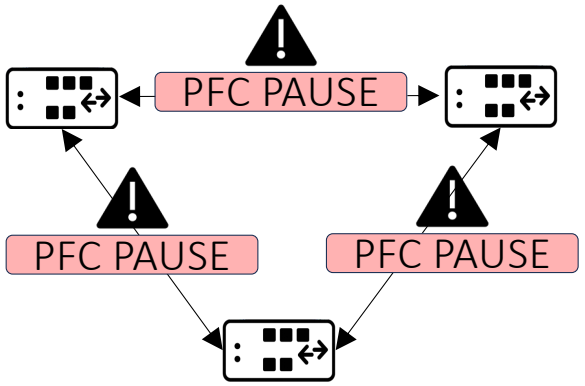
Drawbacks of PFC



Yellow flow would use a different output port

- Congestion spreading
  - it pauses all traffic on a link, including uncongested flows, without distinguishing between them
- Unfairness and head-of-line blocking
  - As packets build up, the input port sends PFC pause frames to stop the sender on that link. However, this also blocks other flows that are trying to use a different output port

Drawbacks of PFC



- Congestion spreading
  - it pauses all traffic on a link, including uncongested flows, without distinguishing between them
- Unfairness and head-of-line blocking
  - As packets build up, the input port sends PFC pause frames to stop the sender on that link. However, this also blocks other flows that are trying to use a different output port
- It can lead to deadlocks
  - They occur when devices pause traffic indefinitely due to cyclic buffer dependencies, with each waiting for buffer space held by another

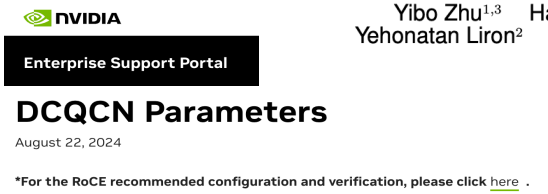
Solution?

- Use one priority per flow ☺

Solution?

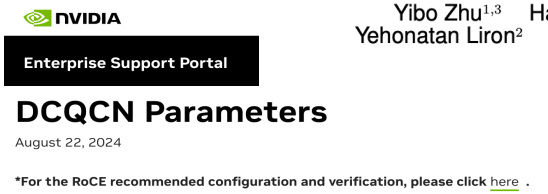
- Use one priority per flow ☺
- We cannot do it as it does not scale: most ASICs support 8 queues only ☹

Datacenter QCN (DCQCN)



- A solution proposed by Microsoft and running in production in their datacenters (…and not only there)
- Built to work with legacy switches on top of the commodity ECN feature + PFC
- Work presented at ACM SIGCOMM 2015 (London)

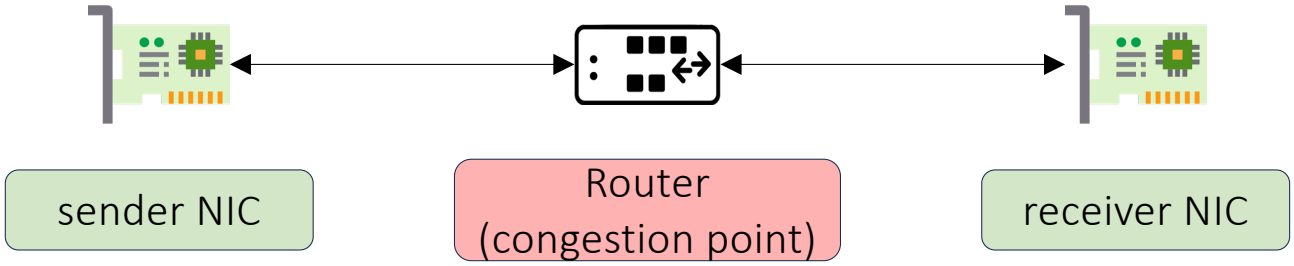
Datacenter QCN (DCQCN)



Leverage per-flow congestion control and use PFC as last resort!

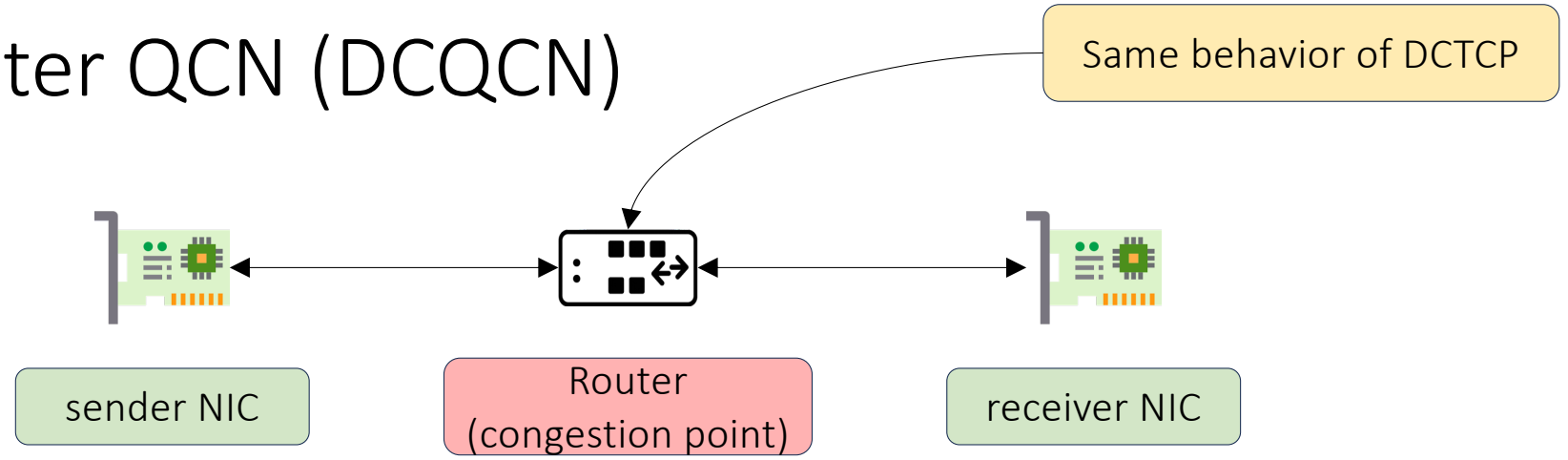
- A solution proposed by Microsoft and running in production in their datacenters (…and not only there)
- Built to work with legacy switches on top of the commodity ECN feature + PFC
- Work presented at ACM SIGCOMM 2015 (London)

Datacenter QCN (DCQCN)

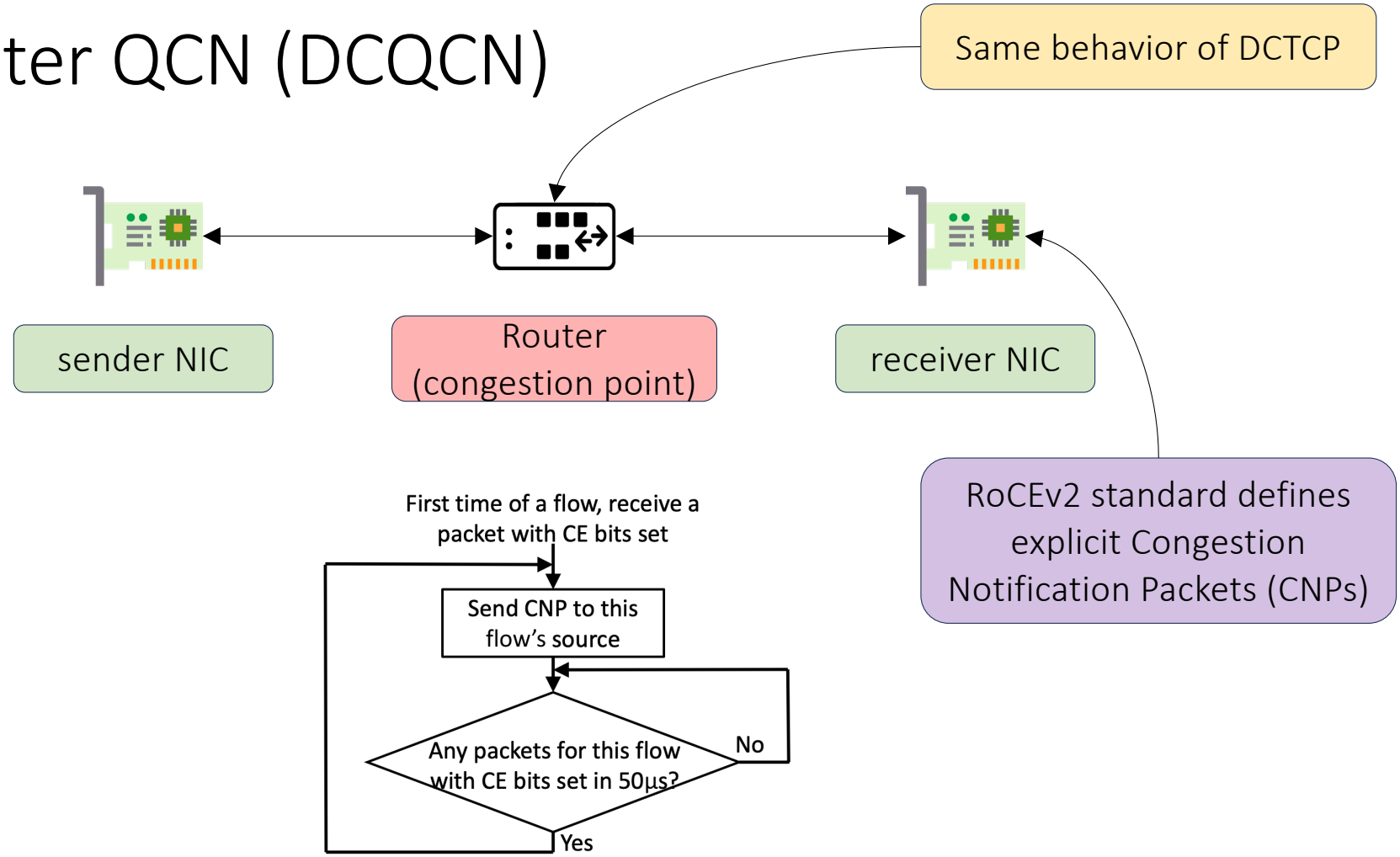


- Router marks ECN based on queue length
- Receiver NIC reflects ECN marks back to the sender NIC
- Sender NIC adjusts sending rate based on information from receiver NIC

Datacenter QCN (DCQCN)

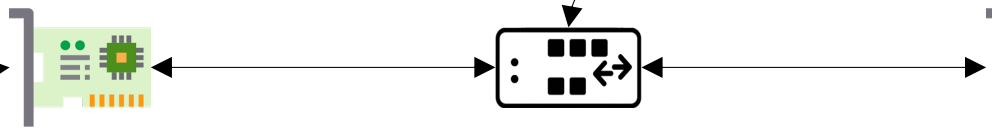


Datacenter QCN (DCQCN)



Datacenter QCN (DCQCN)

Same behavior of DCTCP



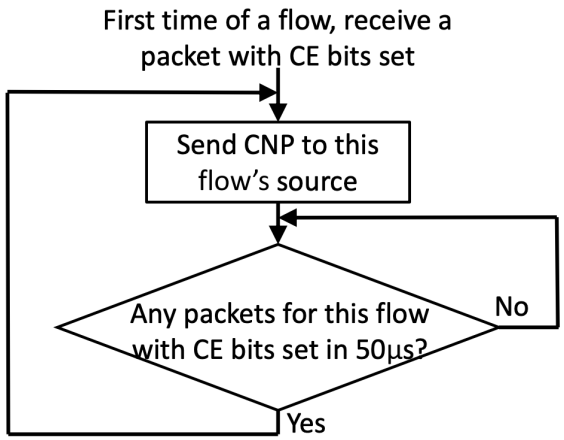
sender NIC

Router (congestion point)

receiver NIC

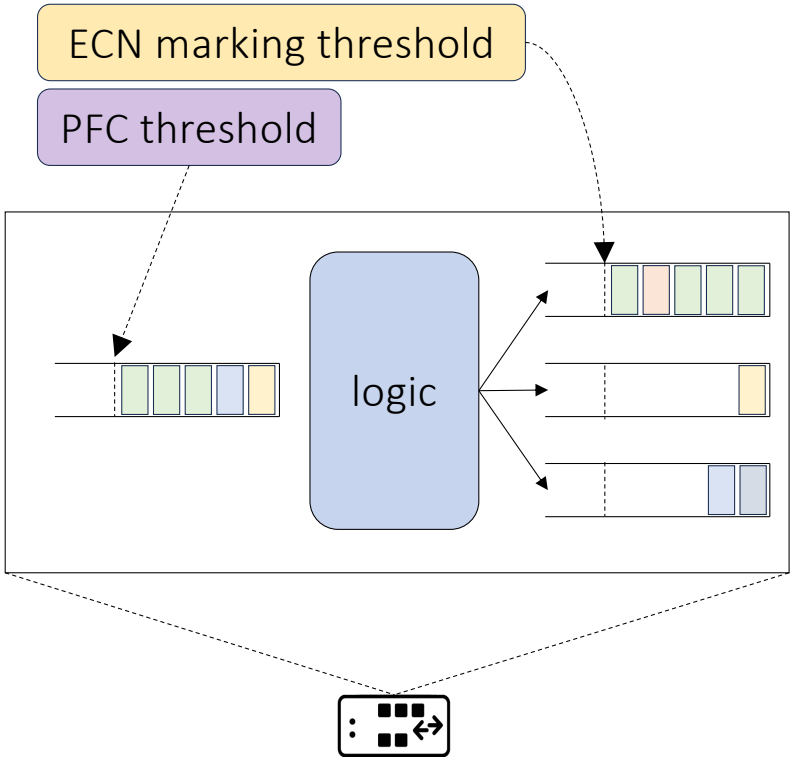
Upon receipt of a CNP, the flow rate is reduced by a factor alpha, similarly to DCTCP

No CNP received, the sender increases its sending rate

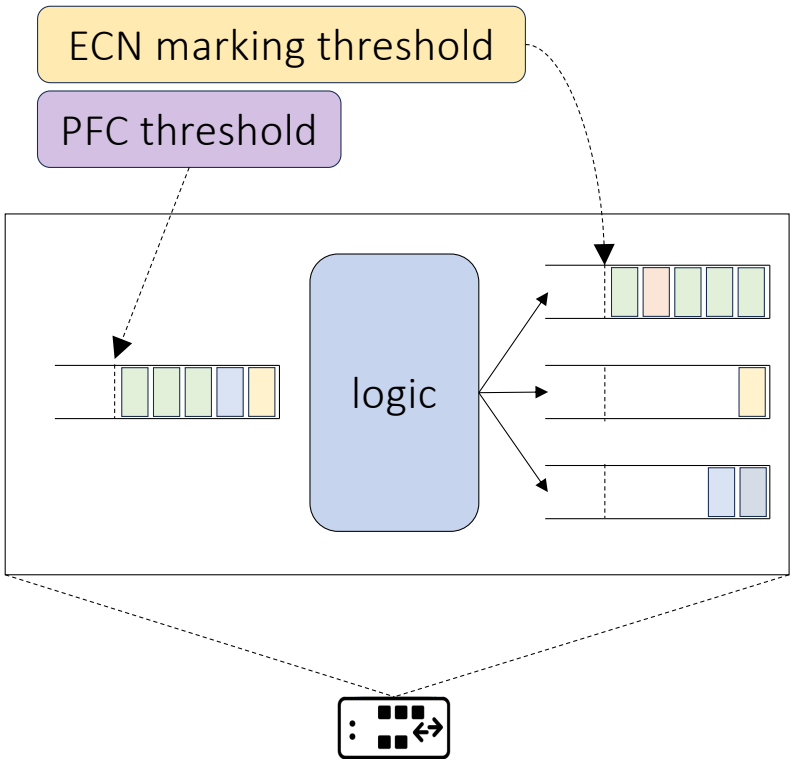


RoCEv2 standard defines explicit Congestion Notification Packets (CNPs)

Easier said than done: router buffer tuning

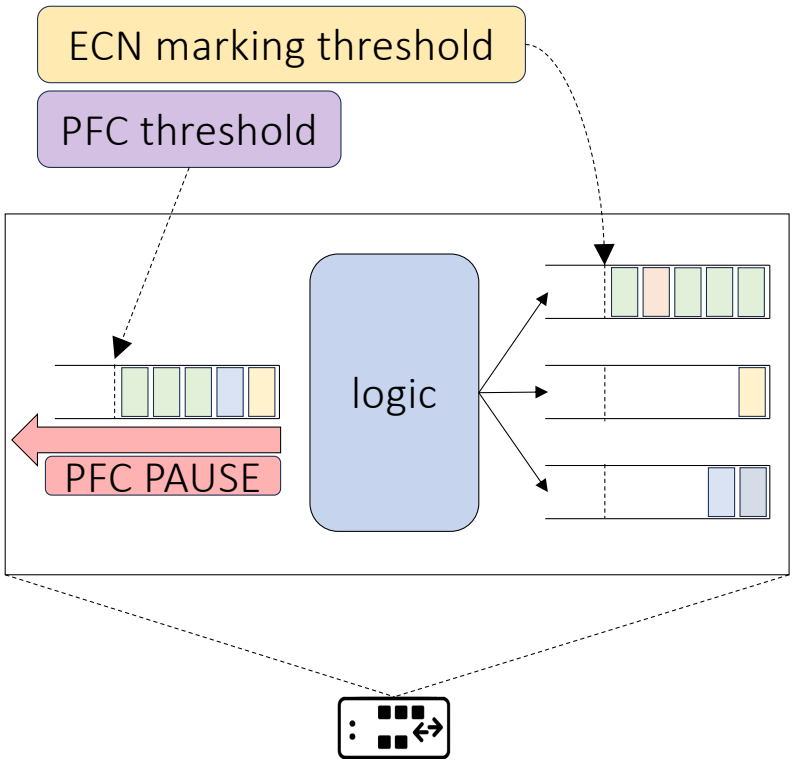


Easier said than done: router buffer tuning



Problem: ECN marked on egress, PFC on ingress

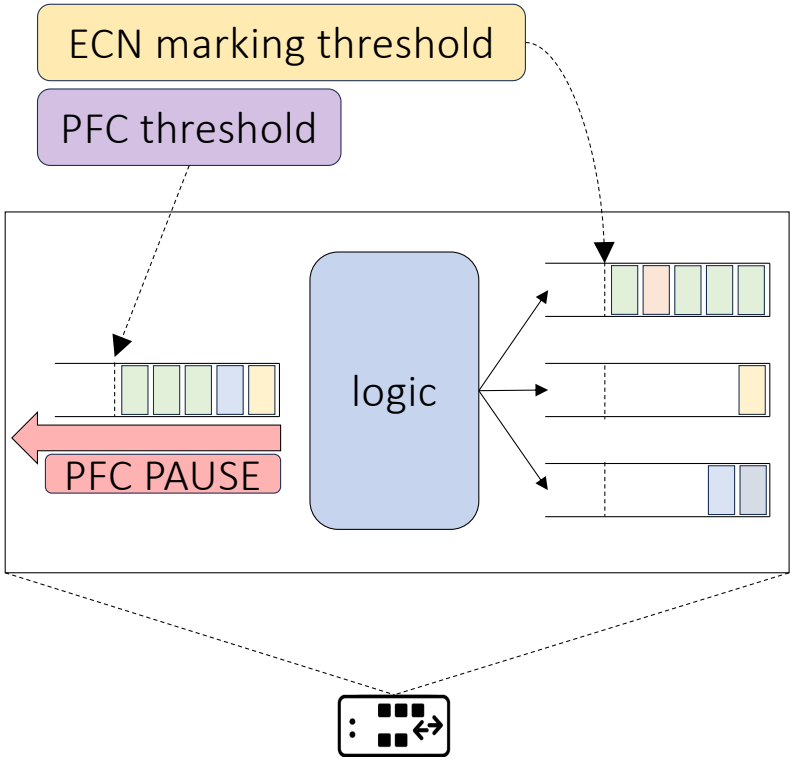
Easier said than done: router buffer tuning



Problem: ECN marked on egress, PFC on ingress

Goal: we shall mark ECN before firing PFC

Easier said than done: router buffer tuning

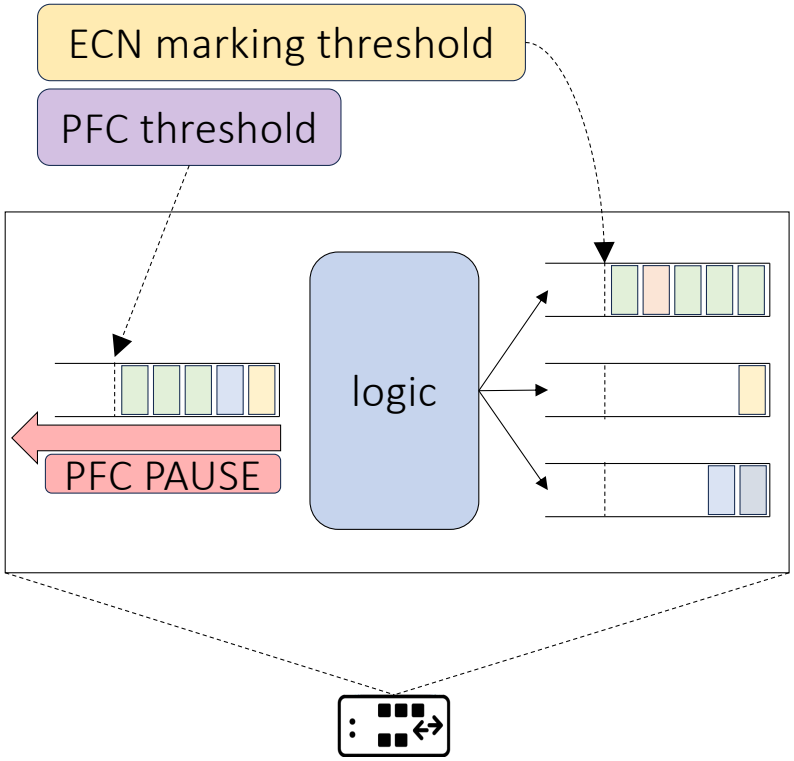


Problem: ECN marked on egress, PFC on ingress

Goal: we shall mark ECN before firing PFC

Finding the right configuration of threshold is hard!

Easier said than done: router buffer tuning



Problem: ECN marked on egress, PFC on ingress

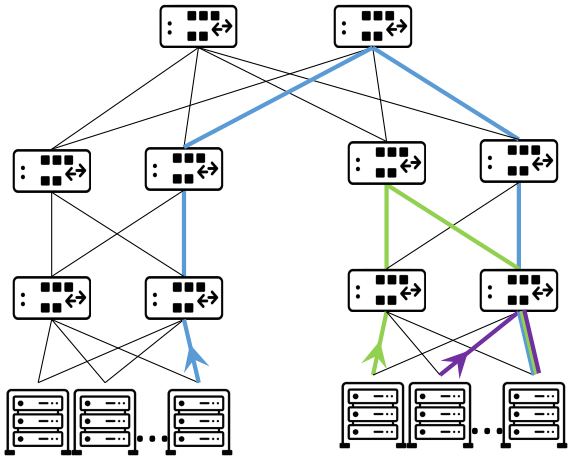
Goal: we shall mark ECN before firing PFC

Finding the right configuration of threshold is hard!

The paper provides a detailed analysis about this

The analysis does not eliminate PFC, but ensures ECN is triggered first; PFC may still occur while senders react to ECN feedback

DCQCN in action

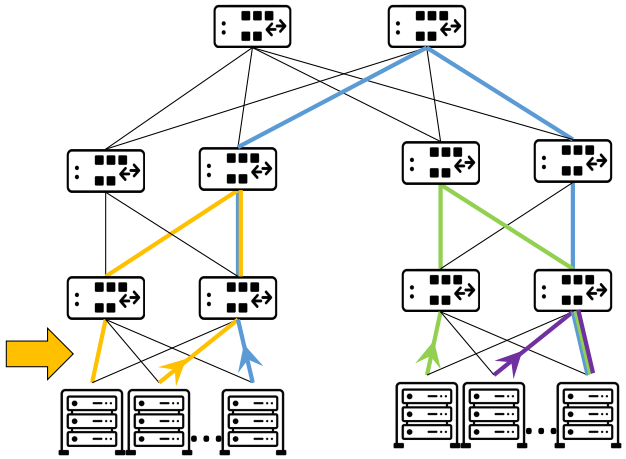


40Gbps links with ECMP load balancing

Workload:

- each host communicates with one or more randomly selected hosts, and transfers data using distributions derived from real traffic traces
- Single disk rebuild event with a varying degree of incast from 2 to 10

DCQCN in action



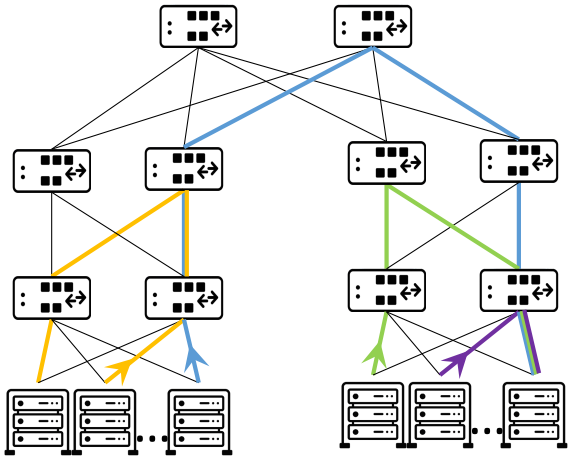
40Gbps links with ECMP load balancing

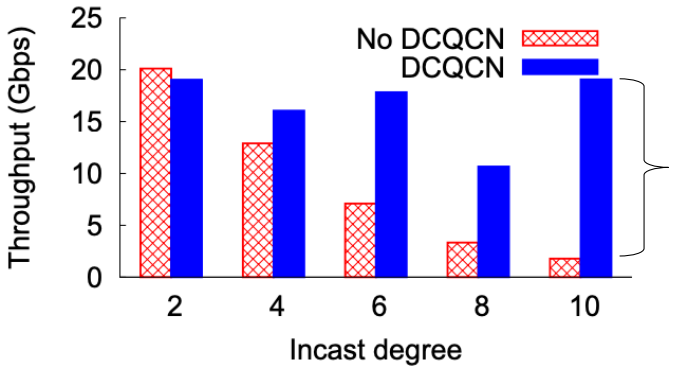
Idea: assess the case of the victim flow example from before

Workload:

- each host communicates with one or more randomly selected hosts, and transfers data using distributions derived from real traffic traces.
- Single disk rebuild event with a varying degree of incast from 2 to 10

DCQCN in action





This is the PFC damage

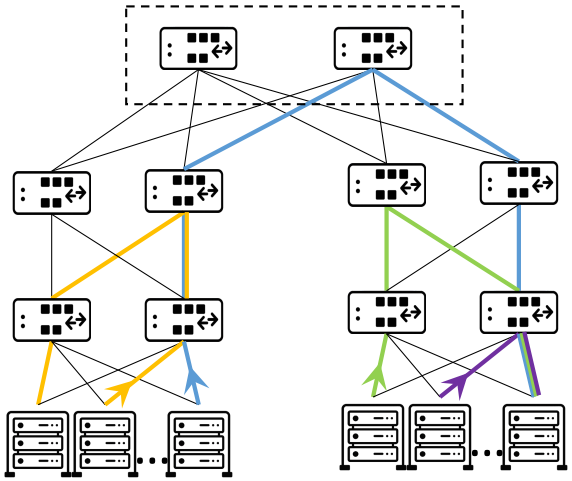
40Gbps links with ECMP load balancing

Idea: assess the case of the victim flow example from before

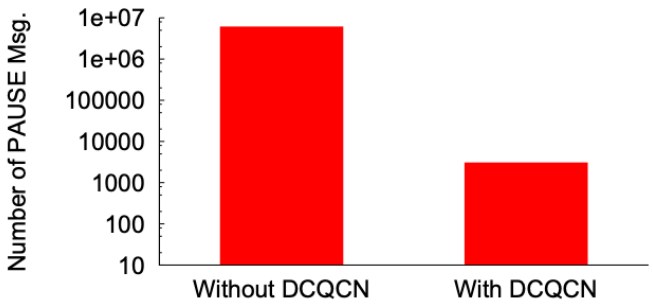
Workload:

- each host communicates with one or more randomly selected hosts, and transfers data using distributions derived from real traffic traces.
- Single disk rebuild event with a varying degree of incast from 2 to 10

DCQCN in action



Number of PFC PAUSE messages received at the core



40Gbps links with ECMP load balancing

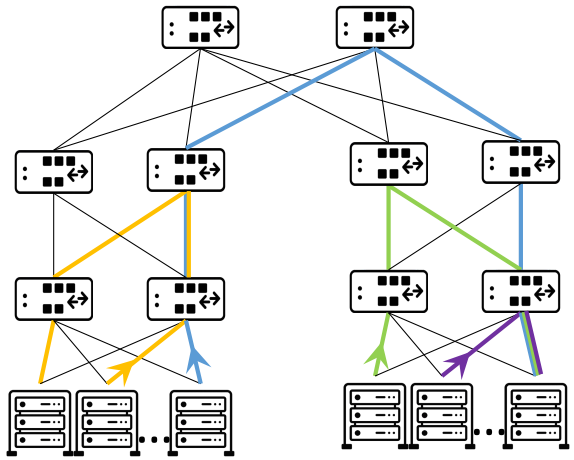
Idea: assess the case of the victim flow example from before

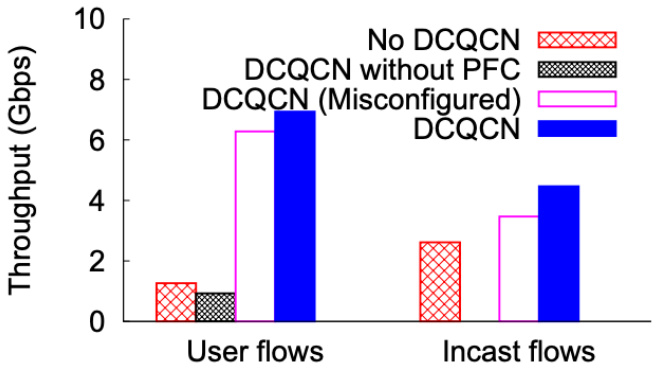
Workload:

- each host communicates with one or more randomly selected hosts, and transfers data using distributions derived from real traffic traces.
- Single disk rebuild event with a varying degree of incast from 2 to 10

DCQCN in action

8:1 incast
Misconfigured: PFC and ECN thresholds are wrong





40Gbps links with ECMP load balancing

Idea: assess the case of the victim flow example from before

Workload:

- each host communicates with one or more randomly selected hosts, and transfers data using distributions derived from real traffic traces.
- Single disk rebuild event with a varying degree of incast from 2 to 10

RDMA is gaining traction!

RDMA over Commodity Ethernet at Scale

Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni,
Jianxi Ye, Jitendra Padhye, Marina Lipshteyn
Microsoft
{chguo, hwu, zdeng, gasoni, jiye, padhye, malipsht}@microsoft.com

ABSTRACT

Over the past one and half years, we have been using RDMA over commodity Ethernet (RoCEv2) to support some of Microsoft's highly-reliable, latency-sensitive services. This paper describes the challenges we encountered during the process and the solutions we devised to address them. In order to scale RoCEv2 beyond VLAN,

Ethernet switches and network interface cards (NICs) A state-of-the-art DCN must support several Gb/s o higher throughput between any two servers in a DC.

TCP/IP is still the dominant transport/network stac in today's data center networks. However, it is increas ingly clear that the traditional TCP/IP stack canno meet the demands of the new generation of DC work

Empowering Azure Storage with RDMA

Wei Bai, Shanim Sainul Abdeen, Ankit Agrawal, Krishan Kumar Attre, Paramvir Bahl, Ameya Bhagat, Gowri Bhaskara, Tanya Brokhman, Lei Cao, Ahmad Cheema, Rebecca Chow, Jeff Cohen, Mahmoud Elhaddad, Vivek Ette, Igal Figlin, Daniel Firestone, Mathew George, Ilya German, Lakhmeet Ghai, Eric Green, Albert Greenberg*, Manish Gupta, Randy Haagens, Matthew Hendel, Ridwan Howlader, Neetha John, Julia Johnstone, Tom Jolly, Greg Kramer, David Kruse, Ankit Kumar, Erica Lan, Ivan Lee, Avi Levy, Marina Lipshteyn, Xin Liu, Chen Liu*, Guohan Lu, Yuemin Lu, Xiakun Lu, Vadim Makhervaks, Ulad Malashanka, David A. Maltz, Ilias Marinos, Rohan Mehta, Sharda Murthi, Anup Namdhari, Aaron Ogus, Jitendra Padhye, Madhav Pandya, Douglas Phillips, Adrian Power, Suraj Puri, Shachar Raindel*, Jordan Rhee*, Anthony Russo, Maneesh Sah, Ali Sheriff, Chris Sparacino, Ashutosh Srivastava, Weixiang Sun*, Nick Swanson, Fuhou Tian, Lukasz Tomczyk, Vamsi Vadlamuri, Alec Wolman, Ying Xie, Joyce Yom, Lihua Yuan, Yanzhao Zhang, Brian Zill
Microsoft

Abstract

Given the wide adoption of disaggregated storage in public clouds, networking is the key to enabling high performance and high reliability in a cloud storage service. In Azure, we choose Remote Direct Memory Access (RDMA) as our transport and aim to enable it for both storage frontend traffic (between compute virtual machines and storage clusters) and backend traffic (within a storage cluster) to fully realize its benefits. As compute and storage clusters may be located in different datacenters within an Azure region, we need to support RDMA at regional scale.

This work presents our experience in deploying intra-region RDMA to support storage workloads in Azure. The high complexity and heterogeneity of our infrastructure bring a series of new challenges, such as the problem of interoperability between different types of RDMA network interface cards.

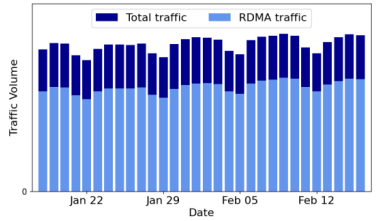


Figure 1: Traffic statistics of all Azure public regions between January 18 and February 16, 2023. Traffic was measured by collecting switch counters of server-facing ports on all Top of Rack (ToR) switches. Around 70% of traffic was RDMA.

1RMA: Re-envisioning Remote Memory Access for Multi-tenant Datacenters

Arjun Singhvi[†‡] Aditya Akella[†‡] Dan Gibson[‡] Thomas F. Wenisch[‡] Monica Wong-Chan[‡]
Sean Clark[‡] Milo M. K. Martin[‡] Moray McLaren[‡] Prashant Chandra[‡] Rob Cauble[‡]
Hassan M. G. Wassel[‡] Behnam Montazeri[‡] Simon L. Sabato[*] Joel Scherpelz[°]
Amin Vahdat[‡]
[†]*University of Wisconsin - Madison* [‡]*Google Inc* [*]*Lilac Cloud* [°]*Unaffiliated*

Abstract

Remote Direct Memory Access (RDMA) plays a key role in supporting performance-hungry datacenter applications. However, existing RDMA technologies are ill-suited to multi-tenant datacenters, where applications run at massive scales, tenants require isolation and se-

and machine learning, demand networks that support high bandwidth and operation (op) rates while achieving low tail latencies. Remote Direct Memory Access (RDMA) is an attractive option for such distributed systems because of the latency and op-rate benefits provided by one-sided reads and writes, as these ops involve no

When Cloud Storage Meets RDMA

consumption, thus

t Memory Access
3y offloading the

Yixiao Gao[♠♡], Qiang Li[♡], Lingbo Tang[♡], Yongqing Xi[♡], Pengcheng Zhang[♡], Wenwen Peng[♡], Bo Li[♡],
Yaohui Wu[♡], Shaozong Liu[♡], Lei Yan[♡], Fei Feng[♡], Yan Zhuang[♡], Fan Liu[♡], Pan Liu[♡], Xingkui Liu[♡],
Zhongjie Wu[♡], Junping Wu[♡], Zheng Cao[♡], Chen Tian[♠], Jinbo Wu[♡], Jiaji Zhu[♡], Haiyong Wang[♡], Dennis
Cai[♡], and Jiesheng Wu[♡]

[♠]Nanjing University, [♡]Alibaba Group

Abstract

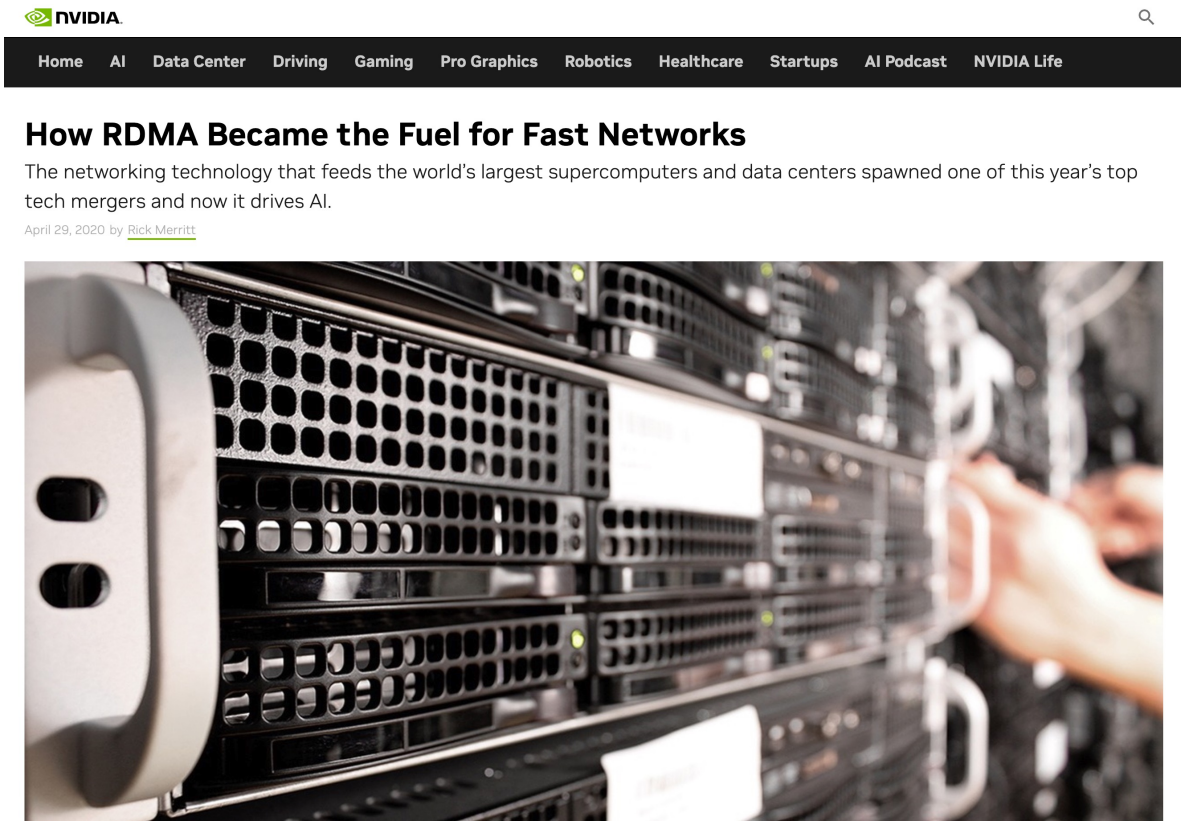
A production-level cloud storage system must be high performing and readily available. It should also meet a Service-Level Agreement (SLA). The rapid advancement in storage media has left networking lagging behind, resulting in a major performance bottleneck for new cloud storage generations. Remote Direct Memory Access (RDMA) running on lossless fabrics can potentially overcome this bottleneck. In this paper, we present our experience in introducing RDMA into the storage networks of Pangu, a cloud storage system developed by Alibaba. Since its introduction in 2009, it has proven to be

(i) *High performance:* Small latency and high throughpu provide competitive advantages across many scenarios

(ii) *High availability:* System disruptions incur significan financial/reputation loss for both tenants and their cloud providers.

(iii) *Service-Level Agreement (SLA):* A cloud storage system must be resilient, and thus its performance should gracefully downgrade when various software/hardware failures happen.

RDMA is important in today's networks!



https://blogs.nvidia.com/blog/what-is-rdma/

Many use-cases!

- key-value stores acceleration
- distributed file systems
- NVMe over Fabric
- search queries

Fast RDMA-based Ordered Key-Value Store using Remote Learned Cache

Xingda Wei, Rong Chen, Haibo Chen
Engineering Research Center for Domain-specific Operating Systems, Ministry of Education, China
Institute of Parallel and Distributed Systems, Shanghai Jiao Tong University

FaRM: Fast Remote Memory

Aleksandar Dragojević, Dushyanth Narayanan, Orion Hodson, Miguel Castro

Microsoft Research

Efficient Crash Consistency for NVMe over PCIe and RDMA

XIAOJIAN LIAO, YOUYOU LU, ZHE YANG, and JIWU SHU, Tsinghua University, China

PRISM: Rethinking the RDMA Interface for Distributed Systems

Matthew Burke
Cornell University
United States
matthelb@cs.cornell.edu

Sowmya Dharanipragada
Cornell University
United States
sjd266@cornell.edu

Shannon Joyner
Cornell University
United States
saj9191@gmail.com

Adriana Szekeres
VMware Research
United States
aaasz@cs.washington.edu

Jacob Nelson
Microsoft Research
United States
jacob.nelson@microsoft.com

Irene Zhang
Microsoft Research
United States
irene.zhang@microsoft.com

Dan R. K. Ports
Microsoft Research
United States
dan@drkp.net

Orion: A Distributed File System for Non-Volatile Main Memories and RDMA-Capable Networks

Jian Yang Joseph Izraelevitz Steven Swanson
UC San Diego
{jianyang, jizraelevitz, swanson}@eng.ucsd.edu

Octopus: an RDMA-enabled Distributed Persistent Memory File System

Youyou Lu
Tsinghua University

Jiwu Shu*
Tsinghua University

Youmin Chen
Tsinghua University

Tao Li
University of Florida

Lots of research around RDMA

Novel congestion controls (competitors of DCQCN)

TIMELY: RTT-based Congestion Control for the Datacenter

Radhika Mittal*(UC Berkeley), Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel,
Monia Ghobadi*(Microsoft), Amin Vahdat, Yaogong Wang, David Wetherall, David Zats

Google, Inc.

SwCC: Software-Programmable and Per-Packet Congestion Control in RDMA Engine

Hongjing Huang†, Jie Zhang†, Xuzheng Chen, Ziyu Song, Jiajun Qin, Zeke Wang*
Department of Computer Science, Zhejiang University, China

Rethinking Intra-host Congestion Control in RDMA Networks

Zirui Wan[1,2], Jiao Zhang[1,3,*], Yuxiang Wang[1], Kefei Liu[1], Haoyu Pan[1], Tao Huang[1,3]
State Key Laboratory of Networking and Switching Technology, BUPT, China[1]
China Mobile (Suzhou) Software Technology Co., Ltd[2]
Purple Mountain Laboratories[3]

HPCC: High Precision Congestion Control

Yuliang Li♠♡, Rui Miao♠, Hongqiang Harry Liu♠, Yan Zhuang♠, Fei Feng♠, Lingbo Tang♠, Zheng Cao♠, Ming Zhang♠,
Frank Kelly♢, Mohammad Alizadeh♣, Minlan Yu♡
Alibaba Group♠, Harvard University♡, University of Cambridge♢, Massachusetts Institute of Technology♣

Lots of research around RDMA

Improved NIC designs for better loss recovery mechanisms

NVIDIA Docs Hub

Out-of-Order (OOO) Data Placement Experimental Verbs

Overview

In certain fabric configurations, InfiniBand packets for a given QP may take up different paths in a network from source to destination. This results into packets being received in an out-of-order manner. These packets can now be handled instead of being dropped, in order to avoid retransmission, by:

- Achieving better network utilization
- Decreasing latency

Data will be placed into host memory in an out-of-order manner when out-of-order messages are received.

US20180004705A1
United States

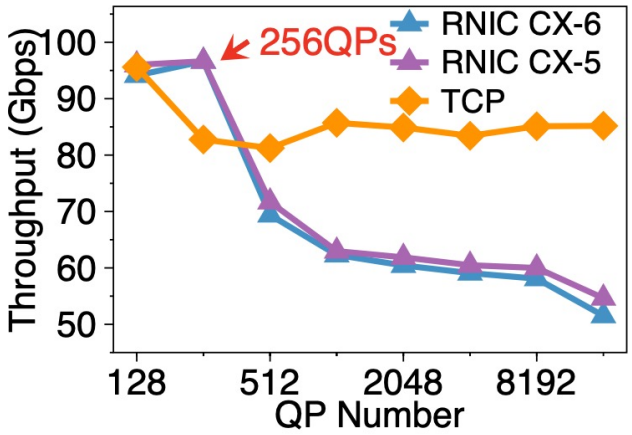
Selective acknowledgment of RDMA packets

Abstract

A method for data transfer includes transmitting a sequence of data packets, including at least a first packet and a second packet transmitted subsequently to the first packet, from a first computer over a network to a second computer in a single remote direct memory access (RDMA) data transfer transaction. Upon receipt of the second packet at the second computer without previously having received the first packet, a negative acknowledgment (NAK) packet is sent from the second computer over the network to the first computer, indicating that the first packet was not received. In response to the NAK packet, the first packet is retransmitted from the first computer to the second computer without retransmitting the second packet.

Lots of research around RDMA

Improved NIC designs for higher scalability

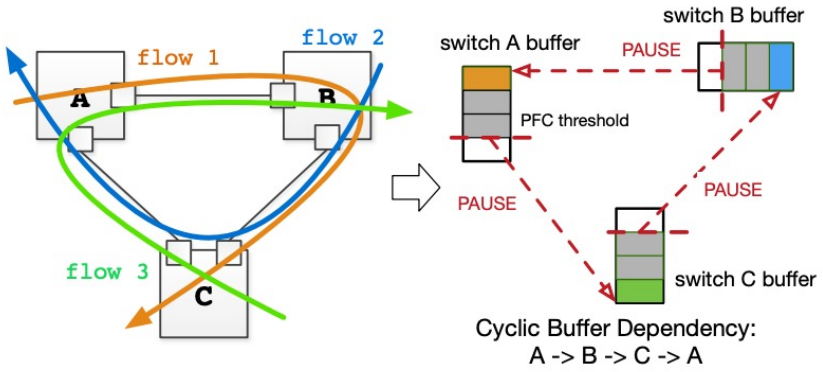


SRNIC: A Scalable Architecture for RDMA NICs

Zilong Wang[1*] Layong Luo[2] Qingsong Ning[2] Chaoliang Zeng[1*] Wenxue Li[1] Xinchen Wan[1*]
Peng Xie[2] Tao Feng[2] Ke Cheng[2] Xiongfei Geng[2] Tianhao Wang[2] Weicheng Ling[2]
Kejia Huo[2] Pingbo An[2] Kui Ji[2] Shideng Zhang[2] Bin Xu[2] Ruiqing Feng[2] Tao Ding[2]
Kai Chen[1] Chuanxiong Guo[3]
[1]*Hong Kong University of Science and Technology* [2]*ByteDance* [3]*Unaffiliated*

Lots of research around RDMA

Better PFC handling

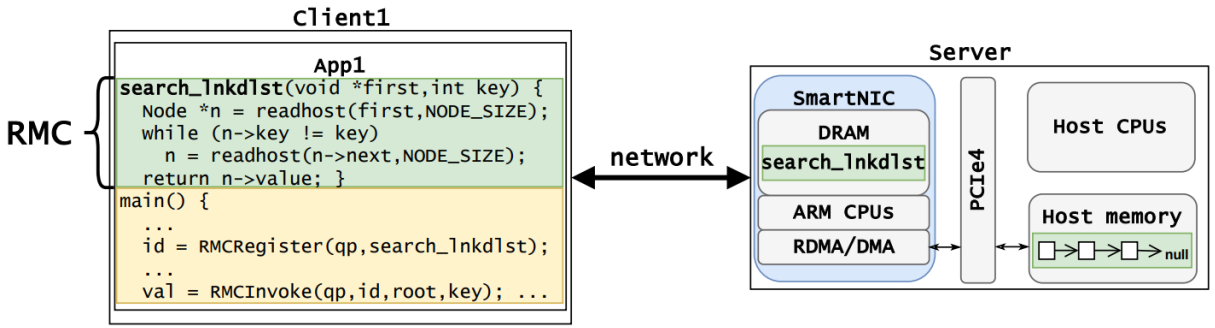


Tagger: Practical PFC Deadlock Prevention in Data Center Networks

Shuihai Hu[1,2], Yibo Zhu[1], Peng Cheng[1], Chuanxiong Guo[1]
Kun Tan[1*], Jitendra Padhye[1], Kai Chen[2]
[1] Microsoft [2]Hong Kong University of Science and Technology

Lots of research around RDMA

New RDMA primitives



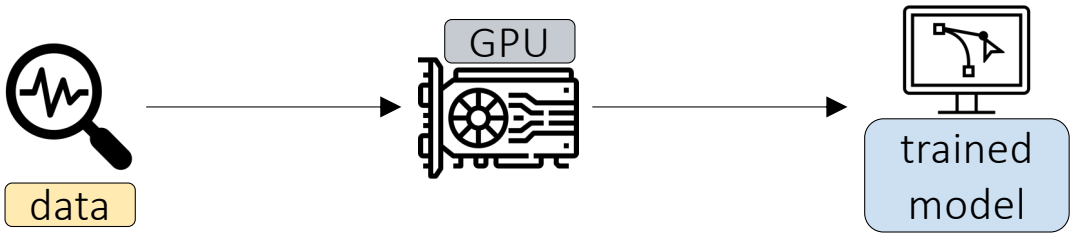
Remote Memory Calls

Emmanuel Amaro★ Zhihong Luo★ Amy Ousterhout★ Arvind Krishnamurthy°
Aurojit Panda† Sylvia Ratnasamy★ Scott Shenker★‡
★ UC Berkeley ° University of Washington † NYU ‡ ICSI

Why so much interest?

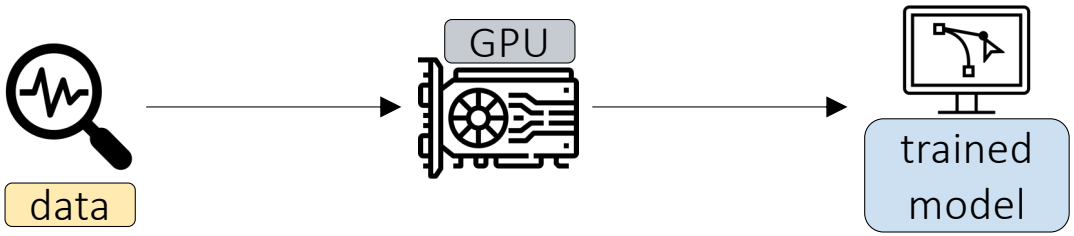
Why so much interest?

training a simple model

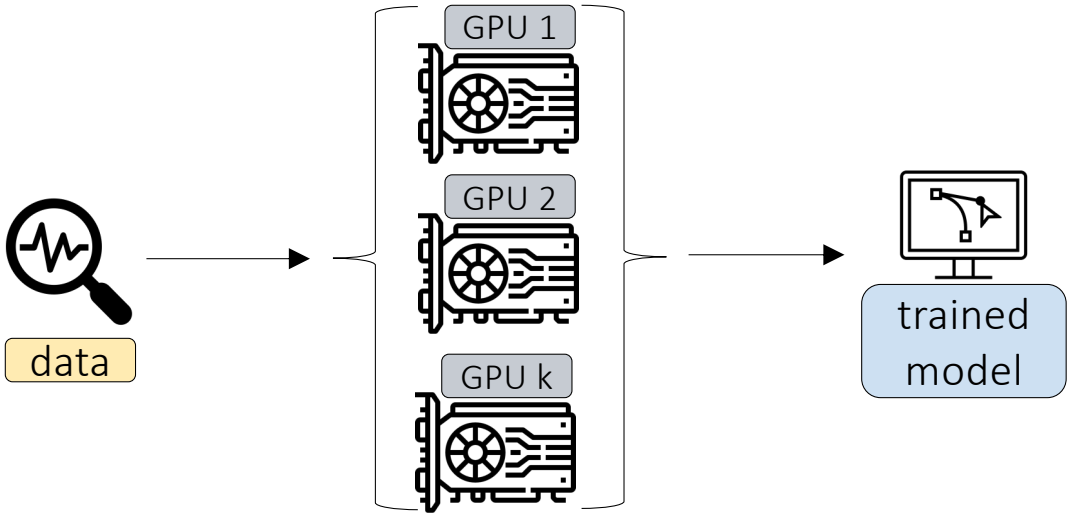


Why so much interest?

training a simple model



training a complex model

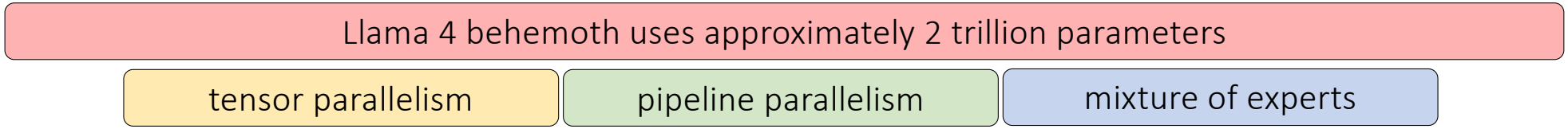


Problems of scale

- Way too many parameters to hold in one GPU

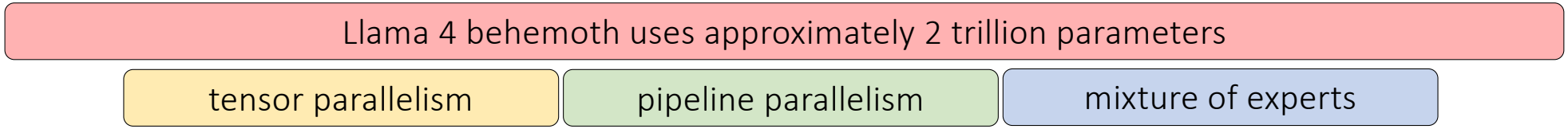
Problems of scale

- Way too many parameters to hold in one GPU



Problems of scale

- Way too many parameters to hold in one GPU



- Way too much data (all the Internet): it takes forever to train if we feed it sequentially

Problems of scale

- Way too many parameters to hold in one GPU

Llama 4 behemoth uses approximately 2 trillion parameters

tensor parallelism | pipeline parallelism | mixture of experts

- Way too much data (all the Internet): it takes forever to train if we feed it sequentially

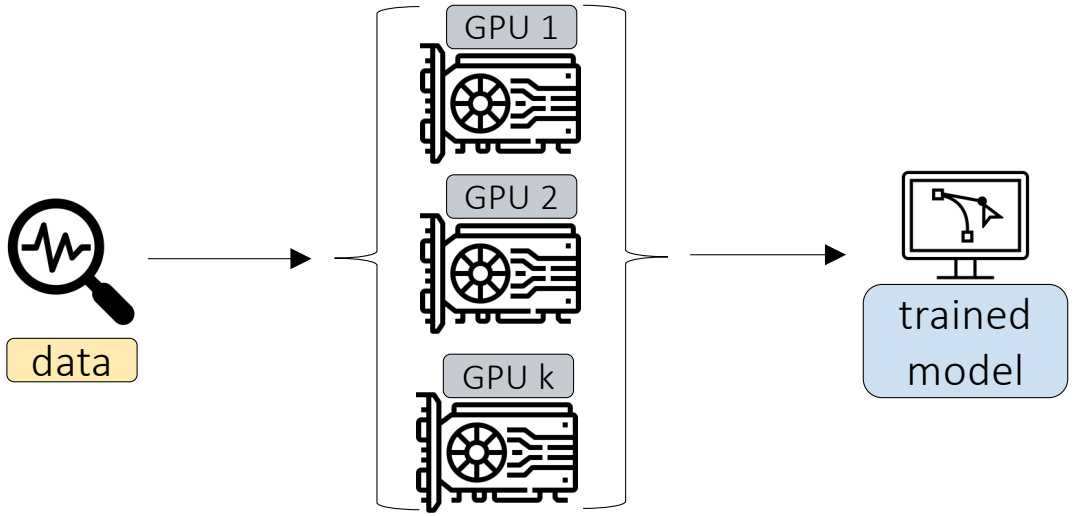
Llama 4 trained on more than 30 trillion tokens

data parallelism

Communication patterns

GPUs work towards a common goal

They cannot work independently: they need to keep their state synchronized during training



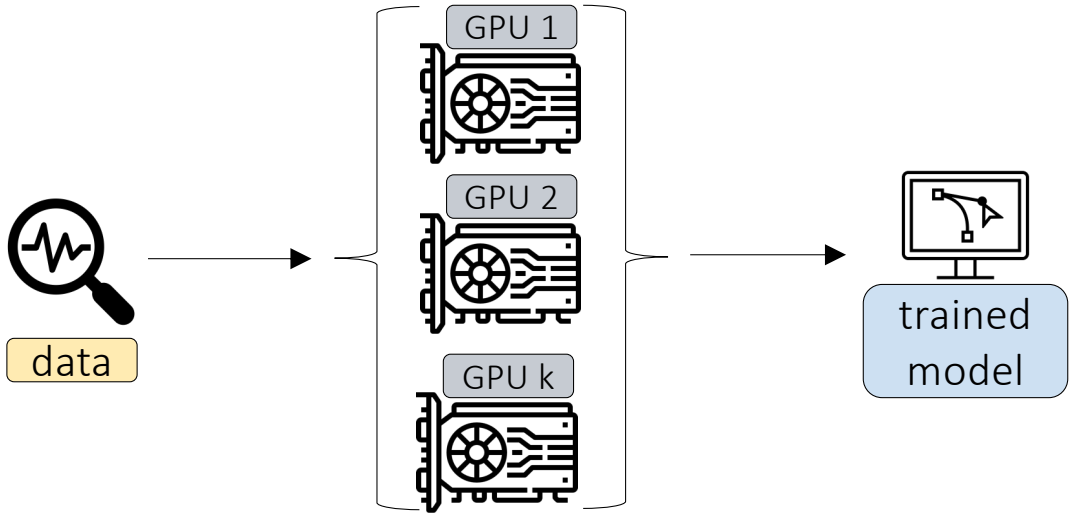
Communication patterns

GPUs work towards a common goal

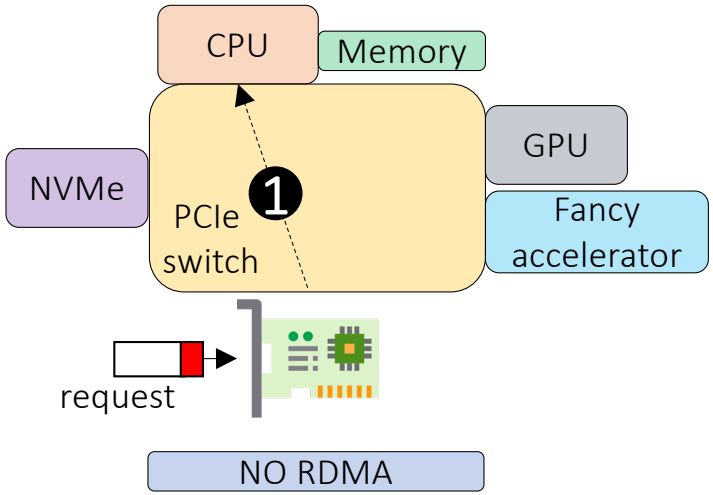
They cannot work independently: they need to keep their state synchronized during training

Consequence: transferring large amount of data between them very often

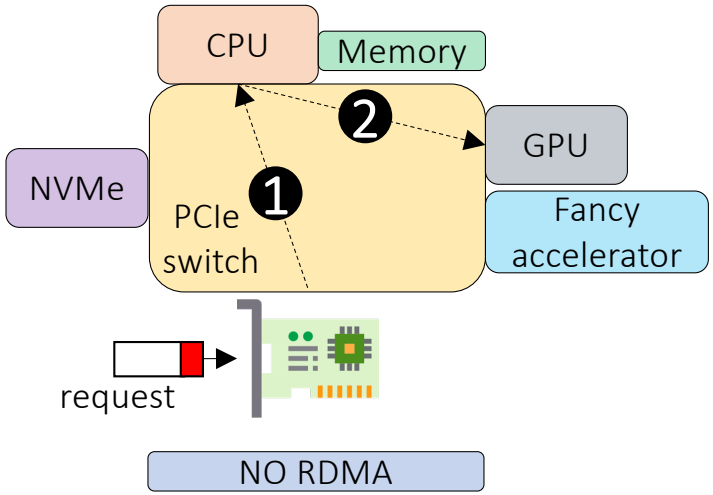
We need a performant network!



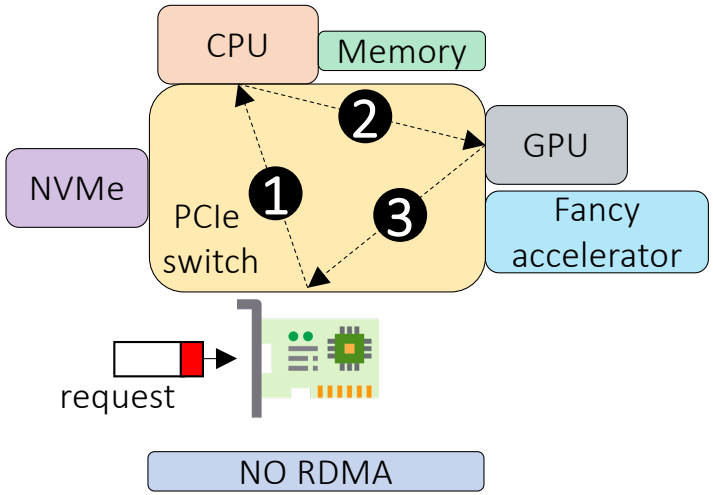
GPU networking example



GPU networking example



GPU networking example



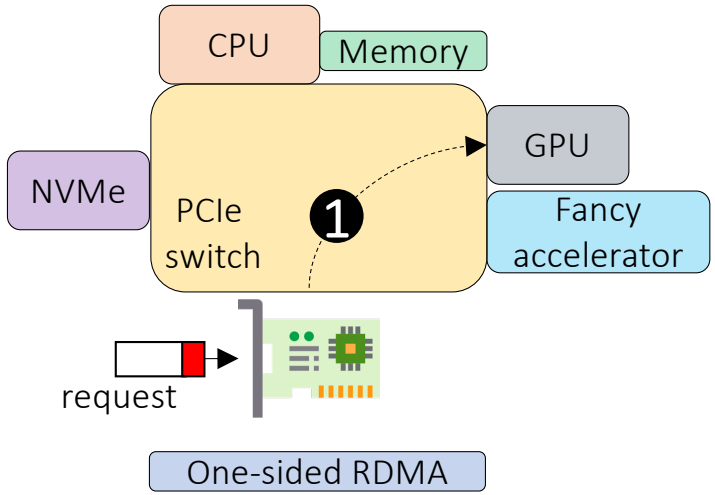
GPU networking example

NVIDIA GPUDirect

Enhancing Data Movement and Access for GPUs

Whether you are exploring mountains of data, researching scientific problems, training neural networks, or modeling financial markets, you need a computing platform with the highest data throughput. GPUs consume data much faster than CPUs and as the GPU computing horsepower increases, so does the demand for IO bandwidth.

NVIDIA GPUDirect® is a family of technologies, part of Magnum IO, that enhances data movement and access for NVIDIA data center GPUs. Using GPUDirect, network adapters and storage drives can directly read and write to/from GPU memory, eliminating unnecessary memory copies, decreasing CPU overheads and reducing latency, resulting in significant performance improvements. These technologies - including GPUDirect Storage, GPUDirect Remote Direct Memory Access (RDMA), GPUDirect Peer to Peer (P2P) and GPUDirect Video - are presented through a comprehensive set of APIs.



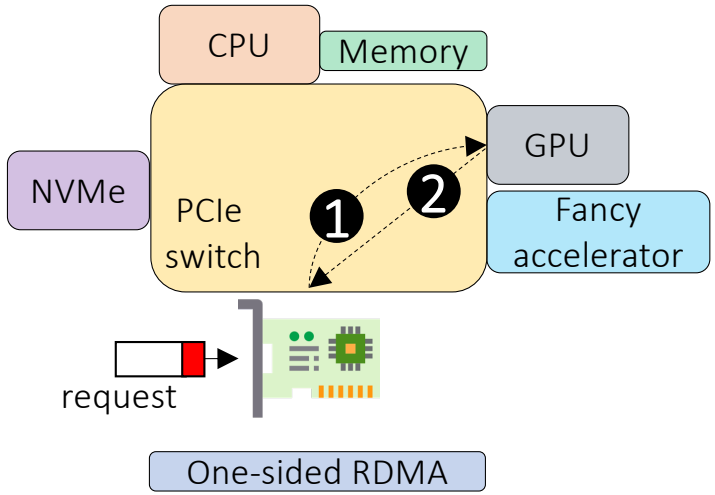
GPU networking example

NVIDIA GPUDirect

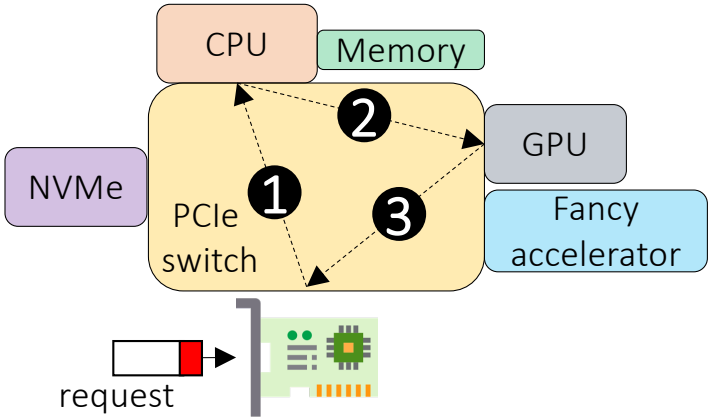
Enhancing Data Movement and Access for GPUs

Whether you are exploring mountains of data, researching scientific problems, training neural networks, or modeling financial markets, you need a computing platform with the highest data throughput. GPUs consume data much faster than CPUs and as the GPU computing horsepower increases, so does the demand for IO bandwidth.

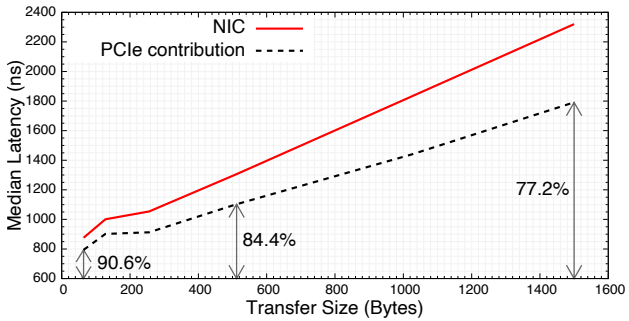
NVIDIA GPUDirect® is a family of technologies, part of Magnum IO, that enhances data movement and access for NVIDIA data center GPUs. Using GPUDirect, network adapters and storage drives can directly read and write to/from GPU memory, eliminating unnecessary memory copies, decreasing CPU overheads and reducing latency, resulting in significant performance improvements. These technologies - including GPUDirect Storage, GPUDirect Remote Direct Memory Access (RDMA), GPUDirect Peer to Peer (P2P) and GPUDirect Video - are presented through a comprehensive set of APIs.

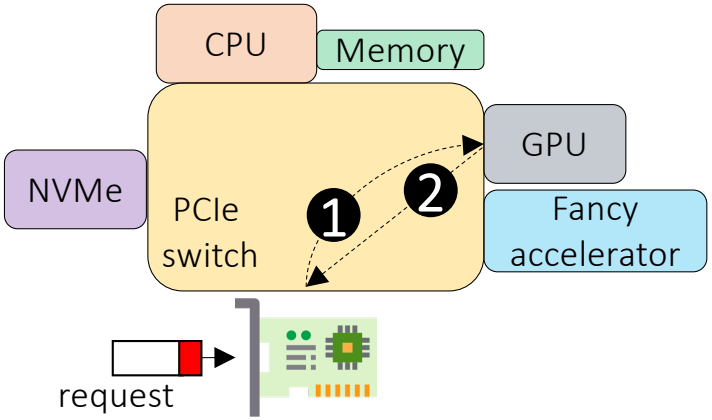


GPU networking example



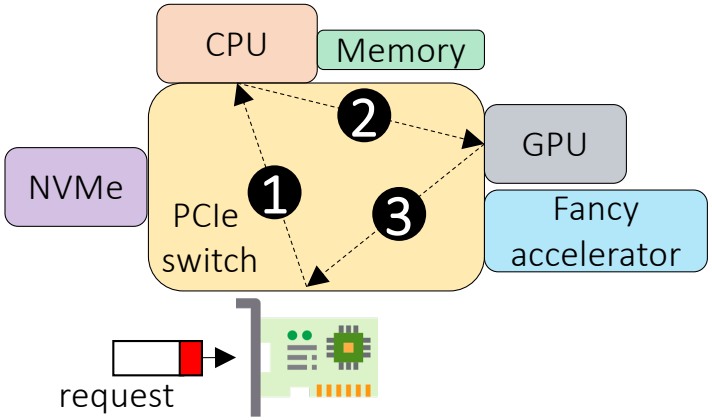
NO RDMA



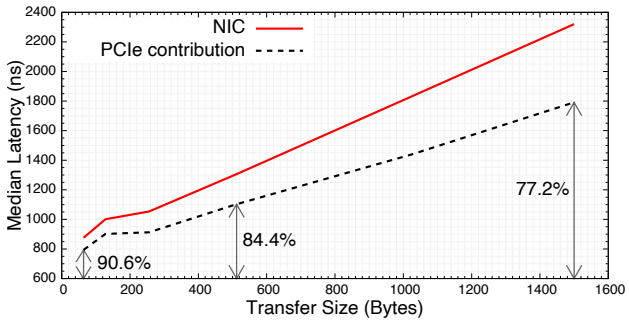


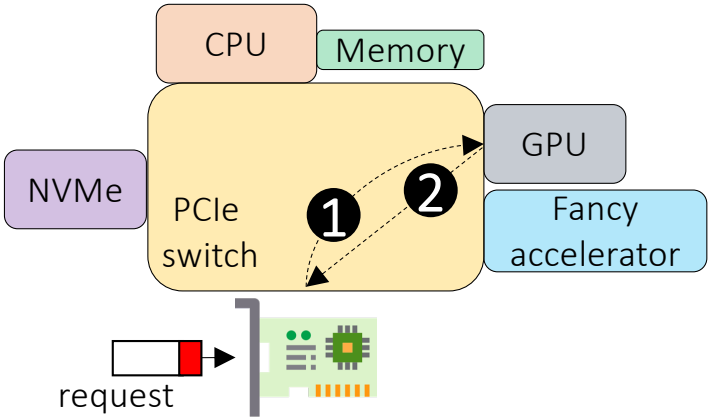
One-sided RDMA

GPU networking example



NO RDMA

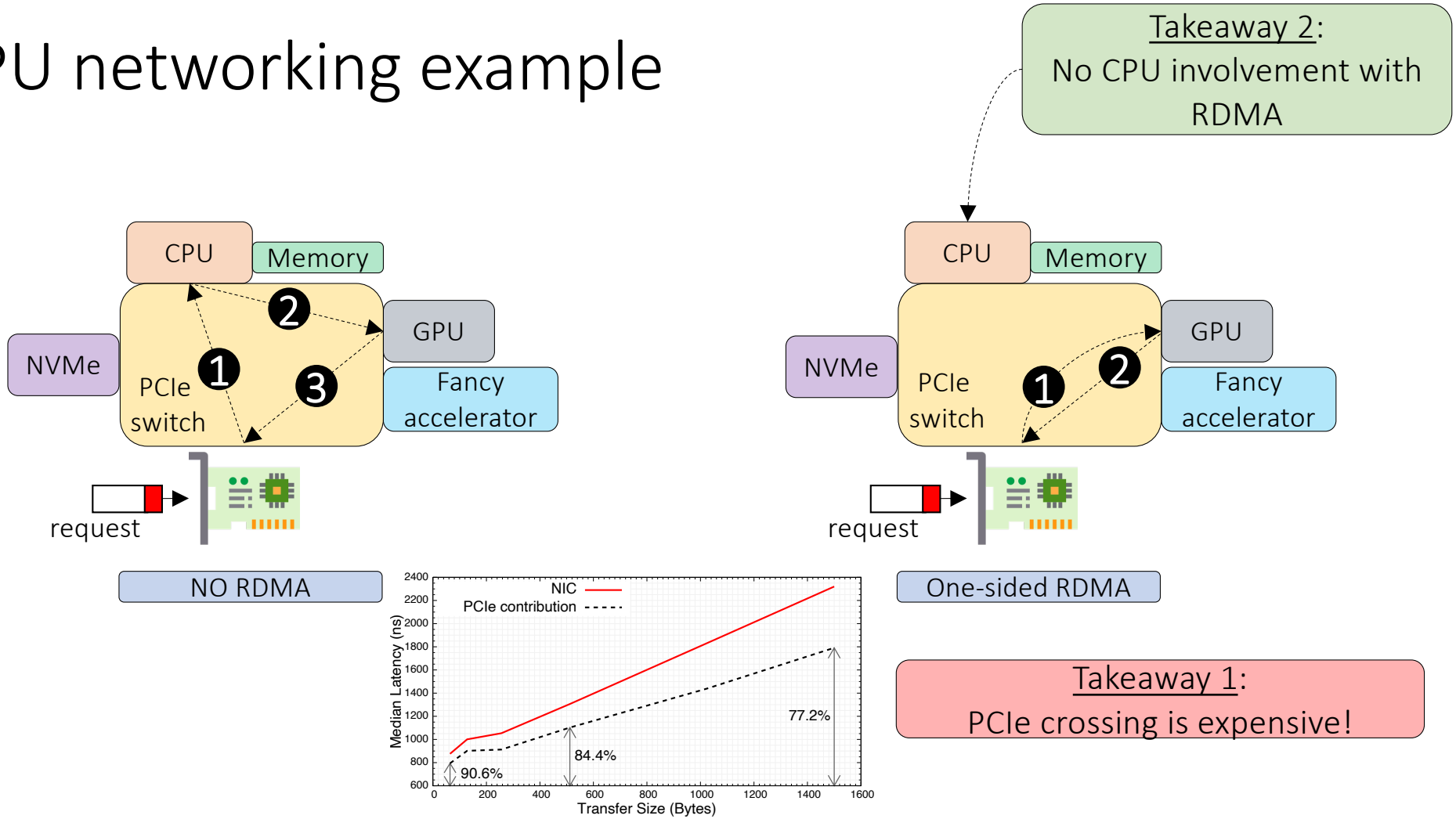




One-sided RDMA

Takeaway 1:
PCIe crossing is expensive!

GPU networking example

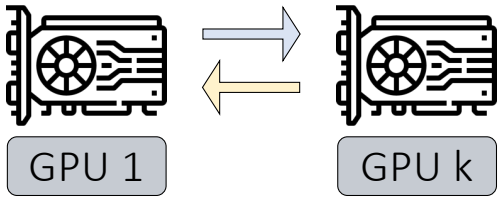


Benefits of using RDMA

- High throughput
- Low end-to-end latencies
- Low CPU utilization
  - some RDMA operations do not involve the remote CPU at all
- Low memory bus contention
  - no data is copied between the user and kernel space

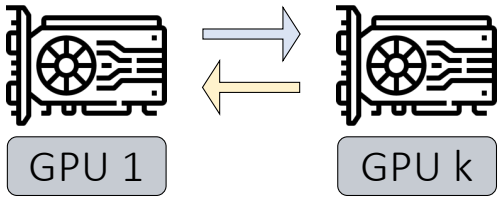
RDMA is only a part of the equation

- ML training optimizes a single shared objective, requiring all replicas to remain consistent
- Each iteration depends on the previous one, creating a tightly coupled process
- Computation proceeds synchronously across GPUs, with all replicas starting and finishing each step together
- This results in frequent global synchronization points



RDMA is only a part of the equation

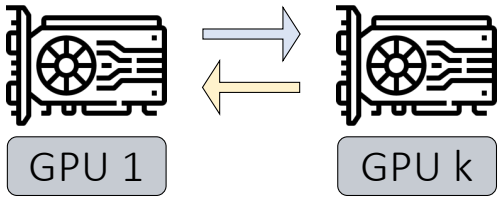
- ML training optimizes a single shared objective, requiring all replicas to remain consistent
- Each iteration depends on the previous one, creating a tightly coupled process
- Computation proceeds synchronously across GPUs, with all replicas starting and finishing each step together
- This results in frequent global synchronization points



Really bad for the network!

RDMA is only a part of the equation

- ML training optimizes a single shared objective, requiring all replicas to remain consistent
- Each iteration depends on the previous one, creating a tightly coupled process
- Computation proceeds synchronously across GPUs, with all replicas starting and finishing each step together
- This results in frequent global synchronization points



Really bad for the network!

If one node is slow, everyone needs to wait!

RDMA over Ethernet for Distributed AI Training at Meta Scale

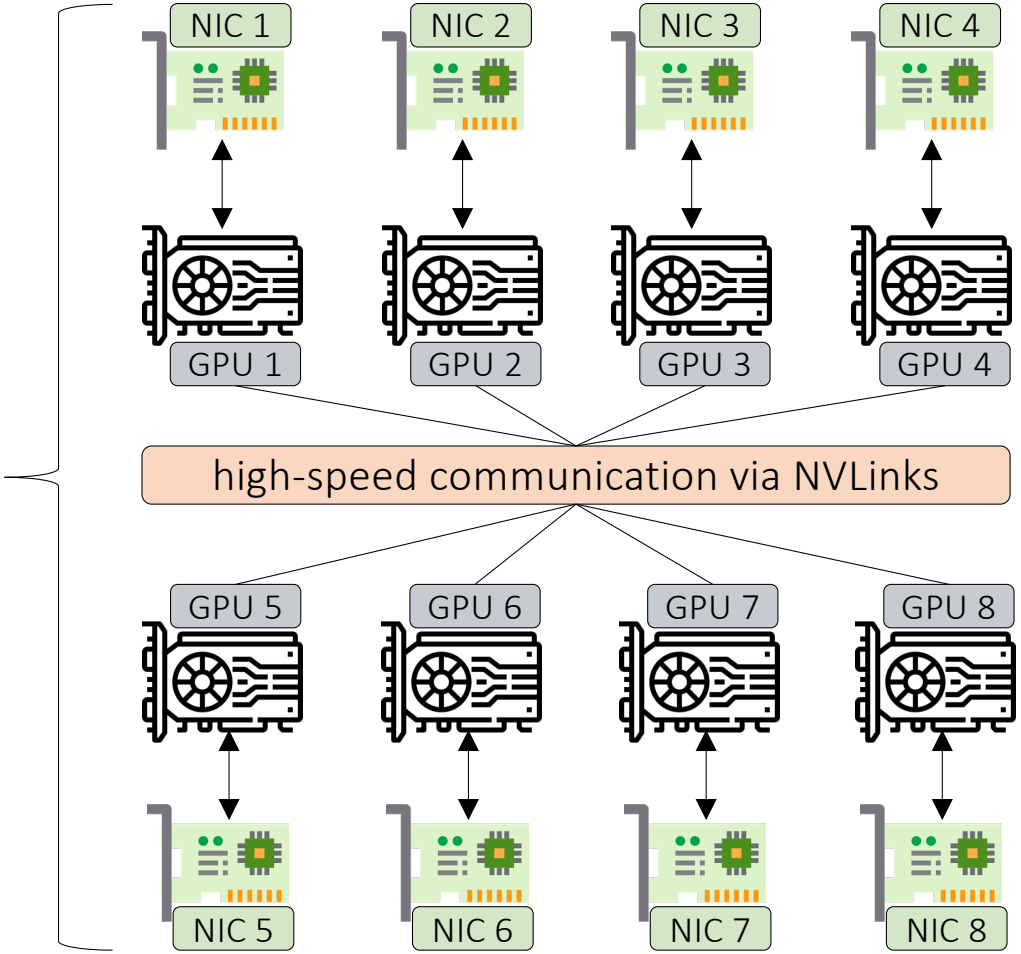
RDMA over Ethernet for Distributed AI Training at Meta Scale

Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, Hongyi Zeng
Meta

- Design, implementation and operation of Meta's RDMA for distributed AI training
- Work presented at ACM SIGCOMM 2024 (Sydney)

Host architecture

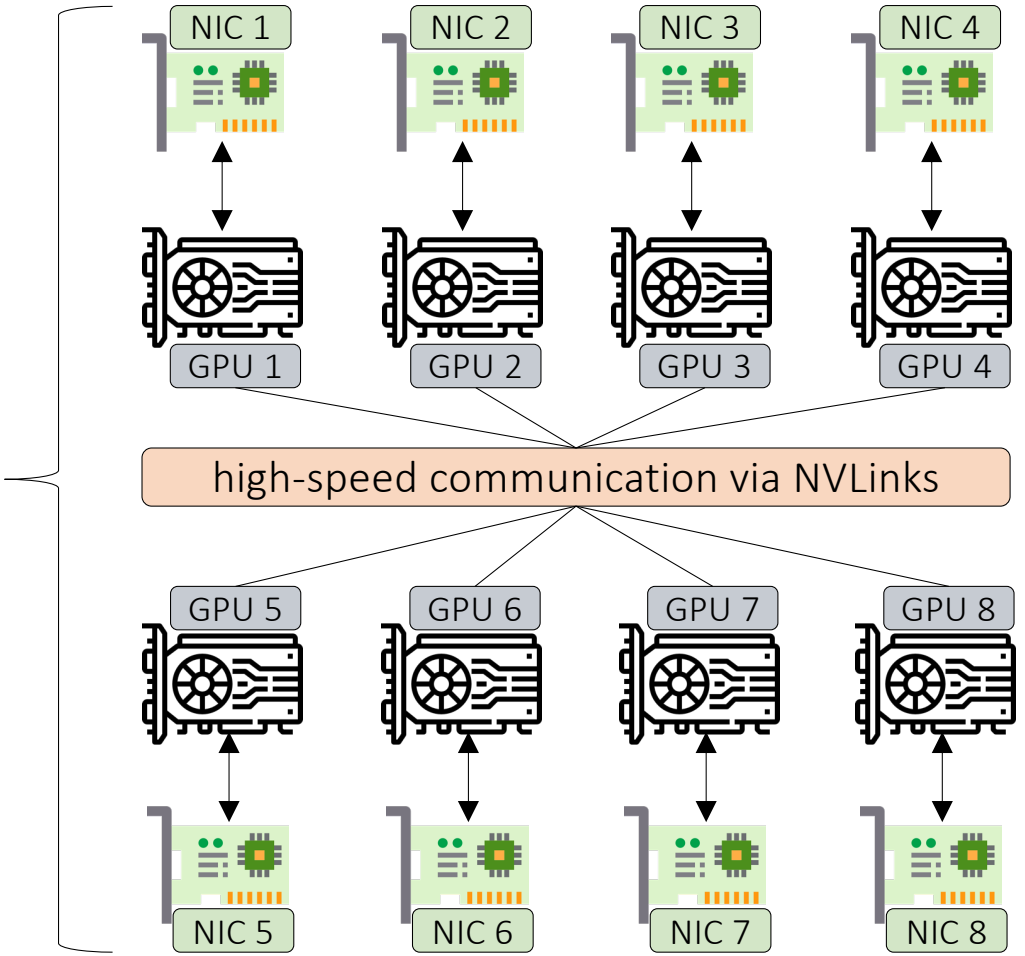
8 GPUs per server, each one coupled with a NIC



Host architecture

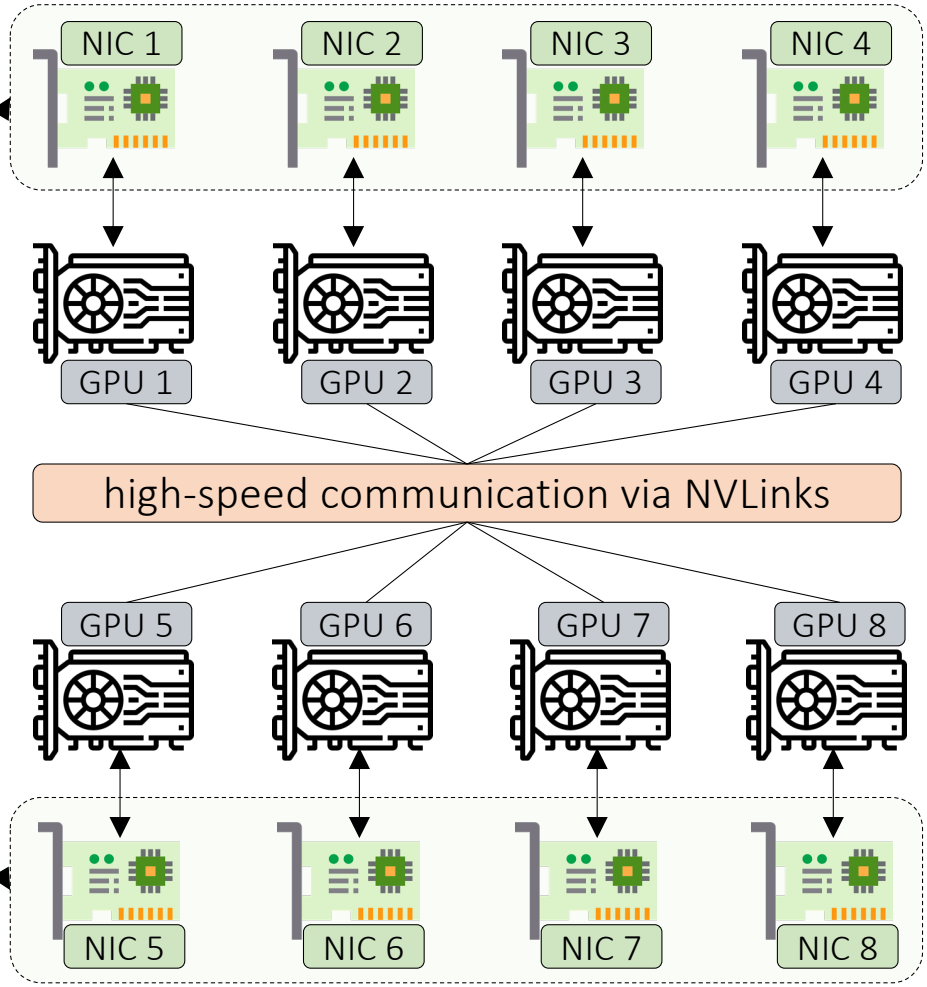
8 GPUs per server, each one coupled with a NIC

Nvidia Collective Communication library is used to provide multi-GPU communication primitives



Host architecture

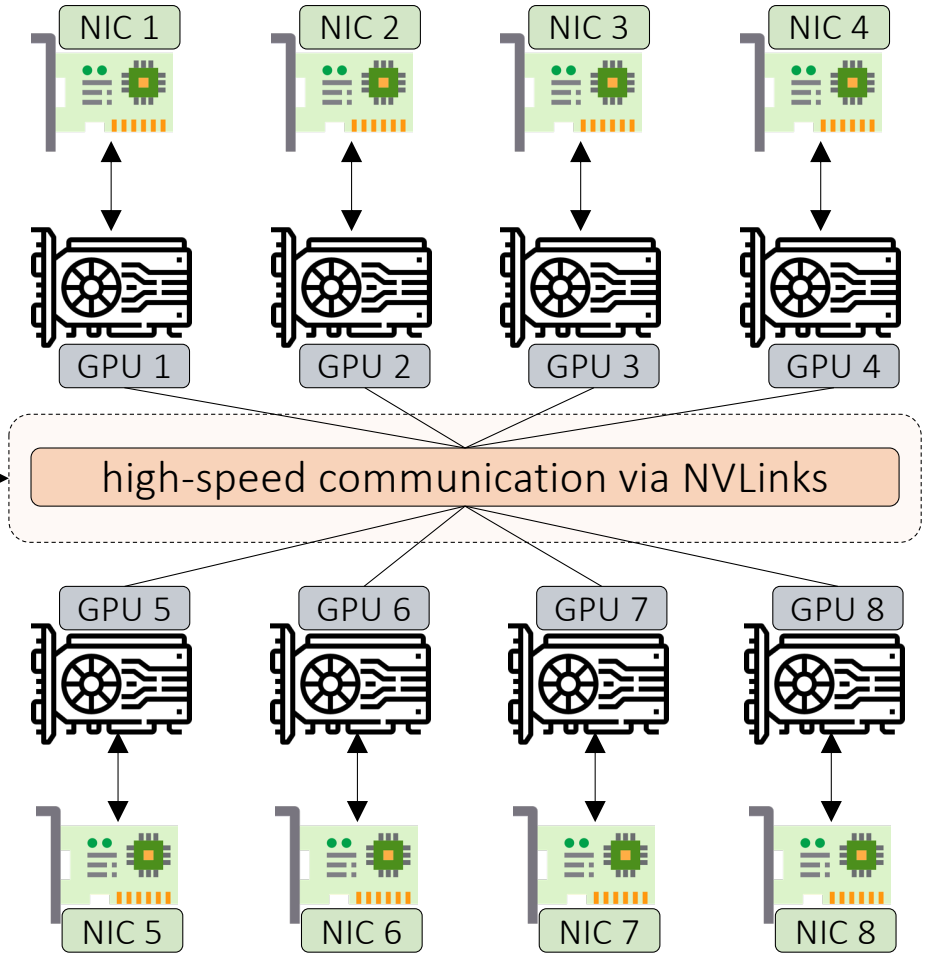
GPUs on different servers interact using their NIC with RDMA via the network fabric



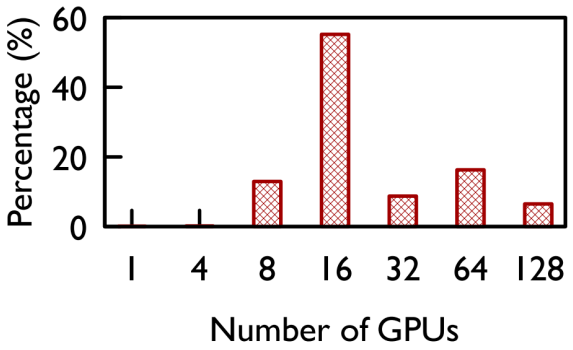
Host architecture

GPUs on the same server interact via NVLinks

NVLink is a high-speed interconnect technology developed by NVIDIA that allows multiple GPUs to communicate with each other much faster than traditional PCIe connections

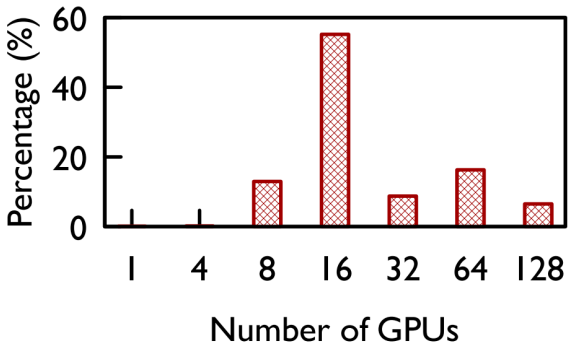


Training workload



- Job size distribution at Meta
- LLM jobs not considered (more than 128 GPUs required)
- Partial host utilization is not recommended

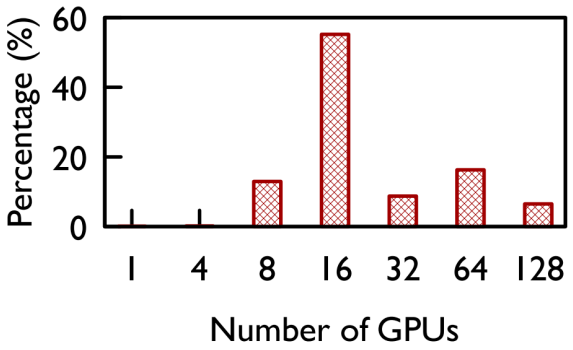
Training workload



- Job size distribution at Meta
- LLM jobs not considered (more than 128 GPUs required)
- Partial host utilization is not recommended

The majority of job sizes are multiple of 8: they need the network to communicate

Training workload



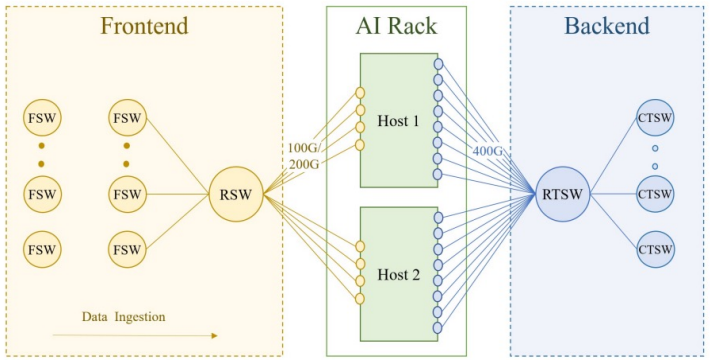
- Job size distribution at Meta
- LLM jobs not considered (more than 128 GPUs required)
- Partial host utilization is not recommended

The majority of job sizes are multiple of 8: they need the network to communicate

A variant of Llama3 was trained with 16K GPUs!

Frontend and backend networks

Input data

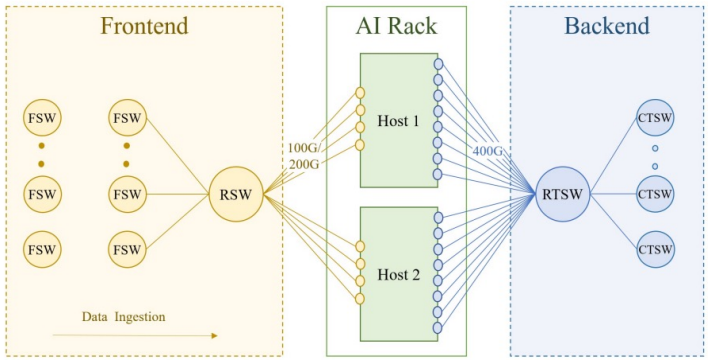


It connects all the RDMA NICs so that any two GPUs in the cluster can talk regardless their physical location

FSW: Fabric Switches
RSW: Rack Switches
RTSW: Rack Training Switches
CTSW: Cluster Training Switches

Frontend and backend networks

Input data

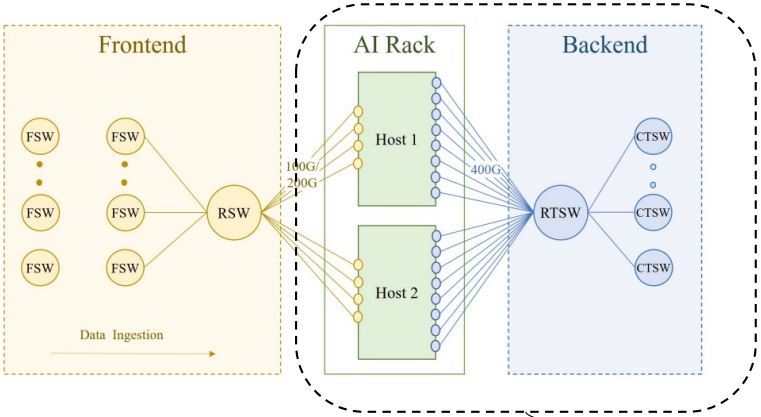


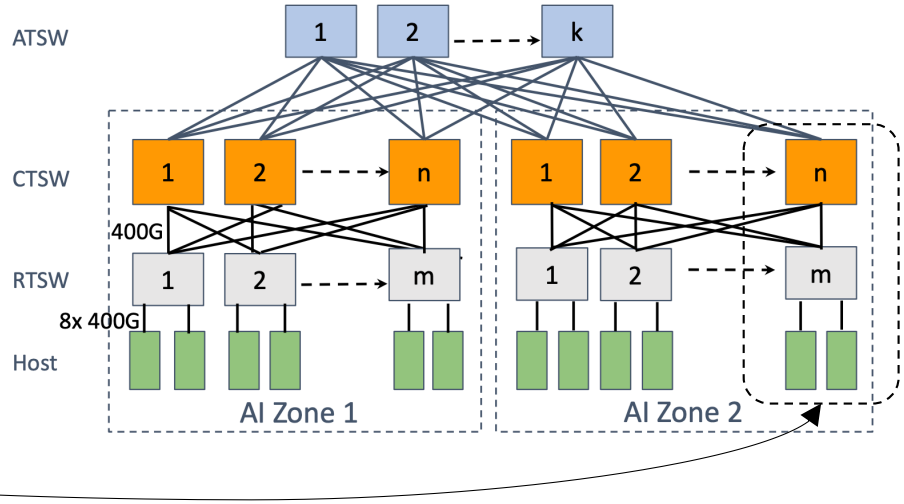
It connects all the RDMA NICs so that any two GPUs in the cluster can talk regardless their physical location

FSW: Fabric Switches
RSW: Rack Switches
RTSW: Rack Training Switches
CTSW: Cluster Training Switches

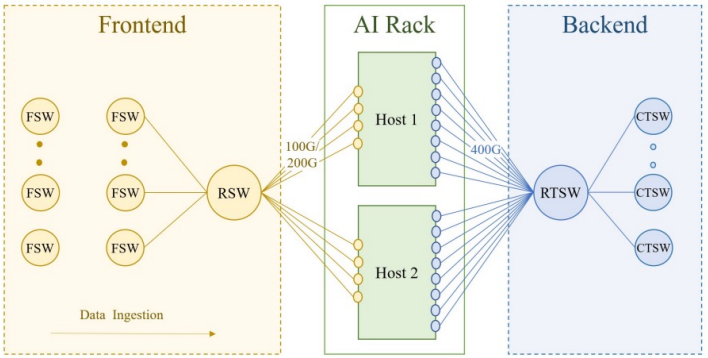
Two networks that can evolve independently as the traffic carried by them have different and conflicting requirements. Physical separation ensured the ideal network environment for latency-sensitive RoCE operations

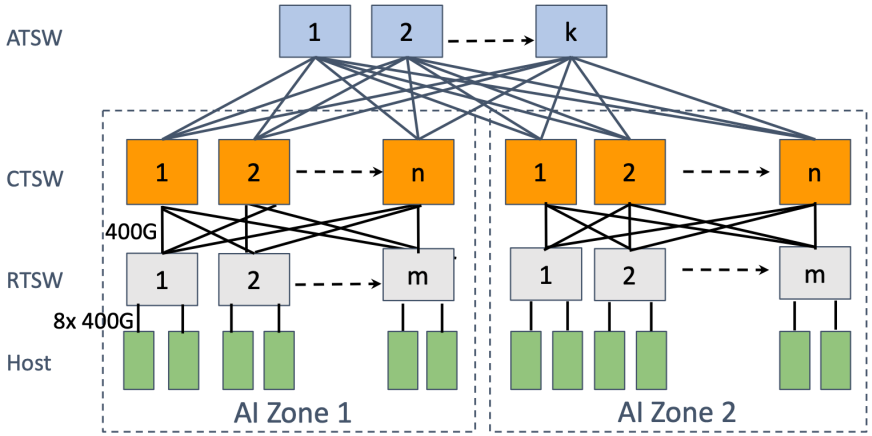
Backend network





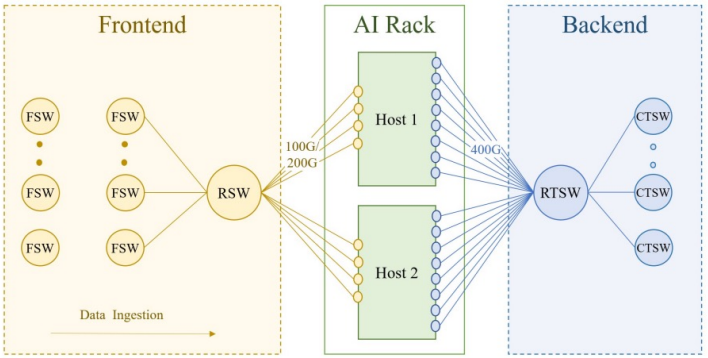
Backend network

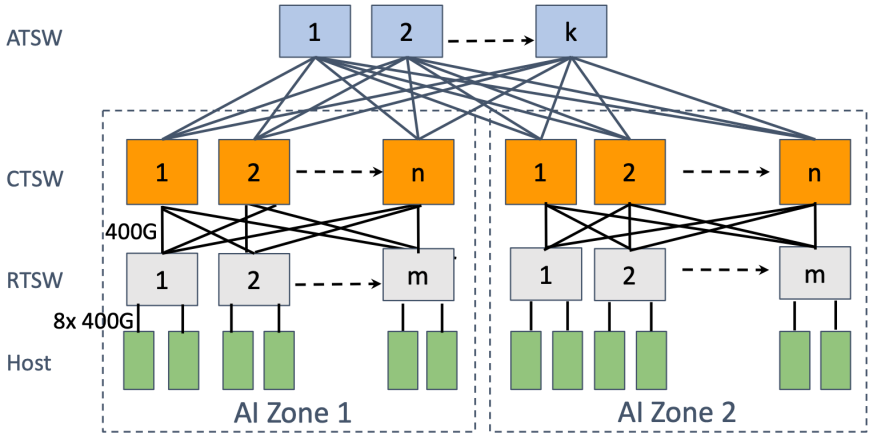




How to efficiently balance and route the training traffic?

Routing

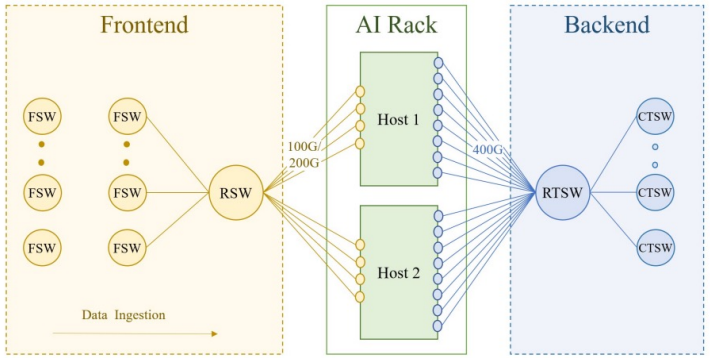


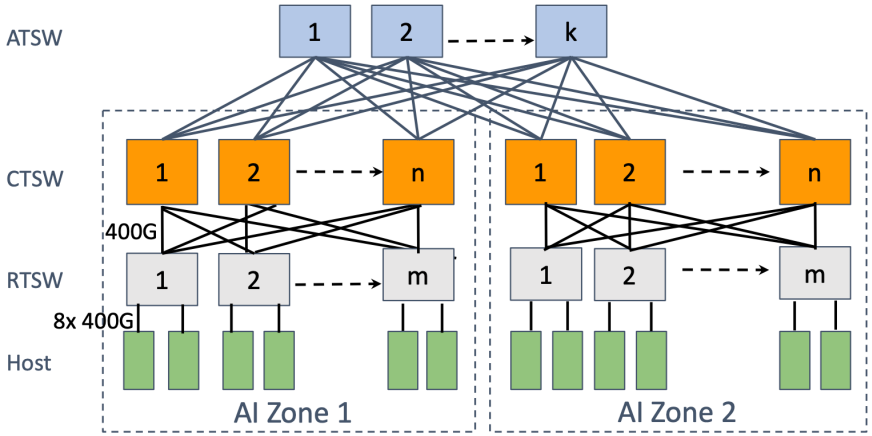


Challenges:

Low entropy: the number and the diversity of flows in AI are much smaller than traditional workloads and the flow patterns are usually repetitive and predictable

Routing



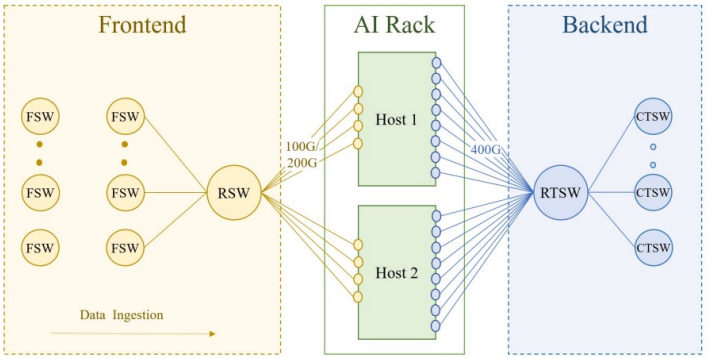


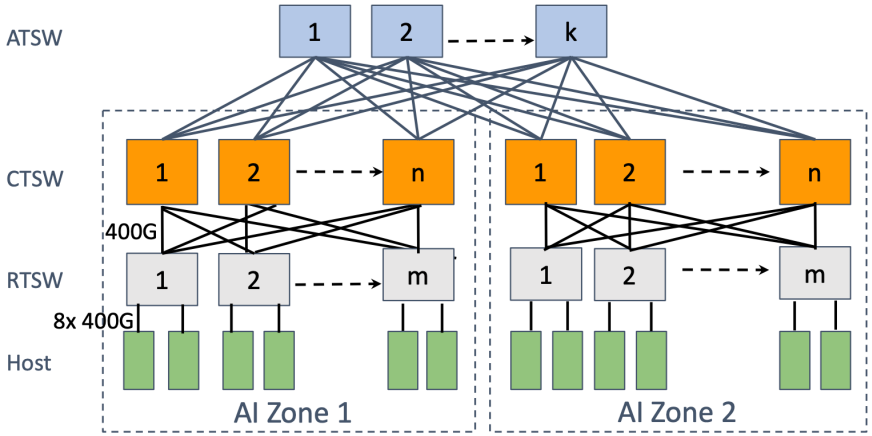
Challenges:

Low entropy: the number and the diversity of flows in AI are much smaller than traditional workloads and the flow patterns are usually repetitive and predictable

Burstiness: flows usually exhibit the “on and off” nature in the time granularity of milliseconds.

Routing





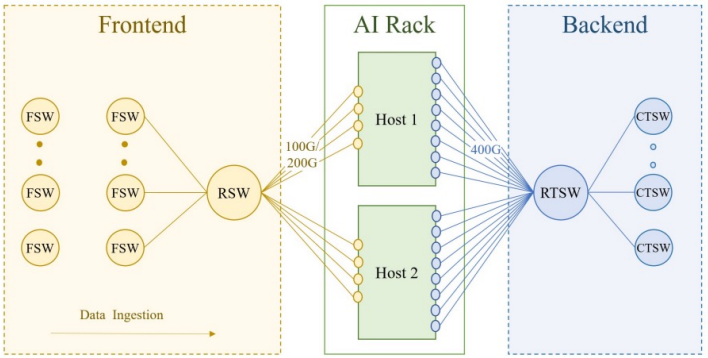
Challenges:

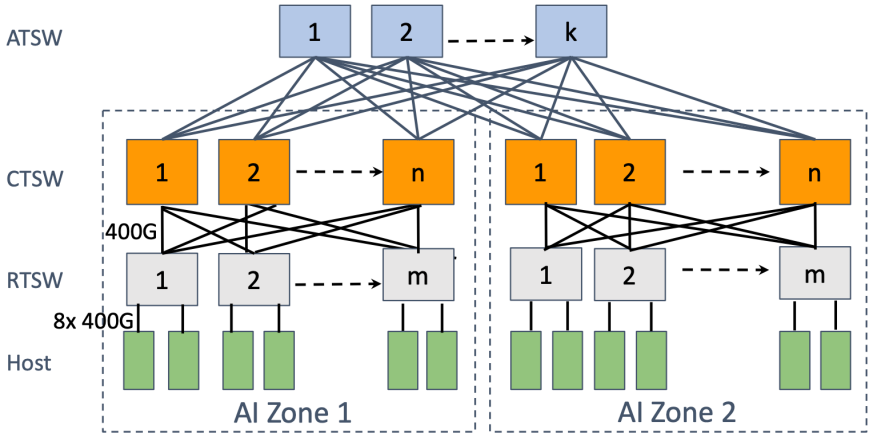
Low entropy: the number and the diversity of flows in AI are much smaller than traditional workloads and the flow patterns are usually repetitive and predictable

Burstiness: flows usually exhibit the “on and off” nature in the time granularity of milliseconds.

Elephant flows: For each burst, the intensity of each flow could reach up to the line rate of NICs.

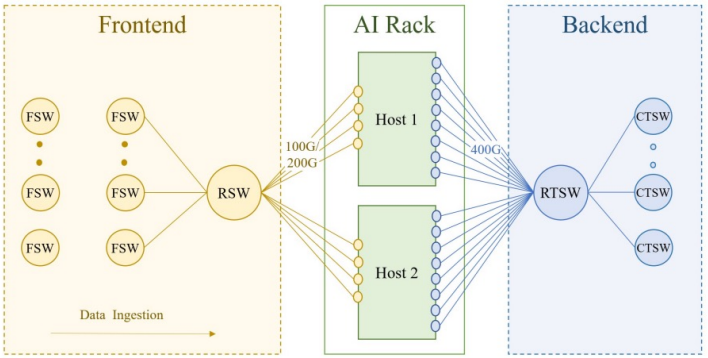
Routing

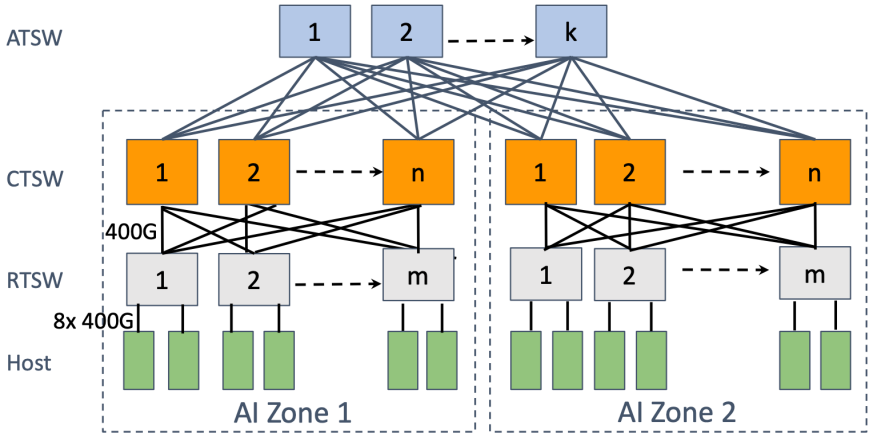




ECMP?

Routing

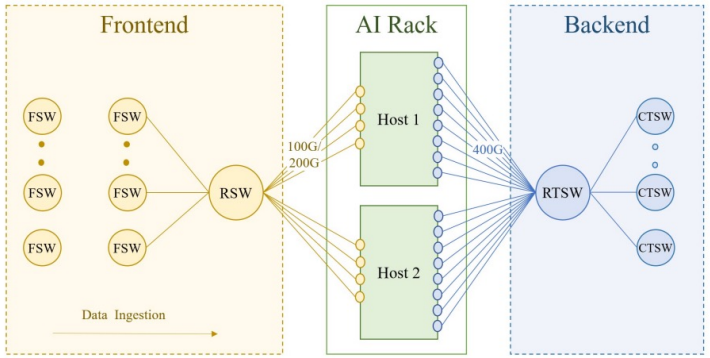


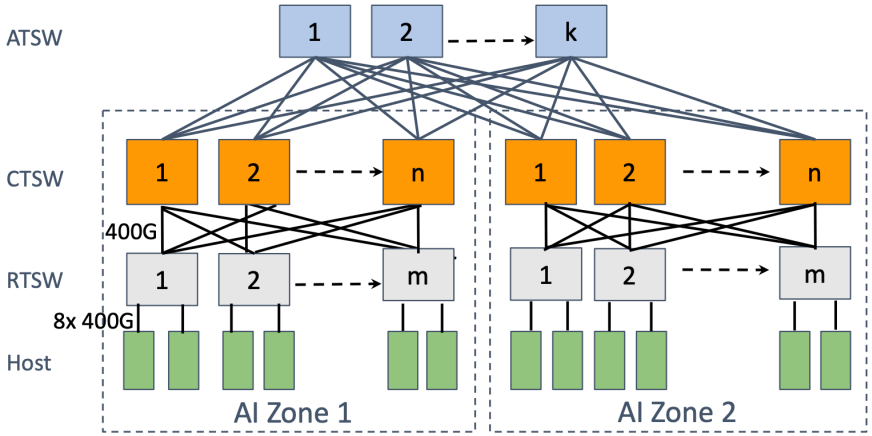


ECMP?

Poor performance because of the low flow entropy ☹

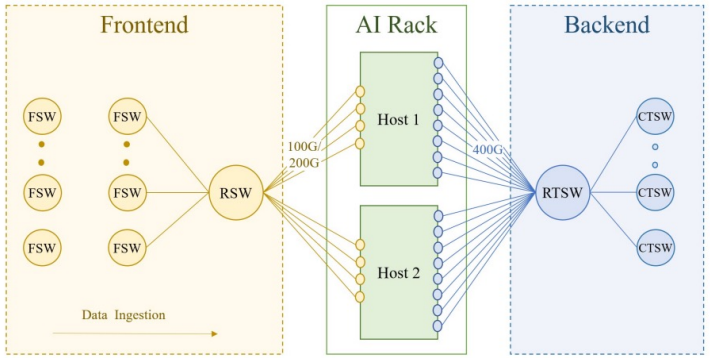
Routing

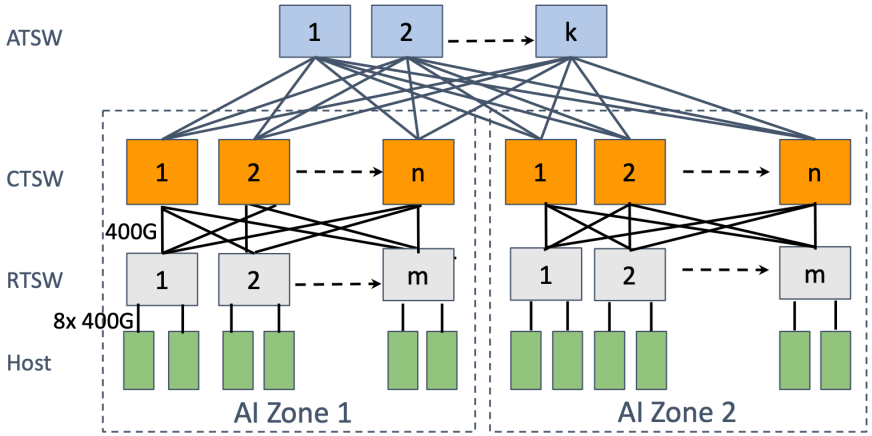




Static Routing using RTSW index + ECMP in case of failures?

Routing





Static Routing using RTSW index + ECMP in case of failures?

Racks can be partially allocated to a job, with only one of the two hosts using the uplink bandwidth. This cause uneven traffic distribution degrading the training performance up to more than 30% ☹

Failures caused the affected flows to be unevenly reassigned by ECMP ☹

Routing

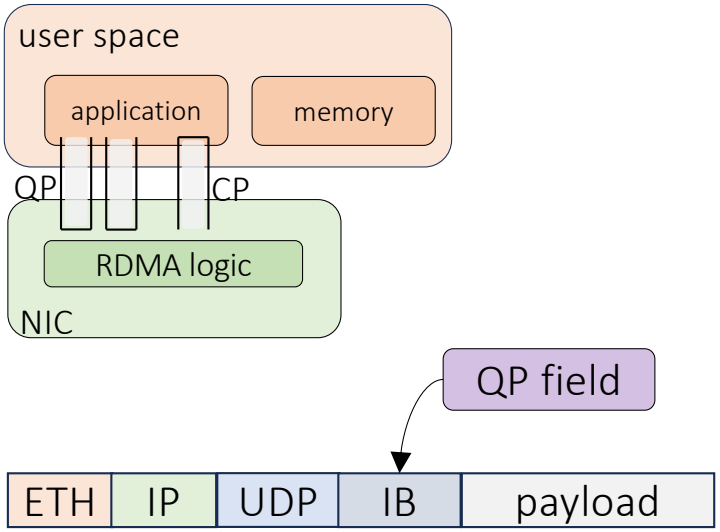
Enhanced ECMP!

Configure the switches to additionally hash on the destination QP field to increase entropy

Routing

Enhanced ECMP!

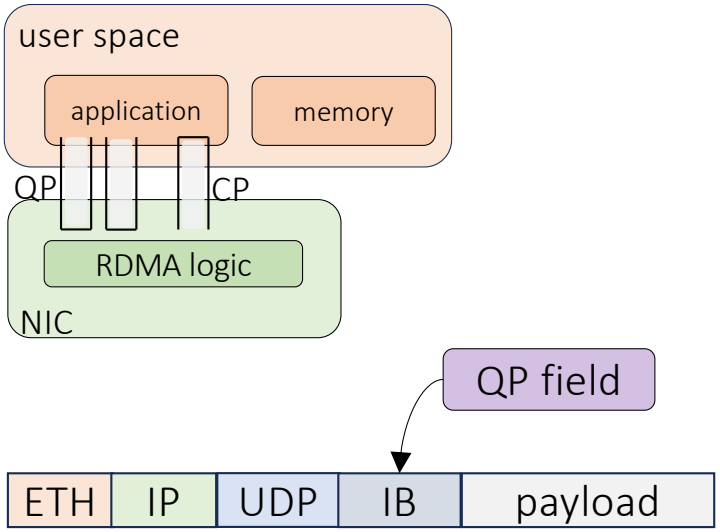
Configure the switches to additionally hash on the destination QP field to increase entropy

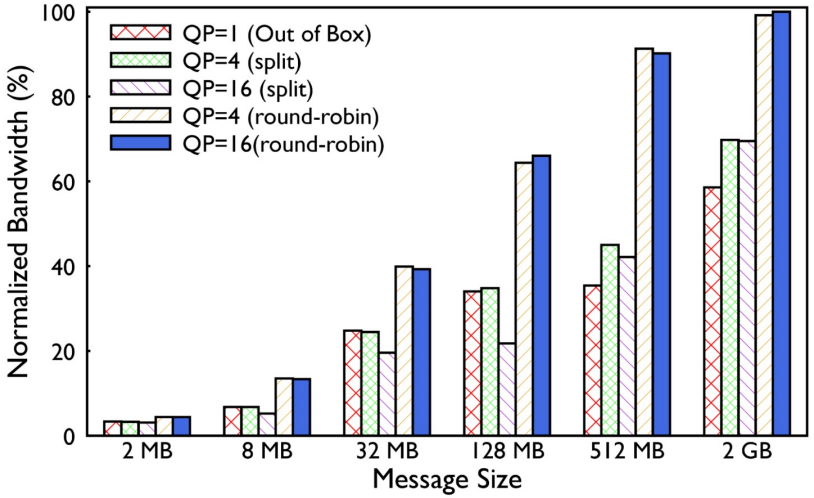


Routing

Enhanced ECMP!

Configure the switches to additionally hash on the destination QP field to increase entropy



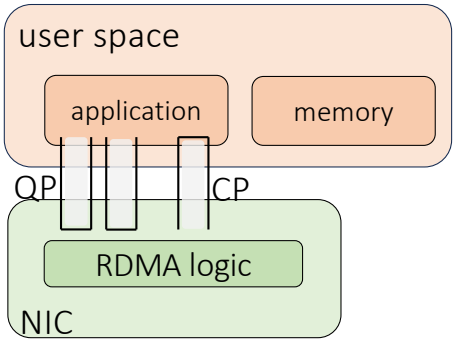


Split: split a message over multiple QPs
Round Robin: posting messages towards the same GPU over multiple QPs

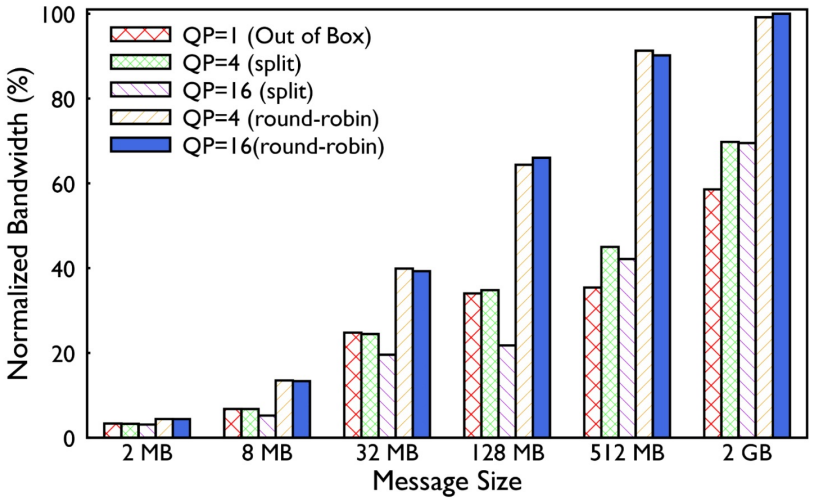
Routing

Enhanced ECMP!

Configure the switches to additionally hash on the destination QP field to increase entropy



More in the paper!



QP field

Split: split a message over multiple QPs
Round Robin: posting messages towards the same GPU over multiple QPs

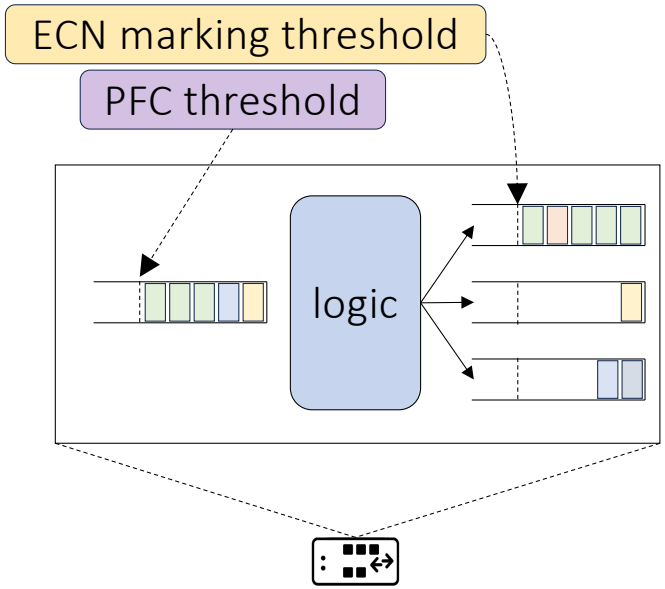
Congestion Control

- Initially deployed DCQCN alongside PFC
- Hard to find the right ECN threshold to minimize PFC traffic

Congestion Control

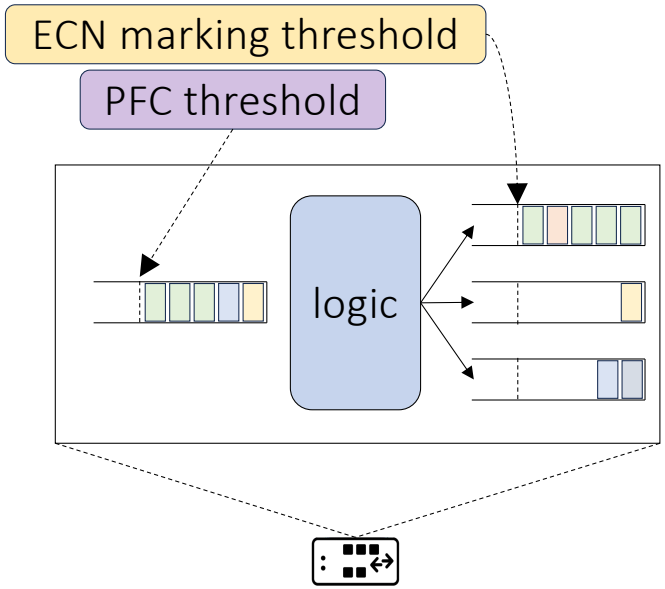
- Initially deployed DCQCN alongside PFC
- Hard to find the right ECN threshold to minimize PFC traffic

A tight ECN threshold helps avoiding PFC frames



Congestion Control

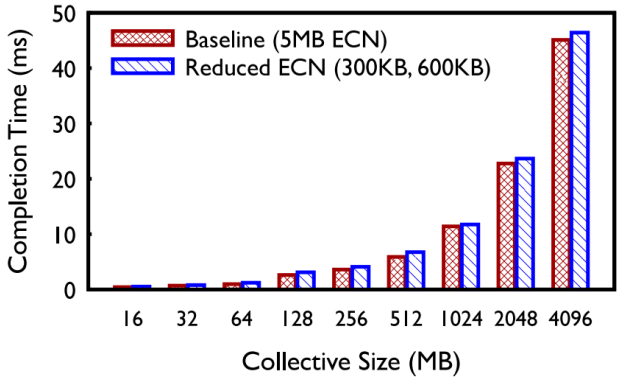
- Initially deployed DCQCN alongside PFC
- Hard to find the right ECN threshold to minimize PFC traffic



A tight ECN threshold helps avoiding PFC frames

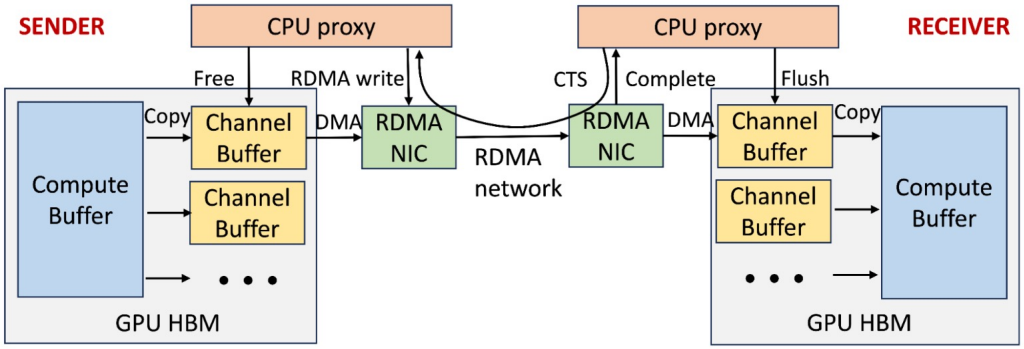
Allreduce completion time on 32 GPUs comparison with CTSW ECN threshold changes

Allreduce is a collective communication task that aggregates data from multiple processes and makes the combined result available to all participating processes



Congestion Control

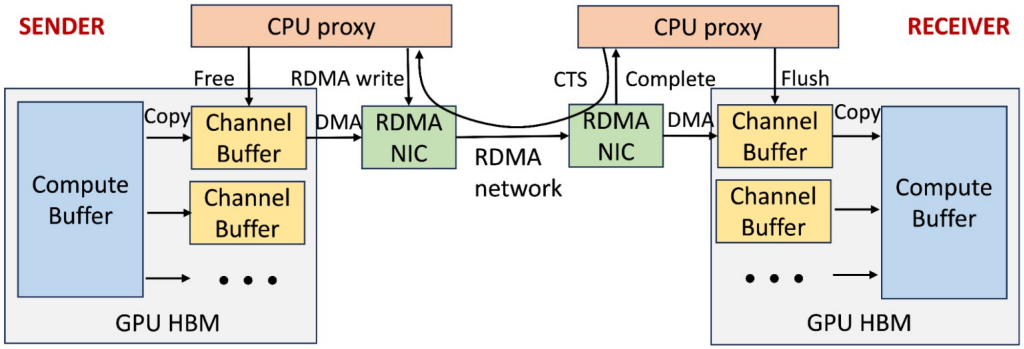
…eventually disable DCQCN and just use PFC with a trick ☺



Congestion Control

…eventually disable DCQCN and just use PFC with a trick ☺

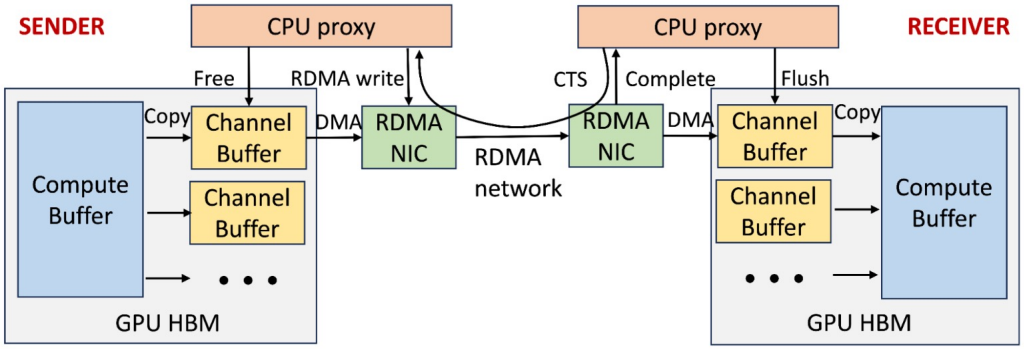
1. The sender GPU copies data from the compute buffer to the channel buffer
2. The sender CPU posts an RDMA write request after receiving a clear-to-send (CTS) packet from the receiver
3. The receiver's GPU copies the channel buffer content to the compute buffer



Congestion Control

…eventually disable DCQCN and just use PFC with a trick ☺

1. The sender GPU copies data from the compute buffer to the channel buffer
2. The sender CPU posts an RDMA write request after receiving a clear-to-send (CTS) packet from the receiver
3. The receiver's GPU copies the channel buffer content to the compute buffer



Idea: leverage this mechanism as a receiver-driven traffic admission to limit the amount of in-flight traffic on the network, especially when congestion starts to build up

More in the paper!

RDMA is not only for AI

Google Cloud

Networking > Virtual Private Cloud > Guides

Was this helpful?

RDMA network profiles

Send feedback

This page provides an overview of Remote Direct Memory Access (RDMA) network profiles in Google Cloud.

Overview

RDMA network profiles let you create Virtual Private Cloud (VPC) networks that provide low-latency, high-bandwidth RDMA communication between the memory or GPUs of VMs that are created in the network.

aws

Nitro v6

- Traffic Mirroring is not supported.
- Up to 400 Gbps* per network card.
- Remote direct memory access (RDMA) read and RDMA write are available with EFA for the following instance type: `p6-b200.48xlarge`.

Alibaba Cloud

eRDMA

Updated at: 2025-08-01 06:45

Product Community

Compared with traditional Remote Direct Memory Access (RDMA), elastic RDMA (eRDMA) can be used in a wide range of scenarios, such as Redis-based cache databases, Spark-based big data analytics, Weather Research and Forecasting Model (WRF) in high performance computing (HPC), and AI training. You can use eRDMA to deploy HPC applications in the cloud to build high-performance application clusters that have high elasticity at low costs. You can also replace a VPC with an eRDMA network to accelerate applications.

Fun read

RDMA is Turing complete, we just did not know it yet!

Waleed Reda
Université catholique de Louvain
KTH Royal Institute of Technology

Marco Canini
KAUST

Dejan Kostić
KTH Royal Institute of Technology

Simon Peter
University of Washington

This paper show how it is possible to implement complex RDMA offloads, lifting the existing RDMA verbs interface to a Turing complete set of programming abstractions!